Detector-Independent Multi-Person Pose Grouping with Skeleton-Aware Graph Attention Embeddings

by

Dean Spencer Mills

Submitted in partial fulfillment of the requirements for the degree

Master of Engineering (Computer Engineering)

in the

Department of Electrical, Electronic and Computer Engineering

Faculty of Engineering, Built Environment and Information Technology

UNIVERSITY OF PRETORIA

August 2026

SUMMARY

Detector-Independent Multi-Person Pose Grouping with Skeleton-Aware Graph Attention Embeddings

by

Dean Spencer Mills

Supervisor:	Mr H. Grobler
Department:	Electrical, Electronic and Computer Engineering
University:	University of Pretoria
Degree:	Master of Engineering (Computer Engineering)
Keywords:	Autonomous person-count estimation, Bayesian hyperparameter sweep, bottom-up pose estimation, COCO, contrastive learning, detector independence, GATv2, graph attention network, hyperbolic embedding, multi-person pose grouping, skeleton-aware embedding, synthetic-to-real transfer.

Bottom-up multi-person human pose estimation can be separated into two stages: detecting keypoints in an image and assigning those keypoints to individual people. Most established grouping approaches are trained jointly with a particular detector, which makes the grouping mechanism dependent on that detector's internal representation. This work investigates an alternative in which grouping is treated as a separate module. A graph neural network - PG-GAT (Pose Grouping Graph Attention Network) - is developed to operate directly on detected keypoint coordinates and produce per-keypoint embeddings from which person identities are recovered using *k*-means. The resulting grouping network does not require access to the detector's feature maps and can therefore be applied to keypoints produced by different detector architectures.

PG-GAT extends GATv2 (graph attention network, version 2) with three modifications derived from the human skeleton structure: a type-pair attention bias that distinguishes different categories of keypoint edges, skeleton-relative position encoding that exposes geometric relationships to the attention mech-

anism, and same-type repulsion heads that provide dedicated capacity for separating corresponding joints belonging to different people. In isolation the individual modifications give margins within seed noise, while their combination performs best under matched architecture and training conditions. The final architecture is selected through a 32-run Bayesian sweep over six architectural parameters, and a separate 24-run loss sweep confirms that the inherited contrastive-loss settings are already near-optimal. Training consists of synthetic pre-training on 400 procedurally generated scenes followed by COCO fine-tuning, with the fine-tuning configuration selected through a further 12-run Bayesian sweep.

The resulting model reaches 0.9715 pose grouping accuracy (PGA) on COCO validation using ground-truth keypoints and 0.9591 end-to-end PGA using HigherHRNet-W32-512 detections. HigherHRNet's native associative-embedding grouping reaches 0.9847 on the same matched detections, giving a measured detector-independence cost of 0.0256 PGA. The same frozen PG-GAT and person-count estimator are then applied zero-shot to YOLO11x-pose and RTMO-L detections. The autonomous pipeline trails their native grouping by 0.0210 and 0.0220 PGA respectively, showing that the cost remains approximately 0.02 across three different detector families. A small person-count (K) estimation head trained on the frozen PG-GAT embeddings achieves 76.1% exact and 91.9% off-by-one person-count accuracy on COCO validation detections. Using its predicted K raises end-to-end PGA from 0.9591 with ground-truth K to 0.9644. Further experiments show that this behaviour is associated with detection incompleteness: the predicted count adapts to the people represented by the detected keypoints, whereas the annotation-level count can require k-means to form clusters for people who are poorly represented or absent from the detection set.

The investigation also establishes several negative results that help identify where additional model complexity is useful. Three learned grouping heads (deep embedded clustering, slot attention, and graph partitioning) and ten SA-DMoN variants fail to improve on simple k-means once synthetic-to-real transfer is considered. In contrast, changes to the embedding network produce substantially larger gains, supporting the conclusion that embedding quality is the main performance lever in the grouping problem studied here. A hyperbolic PG-GAT matches a four-fold larger Euclidean embedding under synthetic-only training but does not retain this advantage after COCO fine-tuning. TriGAT - which replaces joints with skeleton triplets - underperforms the joint-based representation, with its triplet-to-joint voting stage identified as the main limitation. Finally, augmenting PG-GAT with MobileNetV2 appearance features does not improve on the position-and-type model despite adding approximately 2.2×10^6 parameters.

Overall, the results show that detector-independent pose grouping is practical with a measured accuracy trade-off of approximately 0.02 PGA against detector-native grouping across the three detector families tested. They also show that the remaining ground-truth person-count dependency can be removed without reducing operational grouping accuracy under the evaluated detector conditions. Across the wider set of experiments, the consistent finding is that improvements to the per-keypoint embedding have a substantially greater effect on grouping quality than increasing the complexity of the grouping algorithm itself.

LIST OF ABBREVIATIONS

2s-AGCN	two-stream adaptive graph convolutional network
AdamW	Adam optimiser with decoupled weight decay
AE	associative embedding
AI	artificial intelligence
AP	average precision
CE	cross-entropy
CNN	convolutional neural network
COCO	Common Objects in Context (dataset)
DEC	deep embedded clustering
DEKR	disentangled keypoint regression
DETR	detection transformer
DMoN	deep modularity network
DPT	dense prediction transformer
DSNT	differentiable spatial-to-numerical transform
E2E	end-to-end
ED-Pose	explicit box detection for end-to-end pose estimation
GAST-Net	graph attention spatio-temporal network
GAT	graph attention network
GATv2	graph attention network, version 2
GCN	graph convolutional network
GEC	graph edge classification
GELU	Gaussian error linear unit
GNN	graph neural network
GPU	graphics processing unit
GraphSAGE	graph sample and aggregate
GRU	gated recurrent unit
GT	ground truth
GUID	globally unique identifier
HGG	hierarchical graph grouping
HigherHRNet	higher-resolution network (bottom-up HRNet variant)

HOPE-Net	hand-object pose estimation network
HPE	human pose estimation
HRFormer	high-resolution transformer
HRNet	high-resolution network
IEEE	Institute of Electrical and Electronics Engineers
IF	impact factor
InfoNCE	information noise-contrastive estimation
JSON	JavaScript Object Notation
KL	Kullback–Leibler (divergence)
kNN	*k*-nearest neighbours
LeakyReLU	leaky rectified linear unit
MiDaS	robust monocular depth estimation model
MLP	multilayer perceptron
MobileNetV2	MobileNet version 2
MPII	Max Planck Institute for Informatics (pose dataset)
NCut	normalised cut
NTU	NTU RGB+D (action-recognition dataset)
OCHuman	Occluded Human (benchmark)
OKS	object keypoint similarity
PAF	part-affinity field
PCK	percentage of correct keypoints
PCT	pose as compositional tokens
PETR	pose estimation with transformers
PG-GAT	pose grouping graph attention network
PGA	pose grouping accuracy
PyG	PyTorch Geometric
R-CNN	region-based convolutional neural network
RBF	radial basis function
ReID	re-identification
ReLU	rectified linear unit
RGB	red, green, blue
RLE	residual log-likelihood estimation
RQ	research question

RTMO	real-time multi-person one-stage pose estimation
RTMPose	real-time multi-person pose estimation
RTMW	real-time multi-person whole-body pose estimation
SA-DMoN	skeleton-aware deep modularity network
SA-GAT	skeleton-aware graph attention network (former name of PG-GAT)
SemGCN	semantic graph convolutional network
SimCC	simple coordinate classification
SimCLR	simple framework for contrastive learning of visual representations
ST-GCN	spatio-temporal graph convolutional network
TriGAT	triplet graph attention network
ViTPose	vision transformer for pose estimation
ViTPose++	vision transformer for pose estimation (multi-dataset extension)
W&B	Weights & Biases
YAML	YAML Ain't Markup Language
YOLO	you only look once

LIST OF SYMBOLS

Sets, spaces and operators

$\mathcal{G}$	keypoint graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
$\mathcal{V}$	node (vertex) set of the graph
$\mathcal{E}$	edge set of the graph
$\mathcal{N}(i)$	neighbourhood of node i
$\mathcal{X}$	set of keypoint detections
$\mathcal{P}^{+}$, $\mathcal{P}^{-}$	sets of positive (same-person) and negative pairs
$\mathbb{R}^{d}$	d-dimensional Euclidean space
$\mathbb{S}^{d-1}$	unit hypersphere of L_2-normalised embeddings
$\mathbb{H}^{d}$	d-dimensional Lorentz-model hyperbolic space
PGA	pose grouping accuracy (per-joint accuracy after Hungarian matching)
$Q(\mathbf{S})$	Newman modularity of a partition $\mathbf{S}$
$\mathrm{Tr}(\cdot)$	matrix trace
$\mathbb{1}[\cdot]$	indicator function
$\lVert\cdot\rVert_2$	Euclidean (L_2) norm
$\lVert\cdot\rVert_F$	Frobenius norm

Latin symbols

$\mathbf{a}$	learned attention vector
$\mathbf{A}$	graph adjacency matrix
$\mathbf{B}$	modularity matrix, $\mathbf{A} - \mathbf{d}\mathbf{d}^{\top}/2m$
$\mathbf{B}_{\text{pose}}$	skeleton-aware (pose) modularity matrix, $\mathbf{A} - \tilde{\mathbf{R}}$
d	embedding / feature dimension ($d = 128$ at read-out)
$\mathbf{d}$	node-degree vector
$\mathbf{D}$	degree matrix
$d_{\mathbb{H}}(\cdot,\cdot)$	Lorentz geodesic distance
e_{ij}	unnormalised attention score for edge (i, j)
$\mathbf{h}_i^{(l)}$	hidden state of node i at layer l
H	number of attention heads, $H = H_{\text{std}} + H_{\text{rep}}$
k	cluster count in k-means; neighbour count in the kNN graph
K	number of people (clusters) in an image

$\hat{K}$	predicted number of people (K-estimation head)
L	number of GNN layers (L-hop receptive field)
$\mathcal{L}_{(\cdot)}$	training loss; subscript names the objective (contrastive, DEC, DMoN, $\mathbb{H}$)
m	total edge count of the graph
N	number of nodes / keypoint detections
$\mathbf{p}_i$	normalised image-plane position of detection i
p_{ij}	DEC target assignment distribution
P_{ij}	Gaussian spatial-proximity kernel between nodes i and j
q_{ij}	DEC soft assignment of node i to cluster j
$\tilde{\mathbf{R}}$	skeleton-aware null-model matrix
$\mathbf{S}$	soft cluster-assignment matrix ($N \times K$)
t_i	joint (keypoint) type of detection i
$\mathbf{T}$	17×17 skeleton type-affinity matrix
v_i	visibility flag of detection i
$\mathbf{W}^{(l)}$	learnable weight matrix at layer l
$\mathbf{x}_i$	input feature vector of detection i
$\hat{\mathbf{y}}$	predicted per-node person labels
$\mathbf{y}^{\star}$	ground-truth person labels
z	estimated relative depth coordinate
$\mathbf{z}_i$	final learned embedding of node i

Greek symbols

α_{ij}	normalised attention weight for edge (i, j)
γ	contrastive-loss margin ($\gamma = 0.5$)
$\gamma_{\mathbb{H}}$	hyperbolic contrastive-loss margin ($\gamma_{\mathbb{H}} = 2.0$)
Δ	change in PGA relative to a reference row
θ	pivot angle of a joint triplet (TriGAT)
$\lambda_{(\cdot)}$	weight of a loss term (orthogonality, cluster, spectral, type)
$\boldsymbol{\mu}_j$	cluster centre j
$\pi^{\star}$	optimal cluster-to-person matching (Hungarian)
σ	Gaussian spatial-kernel bandwidth (SA-DMoN)
τ	learnable tangent-scale of the hyperbolic map
ξ	free parameter for the learnable bandwidth, $\sigma = e^{\xi}$

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1   INTRODUCTION

The purpose of this chapter is to motivate multi-person pose grouping (the stage that decides which keypoint belongs to which person) as a separable problem with respect to the broader bottom-up human pose estimation task - identifying the gap for a modular detector-independent grouping method. It states the hypothesis the work defends as well as the research questions, approach and contributions which will ultimately define the rest of the document.

1.1   CONTEXT

Multi-person pose estimation is the task of identifying and locating a fixed set of anatomical keypoints - the seventeen joints of the COCO skeleton [1] - on every person present in an image. The bottom-up family of methods breaks the task up into two distinct stages: a detector first identifies every keypoint in the image (regardless of which person it belongs to) and then the grouping stage assigns each detected keypoint to a person (Figure 1.1). This separation of concerns is what defines the bottom-up methods starting from the smallest unit available (a joint) and working up towards a person. This allows the runtime to be approximately constant in the number of people but means that the grouping decision itself becomes difficult.

(a) keypoints without person identity (b) keypoints assigned to person identities

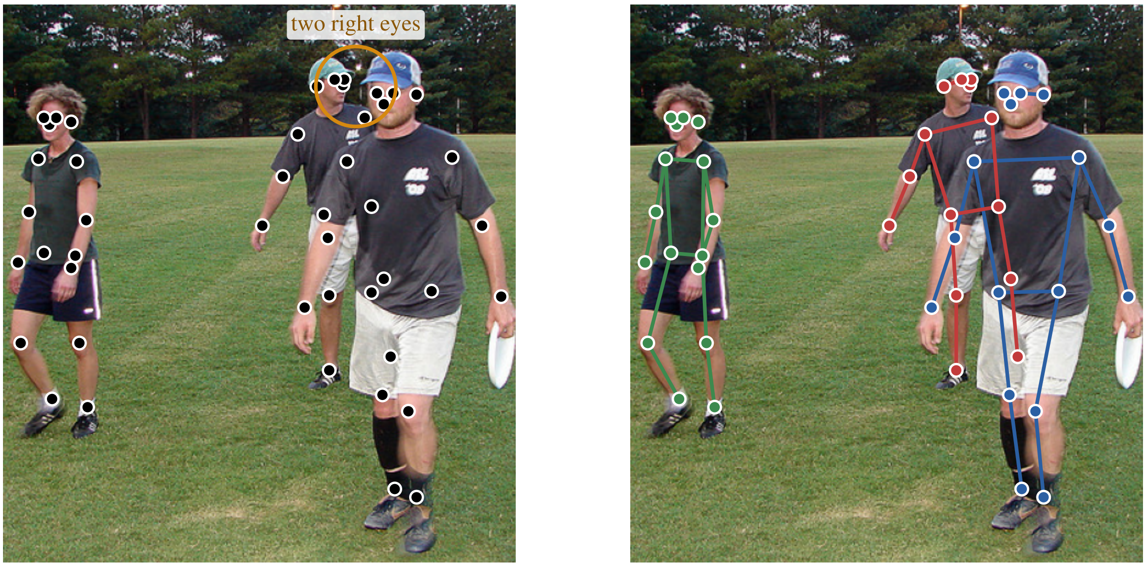

Figure 1.1. Multi-person keypoints from COCO `val2017` (image 482800), drawn from the ground-truth annotations. In (a) every keypoint carries a joint type but no person identity, which is what the grouping stage is given; the circled section shows the right eyes of two different people, 33 pixels apart. In (b) the same keypoints are coloured by the person they belong to and joined by the COCO skeleton, which is what the grouping stage has to produce. The two people on the right overlap while the person on the left does not.

In this context multi-person pose estimation is then two things: keypoint detection and keypoint grouping. The detection stage is well studied with potential solutions including high-resolution convolutional backbones [2, 3], transformer estimators [4], and human-centric foundation models [5] which have pushed per-keypoint localisation accuracy on the COCO benchmark toward saturation. Residual errors of modern bottom-up pipelines happen downstream of detection in deciding which person each detected keypoint belongs to. This decision is quite difficult for a number of structural reasons:

- The number of people in the scene is unknown at inference time.
- Every person can contribute at most one keypoint of each joint type.
- Occlusion and truncation remove keypoints unpredictably.
- People who overlap in the image (without occluding) place same-type keypoints close to each other, for example two left elbows.

In established bottom-up methods, this grouping decision is produced by a mechanism that is trained jointly with the keypoint detector. Part-affinity fields [6], associative-embedding tags [7, 3], centre-offset regression [8], and end-to-end set prediction [9] all couple the grouping method to a particular estimation backbone. This coupling buys accuracy at the cost of reusability - a grouping mechanism trained with respect to one detector cannot be detached and applied to the output of another one. This work studies the alternative: a grouping module that consumes arbitrary detector-emitted keypoints and types. The question is what that costs in accuracy and whether the modularity is worth it.

1.2 PROBLEM STATEMENT

A given image contains an unknown number of people, denoted by K. A keypoint detector produces N detections, where each detection has an image-plane position, a joint-type label $t_i \in \{1,\ldots,17\}$ corresponding to the 17 COCO joint types, and a visibility flag. The pose-grouping problem is to partition these N detections into K groups such that each group corresponds to a single person. The partition is also subject to the structure of the human skeleton: each person can contain at most one keypoint of a given joint type.

Two restrictions define the problem considered in this work. First, the grouping module operates only on the detected keypoints. It has no access to the source image, intermediate detector feature maps, or the detector's training procedure. This separation is intended to make the grouping stage independent of the upstream detector, provided that the detector produces keypoints using the COCO 17-joint representation.

Second, the number of people K is not assumed to be known at inference time. The complete pipeline must therefore estimate K from the detected keypoints before producing the final partition. An oracle-K configuration, in which the ground-truth person count is supplied, is retained as a diagnostic. This separates the quality of the learned embedding and grouping procedure from errors introduced by person-count estimation. The formal notation, graph construction, and evaluation metric used for this problem are defined in Chapter 3.

1.3 RESEARCH GAP

The grouping mechanisms of established bottom-up methods are tightly coupled to their host detectors. Part-affinity fields [6, 10] share a backbone with the heatmap head; associative-embedding tags [7, 3] are calibrated to the feature scale of the network they are trained with; centre-offset methods [8, 11] fold grouping into a backbone-specific regression head; and end-to-end set-prediction methods [9, 12] never expose a per-keypoint representation at all. In all these cases, swapping the detector after training is not supported.

A *modular* grouping module is one that takes detected keypoints from any detector, runs a separable algorithm and returns per-keypoint person identities. Such modules are fairly unstudied. The closest graph-based precedents are hierarchical graph grouping (HGG) [13] and graph edge classification (GEC) [14]. HGG is trained jointly with its detector and is therefore not modular; GEC is detector-independent but uses a per-edge binary classification rather than a per-keypoint embedding. No prior work, in this modular setting, trains a detector-independent graph-neural-network embedding for multi-person keypoint grouping, or addresses autonomous person-count estimation as part of the same pipeline. This is the research gap the present work addresses; the supporting literature is reviewed in full in Chapter 2.

1.4 HYPOTHESIS

The investigation is organised around a single composite hypothesis.

> *Multi-person pose grouping is fundamentally an embedding-quality problem rather than a grouping-algorithm problem. A skeleton-aware graph attention network trained on synthetic data and fine-tuned on real images can produce per-keypoint embeddings on which a simple unconstrained clustering algorithm recovers person identities at quality competitive with an end-to-end joint-trained alternative, while supporting autonomous person-count estimation that requires no oracle at inference and a modular composition with any keypoint detector.*

This hypothesis has three distinct parts: a methodological, an architectural and an operational claim.

- **Methodological claim:** The binding constraint on bottom-up pose grouping is the embeddings rather than the algorithm that operates on them.
- **Architectural claim:** Skeleton-aware graph attention network (GAT) embeddings (trained on synthetic data then fine-tuned) are the structural realisation of the methodological claim.
- **Operational claim:** The same embeddings would allow the grouping to be deployed using arbitrary detectors and without an oracle at inference.

The three clauses are testable on their own and each decomposes into the specific research questions in the next section. Chapter 6 discusses each clause and states which parts of the hypothesis the experimental evidence of Chapter 4 supports, and with what caveats. The research questions are then the instruments by which each clause is interrogated. Success or failure on any single question neither confirms nor refutes the hypothesis on its own, but together, the seven questions span its testable surface.

1.5 RESEARCH OBJECTIVE AND RESEARCH QUESTIONS

The objective is to design and evaluate a detector-independent, GAT-based pose grouping module.

- **RQ1 - The lever.** Is the binding constraint on bottom-up pose grouping the per-keypoint embedding, or the algorithm that groups those embeddings into person identities?
- **RQ2 - Inductive biases.** Which skeleton-aware modifications to a GATv2[1] [15] embedding network improve pose grouping, and what is the marginal contribution of each?
- **RQ3 - Detector independence.** Can a grouping module trained separately from the keypoint detector match one trained end-to-end on a real-image benchmark, and at what cost in absolute pose-grouping accuracy?
- **RQ4 - Embedding geometry.** Does the choice of embedding space (Euclidean versus hyperbolic) affect pose grouping quality, and if so, in which regime?
- **RQ5 - Graph primitive.** Does a higher-order graph primitive (skeleton triplets) produce a better embedding for pose grouping than single joints?
- **RQ6 - Synthetic-to-real transfer.** How much does COCO fine-tuning improve a synthetically-trained embedding on real images, and does the lift come at a cost to synthetic-domain accuracy?
- **RQ7 - Autonomous deployment.** Can the number of people in a scene be estimated from the embedding alone at inference, and does the resulting fully-autonomous pipeline match the configuration in which the person count is supplied as an oracle?

1.6 APPROACH

The grouping module is initially built on a standard implementation of a GATv2 (graph attention network, version 2) [15] model trained with contrastive loss and k-means. This configuration is used as the baseline. The detected keypoints are used to build a graph per image, where each keypoint is a

[1] GATv2 is the graph attention variant discussed in Section 3.1.3.

node, and training forces same person keypoints closer together in embedding space while separating different person ones. These embeddings are grouped using k-means to find person identities. The training pipeline uses a generated synthetic dataset with precise labels and can be further fine-tuned on COCO [1]. Three grouping heads are applied to the embeddings (to replace the presumed naive k-means): deep embedded clustering [16], slot attention [17] and graph partitioning [18], as well as ten variants of a novel skeleton-aware deep modularity network (SA-DMoN), an extension of DMoN [19]. These all fail to outperform k-means, forcing a pivot to focus on the embeddings themselves, leading to the development of the novel PG-GAT, which is a GATv2 with three skeleton-aware modifications to the attention mechanism. With the success of this module (as compared to the grouping attempts), its architecture, loss and fine-tuning hyperparameters are then selected by Bayesian sweeps. Variants are then created to stress the design including a hyperbolic embedding space, a triplet-based graph primitive, and visual-feature augmentation. Finally, a frozen-backbone person count head is added to remove the oracle-K dependency, closing the loop with a fully autonomous end-to-end pipeline.

The evaluation covers three settings: the synthetic distribution, COCO with ground-truth keypoints, and a full end-to-end pipeline, where the keypoints are obtained from a detector. For the end-to-end setting, that detector is a frozen HigherHRNet [3]. Modularity and detector independence are then shown by applying the same grouping module, without retraining, to detections from two further detector families: YOLO11x-pose [20] and RTMO-L [21].

1.7 RESEARCH CONTRIBUTIONS

The contributions of this work are:

1. **PG-GAT**, a novel skeleton-aware graph attention network for detector-independent pose grouping. Three modifications are applied over the standard GATv2: type-pair attention bias, skeleton-relative position encoding, and same-type repulsion heads; which when combined provide a larger gain on the synthetic set than any single modification alone (while the individual differences are too small to cleanly separate from noise). After the hyperparameter sweeps, the optimised model reaches 0.9715 COCO val PGA on ground-truth keypoints and 0.9591 end-to-end PGA on HigherHRNet detections, against the joint-trained associative-embedding (AE) reference [7, 3] of 0.9847 on the same detections.
2. **Evidence that the embedding, not the grouping head, is the binding constraint** with regard to the clustering problem. Three learned grouping heads (deep embedded clustering [16], slot

attention [17], graph partitioning [18]) and ten SA-DMoN variants were tested on the same backbone, none outperforming plain k-means on the same embeddings.

3. **An autonomous deployment pipeline with a detector swap.** A frozen-backbone K-estimation head reaches 76.1% accuracy detecting how many people are in a scene and 91.9% off-by-one person-count accuracy on COCO val. This head plugged into the now autonomous pipeline scores 0.9644 end-to-end PGA - which is marginally better (0.0053) than the oracle-K configuration on the same checkpoint (for a structural reason developed in Chapter 5). The frozen pipeline is then applied zero-shot (no retraining) to YOLO11x-pose [20] and RTMO-L [21] detections, and trails each detector's native grouping by 0.0210 and 0.0220 PGA, so the cost of detector independence is approximately 0.02 across three detector families.
4. **Documented negative results on PG-GAT extensions.** Attempted modifications to improve PG-GAT are reported with the reasons for their failures: a hyperbolic (Lorentz-model) embedding matches the performance of a four times larger Euclidean embedding only on the synthetic dataset, a triplet-based primitive (named TriGAT) underperforms the joint based primitive given the same training budget, with the triplet to joint voting identified as the method's failure and a visual feature augmentation (MobileNetV2 [22]) which does not beat the simple position-and-type-only model while adding 2.2×10^6 additional parameters.
5. **A reproducibility chain.** Every cited number traces to a logged run identifier in a public experiment-tracking project, and every PG-GAT hyperparameter is defended by a Bayesian sweep over its operating range.

1.8 RESEARCH OUTPUTS

The primary research output is a journal paper on PG-GAT, targeted at *Image and Vision Computing* (Q1, IF 5.34), with *Pattern Recognition* (Q1, IF 9.84) as a stretch target.

1.9 OVERVIEW OF STUDY

Chapter 2 is the literature review in six parts: the multi-person pose estimation landscape, bottom-up grouping, graph neural networks applied to pose problems, contrastive learning and the grouping heads tested against it, geometric embedding spaces, and a consolidation of the research gap. Chapter 3 develops the methodology: the graph-neural-network and clustering preliminaries, the problem formulation, the dataset pipeline, the baseline and each investigated method, the end-to-end integration, and the evaluation protocol. Chapter 4 reports the experimental results. Chapter 5 is the discussion of the results across three themes: the embeddings as the binding constraint, synthetic-to-real transfer, and the cost of modularity, together with the behaviour of the autonomous pipeline. It closes by answering the research questions. Chapter 6 is the conclusion to this document. It provides a final verdict on the original hypothesis, consolidates the contributions, and identifies future work.

1.10 DECLARATION REGARDING USE OF GENERATIVE AI

Generative AI models, predominantly Claude by Anthropic, were used throughout the work reported in this dissertation. Each use is listed below, together with the level at which it was applied.

- **Literature.** Finding candidate references and summarising papers. Every cited work was read and checked against its source before citation.
- **Research code.** The experimental codebase was written with substantial AI assistance, ranging from IDE-level completion to agent-authored model variants, training and evaluation scripts, and experiment-tracking integration.
- **Figures.** The data figures were produced by AI-assisted Python scripts operating on the logged experimental results, and the architecture and pipeline diagrams were drawn in TikZ with the same assistance.
- **Text.** A first draft of every chapter was produced with AI assistance from the experimental record and lab book. That draft was subsequently rewritten for this document, with AI used to check the numerical and factual consistency of the rewritten text against the logged results.
- **LaTeX and language.** Markup for tables, figures and equations, and corrections to grammar and formatting.

The experimental design, the choice of experiments, and the interpretation of their results are my own.

CHAPTER 2 LITERATURE REVIEW

This chapter situates multi-person pose grouping within the broader human pose estimation literature, reviews the existing families of grouping mechanisms, and surveys the graph neural network and representation-learning work that PG-GAT builds on. The chapter then closes by identifying the specific gap.

2.1 MULTI-PERSON HUMAN POSE ESTIMATION

Human pose estimation (HPE) is the problem of localising a fixed set of anatomical keypoints - such as the seventeen joints of the COCO skeleton [1] - on every person present in a given image. This is a large domain, so the work is restricted to 2D and multi-person. In this context the goal is to find per-image keypoints for an unknown number of people without the use of temporal context or 3D reconstruction. The following three useful surveys provide considerable background and scope: Chen et al. [23] cover monocular HPE end-to-end, Zheng et al. [24] provide a more recent deep-learning-focused survey, and Andriluka et al. [25] anchor the modern benchmark culture with the MPII dataset.

The single-person pose estimators that underpin most multi-person systems emerged from a series of convolutional architectures. Toshev and Szegedy's DeepPose [26] (a seminal work) framed keypoint localisation as direct coordinate regression from a deep network. Subsequent architectures shifted to dense heatmap prediction: convolutional pose machines [27] introduced iterative belief-map refinement, and the stacked hourglass network [28] established the encoder-decoder-with-skip-connections topology that underpins most modern single-person estimators. Xiao et al.'s "simple baselines" work [29] showed that a ResNet trunk with three deconvolution layers matched hourglass accuracy at lower complexity (an important finding given the trend of increasing model size). The HRNet family [2] pushed this further by running several parallel branches at different spatial resolutions throughout the network and exchanging information between them at every stage, rather than recovering resolution through up-sampling at the end of an encoder-decoder pass. The resulting access to high-resolution feature maps yielded sharper keypoint heatmaps and lower localisation error than any prior backbone of comparable parameter count, which is why HRNet became the gold-standard backbone for most top-down systems and for the bottom-up HigherHRNet variant [3] discussed in Section 2.2.

2.1.1 Top-down approaches

Top-down systems handle multi-person HPE by first detecting each person (using a person detector) and then applying a single-person pose estimator to each detected bounding box. This architecture is most notably represented by Mask R-CNN [30], which extends the Faster R-CNN detector with a per-keypoint mask head, and AlphaPose [31], which combines a person-detection front-end with a symmetric spatial transformer and a parametric pose non-maximum suppression step. Top-down methods generally have strong per-person keypoint localisation accuracy due to the simplification of the pose estimation task to a single person at a time. However, there are two main limitations to this approach. Firstly, runtime scales linearly with the number of detected people. Secondly, the performance of these methods is limited by the quality of the person detection stage: any incorrectly detected or missed bounding boxes will lead to missed or incorrectly grouped keypoints, with no opportunity for the pose estimator to recover from these errors.

2.1.2 Bottom-up approaches

Bottom-up methods work in reverse. The first task is to detect all the keypoints in an image and then a separate grouping mechanism assigns each keypoint to a person. Detecting all keypoints in a single pass allows for a runtime that is approximately constant in the number of people, and detection errors are localised to individual keypoints rather than to entire person instances. Grouping, being a critical step (specifically for this work), is reviewed further in Section 2.2. The exception to this is end-to-end transformer methods (Section 2.1.3), which do not require a separate grouping stage and are discussed separately.

2.1.3 Transformer-based approaches

The transformer architecture has been applied to HPE in two distinct ways: as a drop-in replacement for the convolutional backbone of a standard heatmap-based estimator, and as a complete reimagining of the multi-person task using end-to-end set prediction with no separate grouping stage. The latter approach cannot be decoupled from the backbone and does not produce embeddings that a downstream grouping module could use.

TokenPose [32] demonstrated that a pure-transformer network where each keypoint is a learned token could match the heatmap accuracy of CNN baselines at comparable parameter counts. HRFormer [33] preserved the parallel-resolution structure of HRNet while replacing convolutions with local-window self-attention, recovering high-resolution feature representations. ViTPose [4] took the opposite design choice - a plain, monolithic Vision Transformer as the backbone with no high-resolution path and no domain-specific inductive bias for keypoint localisation - it then showed that this minimal architecture can scale cleanly to model sizes from 100M up to 1B parameters (improving with scale). A simple deconvolutional head turns the patch-token grid into per-keypoint heatmaps, so ViTPose plugs into the same downstream pipeline as a CNN backbone. Its successor ViTPose++ [34] extends the original ViTPose by adding a mixture-of-experts (MoE) formulation to cover the wholebody, hand, and animal-pose distributions within a single trunk.

A separate line of transformer work reimagines multi-person HPE as direct set prediction, so the keypoints are generated already grouped by person, meaning there is no need for a separate grouping stage. PETR [9] was the first end-to-end pose detector to apply the DETR [35] set-prediction paradigm to human keypoints: a fixed pool of pose queries cross-attends to image features, and each query learns to predict a complete per-person skeleton (one shot). The predictions form an unordered set with no fixed pairing to the ground-truth people, so Hungarian matching at training time assigns each ground-truth person to one of the queries. Here the binding between keypoints and person is the query identity. QueryPose [36] sparsified the attention pattern by splitting each pose query into part-level sub-queries. ED-Pose [12] and GroupPose [37] modified the query-based formulation with ED-Pose adding a person-box and GroupPose simplifying the architecture to allow keypoints of the same person to communicate; both reaching competitive performance on the COCO benchmark without any post-hoc grouping step.

2.1.4 Heatmap and regression trends

A long-standing issue in regression-based pose estimation is the non-differentiability of the argmax operator which maps continuous outputs to discrete keypoint coordinates. Sun et al. [38] addressed this with integral pose regression, introducing a differentiable soft-argmax that bridged the heatmap and regression paradigms. The differentiable spatial-to-numerical transform (DSNT) [39] later formalised the same idea as a normalised spatial-coordinate layer.

RLE [40] offered a more direct probabilistic treatment, obtained by modelling residuals between predicted and ground-truth coordinates with a flow-based density, and it was the first regression method to exceed heatmap accuracy on COCO. SimCC [41] took a different route, reframing coordinate prediction as two-axis classification, which became the basis for the real-time RTMPose family [42]. RTMO [21] later unified detection and regression in a single stage, and RTMW [43] extended this to whole-body pose at real-time rates. The YOLO family follows the same single-stage pattern, attaching a per-instance keypoint regression head to its object detector, with YOLO11-pose [20] as the current representative.

For this context, the specific paradigm is not critical. The grouping module proposed in Chapter 3 operates on per-keypoint coordinates and is indifferent to whether those coordinates came from a heatmap or a regressor.

2.1.5 Foundation models for pose estimation

Recent work has also moved human pose backbones towards foundation-model scale. Sapiens [5], for example, pretrains Vision Transformer backbones ranging from 0.3 to 2 billion parameters using 300M curated human images. This produces a unified human-vision backbone that achieves state-of-the-art accuracy across pose estimation, segmentation, depth, and surface normal prediction.

A similar move towards promptable and dataset-unified methods can be seen in X-Pose [44], which detects arbitrary keypoint categories based on either a textual or visual prompt across thirteen datasets. PCT [45] takes a different approach using discrete codebook tokens to represent poses and improve robustness to occlusion.

These foundation-model approaches are relevant here mainly as context rather than direct points of comparison. Larger pretrained backbones can reduce the localisation error of individual keypoints to near saturation. This further makes the case for studying grouping as a separate, modular stage after detection.

2.1.6 Benchmarks

Three datasets are particularly common in the 2D multi-person pose literature. MPII Human Pose [25] established much of the modern benchmark protocol, with 25K images and approximately 40K annotated people across 410 activities. However, its focus is mainly on single-person crops, and it has largely been superseded for multi-person research.

COCO [1] is now the de facto benchmark for multi-person pose estimation, using seventeen keypoints per person and containing approximately 250K annotated instances across the train, val, and test partitions. The val2017 set is commonly used for reporting results and is also the set used for the COCO evaluations in Chapter 4.

CrowdPose [46], on the other hand, is designed specifically for crowded scenes. It includes a crowd-index metric and approximately 20K images selected for significant inter-person occlusion, making it a useful benchmark for testing grouping methods under more difficult conditions, although it is not used in the experiments later in this work. The OCHuman benchmark, introduced alongside Pose2Seg [47], also focuses on heavily occluded people and is therefore relevant when considering grouping robustness.

Across these benchmarks, average precision (AP) under the Object Keypoint Similarity (OKS) metric is the dominant evaluation measure. OKS was introduced with the COCO benchmark [1] and measures the agreement between predicted and ground-truth keypoints using a Gaussian-based tolerance, where the variance depends on the keypoint type and the size of the person's bounding box. A prediction is counted as correct above an OKS threshold, with AP normally averaged over thresholds in the range $[0.50, 0.95]$ in steps of 0.05.

The earlier MPII-era metric, Percentage of Correct Keypoints (PCK), uses a different approach: a keypoint is counted as correct when it falls within a fixed fraction of the head segment length. PCK is still sometimes reported for MPII, but AP@OKS has largely replaced it for multi-person benchmarks.

AP@OKS is not reported directly in Chapter 4. Instead, the grouping contribution is evaluated using Hungarian-matched per-joint accuracy (see Section 3.13), which separates the grouping decision from the quality of the initial keypoint detections. The grouping baselines discussed in this chapter are, however, reported using AP@OKS in their original publications, and the relationship between the two metrics is discussed further in Chapter 4.

2.1.7 Pose tracking

Multi-person 2D pose tracking extends per-image HPE by associating people across multiple frames. PoseFlow [48] is a well-known online tracker that maintains person identities using pose-similarity matching over a small temporal window, while LightTrack [49] uses a top-down tracking framework with a learned re-identification head. The PoseTrack benchmark [50] is the standard evaluation set for this video-based pose tracking problem.

Pose tracking is outside the scope of this work, which is limited to grouping keypoints within individual images. However, grouping remains an important step for each frame. Once the per-frame skeletons have been assembled, temporal association can generally be treated as a re-identification problem over a short sequence of frames. The per-image grouping module developed in Chapter 3 can therefore be used as a separate stage before temporal association, with standard ReID methods handling the subsequent tracking step.

2.1.8 Position of the present work

The HPE literature has converged on a relatively small set of strong single-image detection backbones, with per-keypoint localisation accuracy now reaching saturation. HRNet-family backbones already achieve mean OKS-based average precision above 0.75 on COCO val2017, while more recent foundation-model approaches such as Sapiens [5] improve on this further through pretraining on hundreds of millions of human-centric images. For systems where models of this scale are not practical, real-time approaches such as RTMPose [42] and RTMO [21] achieve much of the same accuracy using considerably smaller models. This suggests that, for many practical multi-person systems, localising individual keypoints is no longer the main limitation on performance.

For bottom-up approaches, an important source of error therefore lies after detection, particularly in deciding which detected keypoints belong to the same person. Most of the established methods reviewed above make this grouping decision in a way that is closely tied to the detector. Part-affinity fields, for example, are predicted by the same network that generates the keypoint heatmaps. Associative-embedding tags are similarly trained together with the heatmap loss on a particular backbone. Centre-regression methods such as DEKR also combine detection and grouping through a backbone-specific keypoint-from-centre offset head. End-to-end transformer pose detectors couple the two stages even more closely: in these models there is no separate per-keypoint representation to group, since the keypoints are already associated through the pose query that produced them. As a result, replacing the detector after training is impossible, and grouping methods are often tied to the detector and grouping head used by the original architecture.

In comparison, the literature on *modular* grouping is sparse. Here, modular grouping refers to methods that accept detected keypoints from any upstream detector, perform grouping as a separate stage, and return a person identity for each keypoint. Several recent methods can be viewed in this way. HGG [13] treats the problem as higher-order graph matching between detected keypoints, while GEC [14] formulates it as binary edge classification. The instance-aware method of Yang et al. [51] instead uses a learned per-person assignment head.

The method proposed in this work follows the same modular direction. PG-GAT, presented in Chapter 3, uses a graph neural network to operate on keypoints produced by any upstream detector and generates an embedding for each detected keypoint. These embeddings can then be clustered to recover the individual person identities. The following sections review the literature that most directly supports this approach: grouping mechanisms in Section 2.2 and graph neural networks for pose in Section 2.3.

2.2 GROUPING IN BOTTOM-UP POSE ESTIMATION

The bottom-up methods discussed in Section 2.1.2 generally follow the same structure: a single-image detector first produces the keypoints present in the image, after which a grouping mechanism determines which keypoints belong to each person. It is mainly this grouping stage that separates the different bottom-up approaches. The way grouping is performed determines whether the detector can be replaced independently of the rest of the system.

This section reviews the four main grouping approaches identified in the bottom-up literature, following the order in which they were introduced. It then considers the smaller group of graph-based methods, which is where the contribution of this work is positioned. Finally, the section identifies the main property that the proposed method in Chapter 3 is intended to preserve.

2.2.1 Part-affinity fields

The first end-to-end bottom-up method, OpenPose [6], predicts a dense 2D vector field for each anatomical limb alongside the standard per-keypoint heatmaps. These vector fields are known as part-affinity fields (PAFs). At each pixel, the PAF contains a 2D vector pointing in the direction of the corresponding limb. During inference, candidate pairs of keypoints, such as a left shoulder and left elbow, are scored using the line integral of the PAF between their locations. Bipartite matching is then applied to the resulting weighted graph to assemble the individual person skeletons.

This matching is performed greedily, one limb at a time. While this makes the approach efficient, it also means that decisions are made locally. An incorrectly assigned limb cannot easily be corrected through a later global optimisation step once the matching decision has already been made. PifPaf [10] builds on the PAF approach using a Laplace-distributed regression head, improving keypoint localisation for low-resolution inputs and partially occluded people. However, its grouping stage still relies on a similar greedy bipartite-matching procedure.

An important consequence of the PAF design is that the limb-field and heatmap predictions come from a shared backbone and are trained together. This makes replacing the detector after training impossible: the PAF head is tied to properties of the detector such as its spatial resolution and feature stride, while the matching procedure depends on the relationship between the predicted PAFs and heatmap peaks. This differs from the approach taken by PG-GAT, which is designed to work directly with keypoint coordinates produced by an upstream detector. PG-GAT therefore treats grouping as a separate, detector-independent stage rather than as an extension of the PAF framework.

2.2.2 Associative embeddings

Newell and Deng [7] introduced associative embeddings as another approach to bottom-up grouping. A small head attached to the keypoint detector predicts a scalar, or short vector, “tag” for each detected keypoint. The training loss consists of two main parts: a pull term encourages keypoints belonging to the same person to have tags close to their per-person mean, while a push term encourages the means of different people to remain separated. At inference, the predicted tags can then be clustered using a simple threshold or, for scalar tags, a method such as 1D k-means. The resulting clusters correspond to the different people in the image. HigherHRNet [3] later applied this approach to the high-resolution HRNet [2] backbone family and provides the publicly available reference used for the end-to-end comparisons in Chapter 4.

One of the strengths of associative embeddings is that the grouping representation and keypoint detector are trained together. The associative-embedding loss is calculated using the keypoint detections produced by the heatmap head during the same forward pass, so the tag head can adapt to the feature representation and detection errors of the particular backbone. This can be useful in difficult regions where poses overlap or keypoint locations are ambiguous, and helps explain the strong end-to-end performance of associative embedding methods.

The disadvantage is that this also creates a close dependency between the detector and grouping head.

The learned tags are based on the features of the backbone used during training, meaning that the tag head cannot be moved to a different detector. This is an important difference from the modular approach considered in this work. Chapter 4 therefore uses the AE pipeline's headline E2E PGA as an upper-bound reference point for the proposed method. The difference between this result and the modular approach, referred to as the detector-independence gap in Section 4.9, provides a measure of the performance cost associated with giving up this joint-training advantage.

2.2.3 Mid-range offsets and centre regression

PersonLab [52] introduced another major approach to bottom-up grouping. Alongside the keypoint heatmaps, the network predicts mid-range offset vectors that connect keypoints belonging to the same person. Person skeletons are then constructed by starting from a high-confidence keypoint, such as the nose or shoulder, and following the predicted offsets to the remaining joints.

DEKR [8] takes a simpler centre-based approach. Rather than predicting a full set of pairwise offsets, it predicts offsets between keypoints and a corresponding person centre, making the grouping stage more direct since keypoints can be associated through their predicted centre. CenterGroup [11] develops this idea further using a transformer to generate context-aware embeddings for both detected keypoints and predicted person centres. Multi-head cross-attention is then used to associate each joint with the appropriate centre.

There are two main consequences of centre-regression methods when considering them for downstream grouping. First, the grouping decision is largely determined by the predicted offsets or centre associations at inference, which makes grouping relatively fast but does not provide a general per-keypoint representation that could be passed to a different clustering method. Second, as with associative embeddings, the offset predictions are learned jointly with the keypoint detector, so the grouping behaviour is closely linked to the backbone and features used during training.

These approaches achieve strong end-to-end performance, but this comes with reduced flexibility when the detector or grouping stage needs to be replaced independently. This same trade-off between end-to-end performance and modularity is also present in the approaches discussed in the following two subsections.

2.2.4 End-to-end set prediction

The fourth grouping lineage takes a different approach by removing the explicit grouping stage and replacing it with end-to-end set prediction. Methods such as PETR [9], ED-Pose [12], GroupPose [37], and QueryPose [36] follow the DETR [35] set-prediction paradigm. A fixed set of pose queries attends to the image features, with each query learning to predict a complete skeleton for one person. During training, bipartite matching is used to assign each ground-truth person to a corresponding query. There is no separate grouping stage during inference - the keypoints are already associated with a person through the query that produced them.

These approaches achieve competitive results on COCO without requiring additional grouping logic after detection. However, this structure is less suitable when grouping needs to be treated as an independent problem. The network does not expose a separate per-keypoint representation, since the relationship between a person and their keypoints is contained within the pose query itself. A modular grouping method that works on detached keypoint coordinates therefore cannot be added to a DETR-style pipeline without making significant changes to the pose-query architecture.

For this reason, set-prediction methods are included here as an alternative approach to solving the multi-person pose problem rather than as a direct comparison with the modular grouping method proposed in this work.

2.2.5 Graph-based grouping

A smaller but growing area of the bottom-up literature treats keypoint grouping as a graph problem. This is also the area in which the proposed PG-GAT method is positioned. HGG [13] formulates grouping as hierarchical higher-order graph matching, where detected keypoints form the nodes and candidate connections between them form the edges. A learned discriminator is then used to progressively merge smaller sub-graphs into clusters representing individual people. However, the graph edges and merge predictor are trained together with the detector - this keeps the benefits of joint training, but also means that the grouping method remains closely linked to the detector.

Yang et al. [51] take a different graph-related approach by supervising the self-attention weights within a Vision Transformer to become instance-aware. The resulting attention matrix contains information about which keypoints belong to the same person. In this case, however, grouping is again learned as part of the detector backbone rather than as a separate stage.

The graph-based method most closely related to the approach proposed in this work is Graph Edge Classification (GEC) [14]. GEC predicts a binary same-person label for pairs of detected keypoints and then recovers person identities from connected components in the resulting affinity graph. Importantly, GEC operates on the outputs of an existing detector without requiring the detector itself to be retrained, making it the most direct comparison point for modular grouping methods.

PG-GAT, introduced in Chapter 3, differs from GEC mainly in how the graph information is learned and represented. It uses type-aware edge attention and produces an embedding for each keypoint, rather than predicting only a binary score for each edge. The graph-partitioning ablation in Section 4.2 evaluates an edge-classification alternative using the same backbone as PG-GAT, providing a more direct comparison between the two formulations, and is used to evaluate the choice of a per-keypoint embedding over a per-edge classifier.

2.2.6 The modularity criterion

The five approaches discussed above cover most of the bottom-up grouping literature. In four of these, the grouping mechanism is closely tied to the detector that produces the keypoints. Graph-based grouping with explicit per-keypoint operations is the exception, since the grouping stage can, at least in principle, be used with outputs from a different detector.

This separation is the defining property of the proposed PG-GAT method. The embedding network operates directly on detected keypoint coordinates and does not require access to the detector's internal feature maps or its original training data. The upstream detector can therefore be replaced after PG-GAT has been trained without requiring the grouping network to be retrained.

The detector-independence described in Section 3.12 follows from this modular design, and its practical effect is evaluated through the end-to-end comparison in Section 4.9. The following section reviews the graph neural network literature that provides the basis for the proposed embedding network.

2.3 GRAPH NEURAL NETWORKS FOR HUMAN POSE

The previous section identified graph-based grouping as the bottom-up approach that allows a modular grouping stage. This section looks more broadly at how graph neural networks (GNNs) have been used for human pose problems and identifies where the proposed method fits within this literature.

The aim here is not to provide an exhaustive review of every GNN-based pose method. Instead, the focus is on work that uses graph operations similar to those used by the proposed method, but applies them to different pose-related tasks. In particular, relatively little of this work considers the problem of grouping detected keypoints across multiple people.

The underlying mathematical concepts, including message passing, GCN, GAT, and GATv2, are described formally in Section 3.1. This section instead considers how these methods have been applied within the pose literature and why multi-person keypoint grouping remains a less explored application.

2.3.1 Skeleton-based action recognition

The largest body of GNN-based pose research focuses on skeleton-based action recognition rather than pose estimation itself. ST-GCN [53] introduced the spatio-temporal graph convolutional network as an important architecture for this task. A spatial graph is first constructed using the COCO or NTU skeleton topology, while temporal edges connect each joint to the same joint in the following frame. Spatial-temporal graph convolutions are then applied to produce a global representation of the sequence, which is used to predict an action label.

2s-AGCN [54] extends this idea by replacing the fixed skeleton adjacency with a learned, sample-dependent adjacency, allowing the network to learn connections beyond the standard anatomical skeleton when those connections are useful for recognising an action.

The focus of these methods is quite different from the grouping problem considered in this work. Their inputs are sequences of *already-grouped* skeletons, typically with each graph representing a single person, and their output is a classification label for the sequence. The grouping of keypoints into people is therefore assumed to have taken place before the graph is constructed, so the network is not trained to produce an identity or grouping signal for individual keypoints.

Although the underlying graph operations are relevant, the graph structure and training objective are designed for a different task. Applying an ST-GCN-style architecture directly to grouping would require a graph containing multiple people within the same frame and an output defined at the individual keypoint level - these requirements fall outside the main design of the action-recognition methods in this lineage.

2.3.2 Single-person 3D pose lifting

A second area where GNNs are commonly used in pose research is 3D pose lifting. In this setting, the input is a set of 2D keypoint coordinates for a person and the aim is to predict the corresponding 3D coordinates. SemGCN [55] applies a graph convolutional network to the skeleton of a single person and introduces learned weights for the graph edges. The approach in [56] extends this idea using locally connected layers, allowing information to be shared between joints that are not directly adjacent in the skeleton. GAST-Net [57] combines a spatial GCN with temporal attention to estimate 3D coordinates from a sequence of 2D detections.

The general setup is similar to the action-recognition methods discussed above - each input already represents the skeleton of a single person, and the graph describes relationships between that person's joints. The output is a new coordinate prediction for each keypoint rather than an identity or grouping signal, so these methods do not need to determine which person a detected keypoint belongs to since that association has already been established before the graph is passed to the network.

The graph convolution operations used by these methods are relevant to the approach developed in this work, but they are being applied to a different problem - they operate on already-grouped single-person skeletons rather than using the graph to separate keypoints belonging to multiple people.

2.3.3 Domain-specific graph models

A smaller group of related pose methods also make use of graph-structured models. HOPE-Net [58], for example, predicts 3D joint coordinates in hand-object scenes using a small skeleton graph containing both hand and object keypoints. Yang et al. [51] take a different approach by supervising the self-attention matrix within a Vision Transformer so that it captures person instance membership - this can be viewed as a form of graph attention, although the attention operates over image patch tokens rather than directly over detected keypoints.

The method of Yang et al. [51] is relatively close to the problem considered here in terms of its aim, but differs in how the grouping information is produced. Its attention structure is part of the detector backbone and operates on image features, whereas a modular grouping method needs to work on keypoints after they have been produced by the detector without requiring access to the detector's internal features.

HOPE-Net [58] is also graph-based, but addresses a different pose domain and predicts coordinates rather than person identities. Neither approach therefore provides a per-keypoint embedding model for grouping keypoints in multi-person scenes.

2.3.4 Graph neural networks for clustering on general graphs

The proposed PG-GAT method is structurally more closely related to the broader GNN and clustering literature than to the GNN-based pose methods discussed above. Relevant approaches include Newman modularity [59] and its differentiable relaxation DMoN [19], spectral clustering [60, 61], deep embedded clustering [16], and graph-based face clustering methods [62, 18]. These methods are reviewed in more detail in Section 2.4, together with the related representation-learning literature.

The main connection here is architectural - these approaches use graph operations or learned representations to transform the input nodes into features that can then be used for clustering or classification. In the setting considered in this work, the nodes are the detected keypoints and the required output is a representation that allows keypoints belonging to the same person to be identified.

PG-GAT uses GATv2 [15] as its attention mechanism, operating on a separate graph for each image with keypoint type information included at the node level. It then adds the skeleton-aware modifications described in Section 3.8, adapting the more general graph representation and clustering framework to the specific problem of grouping human keypoints.

2.3.5 The naming convention and its rationale

The proposed method is named PG-GAT (Pose Grouping Graph Attention Network). The name reflects the task the method is intended to solve, rather than a particular architectural feature. An earlier name, SA-GAT (Skeleton-Aware Graph Attention Network), was considered but ultimately not used because similar terminology is already common in the pose literature - in particular, the attention-based skeleton action-recognition methods following 2s-AGCN [54] and 3D pose-lifting methods such as GAST-Net [57] already make extensive use of skeleton-aware and spatial-temporal attention terminology. Several of these methods also publish under acronyms that differ from SA-GAT by only one letter.

Using PG-GAT instead avoids potential confusion with these existing methods and makes the purpose of the proposed model clearer - the name emphasises pose grouping as the main task, while the skeleton-aware attention mechanism remains an architectural part of the method described in Section 3.8.

2.3.6 The empty niche

The literature reviewed above shows that graph neural networks are well suited to a range of pose-related problems, but their use for multi-person keypoint grouping remains relatively limited. Action-recognition GNNs operate on skeletons that have already been grouped, and the same is true for most 3D pose-lifting methods. Other graph-based approaches, including those for hand pose, objects, or instance-aware attention, either address a different task or incorporate the grouping mechanism directly into the detector.

There is therefore relatively little work on GNNs designed specifically for multi-person pose grouping. HGG [13], discussed in Section 2.2.5, is the closest example, but its graph grouping mechanism is trained jointly with the detector and is therefore not modular. GEC [14] provides the closest example of detector-independent grouping, although it approaches the problem through edge classification rather than learning an embedding for each keypoint.

PG-GAT occupies the position that neither of these approaches fills: a detector-independent GNN that produces a per-keypoint embedding for multi-person pose grouping. Sections 2.4 and 2.5 review the representation-learning and embedding-space concepts used to support this approach, before Section 2.6 brings these areas together and defines the resulting research gap.

2.4 REPRESENTATION LEARNING AND CLUSTERING PRIMITIVES

The proposed PG-GAT method uses an embedding network trained with a contrastive loss, followed by a k-means read-out during inference. Chapter 4 also evaluates three learned grouping heads applied to the same embeddings (Section 4.2), as well as ten modularity-based variants (Section 4.3). In the experiments, none of these alternatives performs as well as the simpler combination of contrastive learning and k-means.

This section reviews the representation-learning and clustering approaches that are used or evaluated in the experimental work. They are presented in roughly the same order as they appear in the methodology. Contrastive learning is considered first, as it provides the loss used by the final PG-GAT model. This is followed by deep embedded clustering and slot attention, which form the basis of the alternative learned grouping heads, and then modularity-driven clustering, which provides the structural graph alternatives. Finally, graph-based face clustering is reviewed as the closest related work in terms of using learned graph representations to recover identities.

The mathematical details of these methods are introduced in Section 3.2. The focus here is instead on their role in the existing literature and how they relate to the grouping approaches evaluated in this work.

2.4.1 Contrastive embedding objectives

Contrastive learning defines an embedding space using pairwise relationships between samples. Points that share the same identity are encouraged to move closer together in the embedding space, while points with different identities are pushed further apart. Early Siamese network approaches [63, 64] used this idea with a learned distance metric, pulling positive pairs together while separating negative pairs by a specified margin. FaceNet [65] later extended this idea using a triplet loss, where an anchor is compared with both a positive and a negative example within the same loss term.

More recent self-supervised approaches apply the same general idea at a larger scale. SimCLR [66], for example, constructs positive pairs within a batch using different augmentations of the same sample. InfoNCE [67] instead expresses the contrastive objective through noise-contrastive estimation and provides a connection between the learned representation and mutual information.

The loss used in Section 3.5.2 follows the more direct margin-based approach, using cosine similarity between embeddings. This choice fits the grouping problem for several reasons. First, the training data provides an exact supervisory signal - the synthetic-data generator described in Section 3.4.1 supplies a ground-truth person label for every joint, so positive and negative pairs can be constructed directly rather than inferred through augmentation or other self-supervised techniques.

Second, the network produces L2-normalised embeddings, meaning that the embedding space lies on the unit hypersphere. Cosine similarity is therefore a natural measure of how close two embeddings are. A margin-based objective also keeps the formulation relatively simple, with only a single margin parameter γ controlling the required separation between negative pairs. The loss sweep in Section 4.5 confirms $\gamma = 0.5$ as near-optimal.

Triplet-mining and InfoNCE-based alternatives were not explored further in this work. The simpler margin-based loss already produces embeddings that work effectively with a *k*-means read-out, and Section 4.2 shows that the learned grouping heads tested on the same embeddings do not improve on this result.

2.4.2 Deep clustering and slot attention

The contrastive-plus-k-means approach places most of the grouping responsibility in the learned embedding, while the clustering stage acts mainly as a simple read-out. Another family of methods takes a different approach by learning the clustering behaviour as part of the network itself. Deep Embedded Clustering (DEC) [16] combines an autoencoder with soft cluster assignments based on a Student's t-distribution. The embedding and cluster centres are optimised together by minimising the KL divergence between a sharpened target distribution and the current soft assignments - importantly, this process is unsupervised and does not require ground-truth cluster labels.

Slot Attention [17] approaches the problem differently. It uses an iterative attention process between a set of learned "slot" vectors and the input features, with the aim of having each slot represent a separate object or instance. Both DEC and Slot Attention are established representation-learning approaches for discovering structure within image-level features, and were therefore considered as possible learned grouping heads on top of the contrastive backbone.

Neither approach improved the grouping results in the experiments reported in Section 4.2. For DEC, the main problem is related to the behaviour described in Section 3.6.1 - its unsupervised sharpening step reinforces the current cluster assignments, but has no direct way of correcting an incorrect initial assignment. If two same-type keypoints from different people are already close in the embedding space, the sharpening process can therefore strengthen the incorrect grouping rather than resolve it.

Slot Attention has a different limitation in this setting - its competitive assignment between slots is driven by the embedding features, without an explicit constraint based on keypoint type. Consequently, there is no structural rule preventing two keypoints of the same type, such as two left elbows, from being assigned to the same person slot. These behaviours are examined further in Chapter 4 and discussed in Chapter 5. For the present literature review, the important distinction is that neither approach directly incorporates the type-exclusivity prior available in human pose. PG-GAT instead makes use of this structural information through its skeleton-aware attention mechanism.

2.4.3 Modularity-driven graph clustering

A third family of methods performs clustering directly on the graph structure rather than on a learned embedding. Newman modularity [59] measures how much more edge connectivity exists within a proposed cluster than would be expected under a degree-based null model. DMoN [19] turns this idea into a differentiable clustering objective, allowing the modularity criterion to be trained together with a GNN using a linear projection followed by a softmax assignment.

DMoN provides a useful alternative to the learned grouping heads discussed in the previous subsection because it makes direct use of graph topology rather than relying only on the learned embedding. The keypoint kNN graph described in Section 3.3.2 provides this structural information, making modularity-based clustering a reasonable approach to test for the grouping problem.

However, none of the ten SA-DMoN variants evaluated in Section 4.3 outperform the simpler k-means approach. The analysis in Chapter 5 suggests that this is largely due to a mismatch between the modularity null model and the spatial-proximity graph used for pose grouping - the degree-based null model assumes that node connectivity can be explained mainly through node degree, an assumption that is more appropriate for networks such as co-authorship or protein-protein graphs. In the keypoint kNN graph, however, edges are created primarily from spatial proximity, so nearby keypoints are strongly connected regardless of whether they belong to the same person.

As a result, the modularity objective can favour spatially compact groups that do not correspond to the correct person identities. The null-model modifications tested in the SA-DMoN experiments do not provide enough useful signal to overcome this problem. Although the result is negative, it is still useful for the design of PG-GAT: on the same underlying keypoint graph, directly optimising the graph partition is less effective than learning a contrastive embedding and clustering that representation afterwards.

2.4.4 Graph partitioning and spectral methods

The classical alternative to embedding-based clustering is to partition the graph directly. Normalised cut [60] formulates this as a balanced partitioning problem using the graph Laplacian. The spectral clustering review by von Luxburg [61] describes the relaxations that make the original discrete problem tractable by using the eigenvectors of the Laplacian. A learned version of a related idea appears in the face-clustering work of Yang et al. [62, 18] - their approach uses a GCN to predict pairwise same-identity affinities, after which identities are recovered from connected components in a thresholded affinity graph.

The graph-partitioning head evaluated in Section 4.2 adapts the pair-feature construction of Yang et al.

to the pose embeddings used in this work. It is the third of the three learned grouping heads tested and gives the strongest performance on the synthetic validation data, although this improvement does not carry across to COCO. As described in Section 3.6.3, the binary edge classifier is sensitive to changes in the distribution of the embeddings - the synthetic-to-real domain gap introduces this type of shift and results in poorer transfer to real detections. The corresponding experimental results are reported in Section 4.2.

Importantly, the partitioning head uses the same contrastive embeddings as the k-means baseline, without changing or retraining the embedding network. This provides a useful control in the comparison: both methods receive the same learned representation, so the difference in their performance can be attributed more directly to the grouping head rather than to the quality of the underlying embedding.

2.4.5 Implication for the methodology

Of the four families reviewed in this section, contrastive embedding is the only approach retained in the final PG-GAT method. The three learned grouping heads - deep clustering, slot attention, and graph partitioning - are evaluated using the same backbone in the experimental chapter. The SA-DMoN variants are also tested as an alternative based more directly on graph structure, but none of these approaches improves on the simpler k-means read-out.

These results support the argument developed in the experimental chapter that the embedding itself has a greater effect on grouping performance than adding a more complex grouping head. This also narrows the design space considered for PG-GAT - the contrastive loss and k-means read-out are kept relatively simple, while the embedding network itself is modified to make better use of the structure available in human pose.

Section 2.5 continues this review by considering which embedding *space* is most appropriate for representing skeletons. Section 2.6 then brings the preceding literature together and defines the research gap addressed by the proposed method.

2.5 GEOMETRIC EMBEDDING SPACES

The previous section reviewed the loss functions and clustering methods that operate on the learned embedding, while assuming that the embedding itself lies in a Euclidean vector space. This section considers an alternative representation based on non-Euclidean geometry, with the focus specifically on hyperbolic embedding spaces. The main aim is to identify the property of hyperbolic geometry that motivates the PG-GAT variant evaluated in Section 4.7.

The scope is intentionally limited to hyperbolic geometry - other manifolds, including spherical, product, and mixed-curvature spaces, have also been explored in the wider representation-learning literature, but they are not evaluated in this work. The discussion therefore focuses on the aspects of hyperbolic geometry that are directly relevant to the experiments presented later.

2.5.1 Why an embedding space is a choice

In a standard contrastive-learning pipeline, the embedding space is usually treated as Euclidean. The network produces a vector in $\mathbb{R}^d$, similarity is measured using a Euclidean or cosine metric, and the resulting representations are clustered using Euclidean methods such as k-means. This is the conventional choice, but it is not the only possible one.

For data with a strong hierarchical or tree-like structure, Euclidean space can be a poor geometric match - the volume of a Euclidean ball grows polynomially with its radius, approximately $\mathcal{O}(r^d)$, while the number of nodes in a tree can grow exponentially with depth, approximately $\mathcal{O}(c^r)$ for a branching factor c. Representing this type of structure faithfully in Euclidean space can therefore require either a larger embedding dimension or some amount of distortion.

Human pose provides a natural reason to investigate this difference - a single skeleton can be represented as a kinematic tree rooted at the pelvis in the COCO joint topology, while a multi-person image contains several such structures. It is therefore reasonable to ask whether an embedding geometry better suited to tree-like data can represent the grouping problem more effectively than a standard Euclidean space. The hyperbolic PG-GAT variant described in Section 3.10 provides an experimental test of this idea.

2.5.2 Hyperbolic geometry for tree-like data

Hyperbolic space with curvature -1 has a useful property for representing tree-like structures: the volume of a ball grows exponentially with its radius - this is similar to the way the number of nodes in a tree increases with depth. Sarkar [68] formalised this relationship by showing that trees can be embedded in the hyperbolic plane with arbitrarily low distortion, whereas achieving similarly low distortion in Euclidean space can require the embedding dimension to increase with the size of the tree.

Sala et al. [69] later considered this property in the context of learned representations. Their analysis examined the trade-off between embedding dimension and distortion across several tree-structured datasets, showing that low-dimensional hyperbolic embeddings could represent these structures with distortion comparable to, or lower than, Euclidean embeddings using considerably more dimensions.

Two common representations of hyperbolic space are used in the deep learning literature: the Poincaré ball and the Lorentz, or hyperboloid, model. The Poincaré model represents the manifold inside an open Euclidean unit ball, with the metric increasing towards the boundary, while the Lorentz model represents the same underlying geometry in a higher-dimensional ambient space using a Minkowski inner product.

The Lorentz model is often more convenient for neural-network optimisation because it avoids some of the numerical difficulties that can occur close to the boundary of the Poincaré ball - Nickel and Kiela [70] showed that the Lorentz formulation allows stable optimisation for the embedding magnitudes encountered in practice. This motivates the use of the Lorentz model for the hyperbolic architecture described in Section 3.10.1.

2.5.3 Hyperbolic deep learning

There are several ways of combining hyperbolic geometry with deep learning. One of the simpler approaches is to keep the neural network itself Euclidean and make only the final embedding hyperbolic - the network produces a vector in the Euclidean tangent space, which is mapped onto the hyperbolic manifold using an exponential map. When a standard Euclidean operation is required later, the representation can be mapped back into the tangent space.

A more complete approach is to make the network itself hyperbolic, with linear transformations and aggregation operations replaced by their hyperbolic equivalents - for example using Möbius operations or Fréchet-mean aggregation. This keeps the representations on the manifold throughout the network, but also introduces considerably more implementation complexity.

The hyperbolic PG-GAT variant described in Section 3.10 follows the first approach - the message-passing layers remain Euclidean, while only the final embedding is projected onto the Lorentz hyperboloid. For the downstream k-means step, these embeddings are mapped back to the tangent space at the origin.

This provides a relatively small change to the original PG-GAT architecture and allows the effect of the hyperbolic output space to be tested separately. A fully hyperbolic attention network would require more substantial changes to the architecture, so the simpler variant provides a useful first test of whether hyperbolic geometry offers a measurable benefit for the grouping problem.

2.5.4 Two pre-registered hypotheses

The potential benefit of a hyperbolic embedding depends partly on the depth and branching structure of the data being represented. A COCO skeleton contains only seventeen joints and has a maximum tree depth of approximately four hops - at this relatively small scale, the theoretical advantage of hyperbolic geometry over a Euclidean representation may therefore be limited.

For this reason, two separate hypotheses are defined in Section 3.10.4 before the experiments are run. H1 tests whether a hyperbolic embedding can match or outperform the Euclidean model at the same output dimension, which would indicate an improvement in representation quality. H2 instead tests whether the hyperbolic model can achieve comparable performance using a substantially smaller embedding dimension, indicating an efficiency advantage. The results for both hypotheses are reported in Section 4.7.

Defining these hypotheses in advance is important because an advantage at lower embedding dimensions is different from an improvement in overall grouping accuracy - a small efficiency gain can still be a useful result even if the hyperbolic model does not outperform the Euclidean model at the same dimension. Treating these as separate hypotheses makes this distinction clear and allows both the positive and negative experimental outcomes to be reported against the criteria defined beforehand.

2.5.5 Alternative geometries not pursued

For completeness, two other families of non-Euclidean embeddings are worth noting, although neither is explored in the experimental chapter. The first is spherical embeddings, which are already partly represented by the L2-normalised output of the contrastive backbone and are therefore not treated as a separate experiment. The second includes product and mixed-curvature manifolds, which provide a much broader geometric design space but would also add considerably more implementation and evaluation complexity.

The hyperbolic variant is therefore used as the most direct non-Euclidean alternative to the standard Euclidean embedding - it requires relatively limited changes to the existing architecture while still testing whether a geometry better suited to hierarchical structure provides a useful advantage. The results of this experiment are reported in Section 4.7. Other manifold choices remain possible directions for future work if the results justify a more detailed investigation of embedding geometry.

2.6 RESEARCH GAP AND POSITION

The preceding sections have reviewed the literature surrounding multi-person pose grouping from several related perspectives. Sections 2.1 and 2.2 considered the broader bottom-up HPE landscape and the different mechanisms used to group detected keypoints. Section 2.3 then examined how graph neural networks have been applied to pose-related problems, while Section 2.4 reviewed representation-learning and clustering methods that could be used for grouping. Finally, Section 2.5 considered alternative embedding geometries that may improve the learned representation.

The aim of this review was not to cover every method in each of these areas - instead, the focus was on approaches that are directly relevant to the design choices made in this work, including methods that are adopted, evaluated experimentally, or excluded for specific reasons. This section brings these different areas together and defines the research gap addressed by the methodology in Chapter 3 and the experiments presented in Chapter 4.

2.6.1 Where the literature leaves the design space

The HPE literature now contains strong per-keypoint detectors, with localisation accuracy reaching relatively high levels on the COCO benchmark. This makes the grouping stage the main remaining limitation on performance in bottom-up systems. The four main grouping approaches reviewed earlier - PAFs, associative embeddings, mid-range offsets, and end-to-end set prediction - all connect the grouping mechanism closely to the detector, preventing the grouping stage from being reused with a different upstream detector.

Modular grouping, where the method operates on detached keypoint coordinates produced by an existing detector, is much less explored. The graph-based methods reviewed in this chapter provide the closest precedents for PG-GAT. HGG [13] is trained jointly with the detector and is therefore not modular, while the instance-aware method of Yang et al. [51] incorporates its grouping information into the detector backbone. GEC [14] is the closest detector-independent approach, but uses an edge-classification formulation - the experiments in Chapter 4 examine this type of formulation and show that it is sensitive to the synthetic-to-real transfer considered in this work.

The GNN literature on human pose provides much of the machinery used by PG-GAT, but has mostly applied it to other problems - examples include skeleton-based action recognition with ST-GCN [53] and 2s-AGCN [54], single-person 3D pose lifting with SemGCN [55] and GAST-Net [57], and hand-object pose estimation with HOPE-Net [58]. These methods generally assume that the keypoints have already been associated with a person and do not produce a per-keypoint identity representation, so the use of GNNs specifically for grouping keypoints in multi-person scenes remains relatively limited.

The representation-learning literature provides several possible alternatives to a simple clustering read-out. The approaches considered in this work include deep embedded clustering, slot attention, graph partitioning, and modularity-driven clustering - each provides a possible learned or graph-based alternative to k-means on top of the contrastive embedding. Chapter 4 evaluates these approaches for the pose-grouping problem and finds that none improves on the simpler k-means baseline, which motivates the decision to focus on improving the embedding network itself rather than increasing the complexity of the grouping head.

A second area of interest comes from the literature on non-Euclidean embedding spaces. Hyperbolic embeddings have been used successfully for data with hierarchical or tree-like structure, while the human skeleton can itself be represented as a small kinematic tree - this makes the geometry of the embedding space a reasonable design choice to investigate for pose grouping, although its usefulness for multi-person keypoint grouping remains largely unexplored. The hyperbolic PG-GAT experiments in Chapter 4 therefore examine this second aspect of the proposed embedding approach.

2.6.2 The contribution and how it maps to the experiments

PG-GAT is the embedding network proposed to address the gap identified above. It accepts detected keypoint coordinates from any upstream detector, processes them using GATv2 attention, and produces an embedding for each keypoint. A downstream k-means step then groups these embeddings into person identities. The method introduces three skeleton-aware modifications to the attention mechanism (Section 3.8), adding explicit structure based on the COCO skeleton topology rather than relying on a type-agnostic graph. The resulting embedding is the main component behind the results reported in

Chapter 4.

The remainder of the methodology and experimental work tests different parts of this design and several alternatives identified in the literature:

- Section 4.2 evaluates whether a learned grouping head can improve on k-means when using the same backbone - DEC, slot attention, and graph partitioning are tested, but none improves on the simpler read-out. This provides the experimental basis for focusing on the embedding rather than the grouping head.
- Section 4.3 evaluates ten SA-DMoN variants to determine whether changes to the modularity null model can make structural graph clustering suitable for this problem - none of the tested variants improves on the baseline, suggesting that the modularity objective is poorly matched to the keypoint graph used here.
- The isolation ablation in Section 4.4 examines the three skeleton-aware modifications individually and in combination - the complete PG-GAT configuration performs better than each single-modification variant, although the differences between individual variants remain within the variation observed across random seeds.
- The architecture and loss sweeps in Section 4.5 compare the selected PG-GAT configuration against alternatives found through Bayesian search - this checks that the reported performance is not dependent on a single manually selected architectural setting.
- Section 4.6 evaluates the COCO fine-tuning configuration, confirms the fine-tuning recipe used for the final model, and produces the checkpoint used for the headline result.
- The hyperbolic PG-GAT experiment in Section 4.7 tests the alternative embedding geometry against the two hypotheses defined in Section 2.5.4 - H2 receives partial support under synthetic-only training, but the advantage does not remain after COCO fine-tuning.
- The TriGAT experiment in Section 4.8 tests whether changing the basic graph unit from an individual joint to a skeleton triplet improves the learned representation - it does not, with the subsequent voting stage losing information needed for accurate grouping.
- PG-GAT v2, evaluated in Section 4.11, adds pre-trained ImageNet visual features to test whether appearance provides useful identity information beyond keypoint position and type - no improvement is observed despite the addition of 2.2×10^6 parameters.
- Finally, Section 4.9 integrates PG-GAT with HigherHRNet and evaluates the complete pipeline - this comparison measures the practical detector-independence gap between the modular PG-GAT

design and the jointly trained AE reference.

Together, these experiments test both the main PG-GAT design and the alternatives motivated by the literature reviewed in this chapter. Chapter 3 describes the resulting methodology and its design choices in detail, while Chapter 4 evaluates those choices experimentally and shows which parts of the proposed approach are supported by the results.

CHAPTER 3 METHODOLOGY

This chapter describes the methodology used to produce the results reported in the remainder of this work. It begins with two background sections that establish the main concepts used throughout the chapter. Section 3.1 introduces graph neural networks and develops the required concepts through to the GATv2 attention mechanism used by the proposed method. Section 3.2 then covers the clustering and learned grouping methods considered in the experiments, including *k*-means, DEC, slot attention, graph partitioning, and DMoN.

The problem formulation, dataset pipeline, and baseline are described in Sections 3.3–3.5 - together, these sections define the grouping task and establish the baseline against which the later methods are evaluated. The following sections then present the investigation in the order in which it developed. Three learned grouping heads (Section 3.6) and the SA-DMoN modularity approaches (Section 3.7) are first evaluated as alternatives to *k*-means. Their inability to improve on the *k*-means baseline motivates a change in direction - rather than increasing the complexity of the grouping head, the focus moves to improving the embedding network itself. This leads to the PG-GAT architecture presented in Section 3.8.

Three further variants examine different aspects of the PG-GAT design - these include the addition of visual features (Section 3.9), a hyperbolic embedding variant (Section 3.10), and a triplet-based representation (Section 3.11). Each tests whether additional information or a different representation can improve on the main PG-GAT approach.

The chapter concludes by describing the integration of PG-GAT into a complete pose-estimation pipeline in Section 3.12. Finally, Section 3.13 defines the evaluation protocol and the approach used for hyperparameter selection and comparison - these procedures provide the basis for the experimental results reported in Chapter 4.

3.1 PRELIMINARIES — GRAPH NEURAL NETWORKS

A human skeleton can be represented naturally as a graph, with the seventeen keypoints forming the nodes and the skeletal connections forming the edges. Graph neural networks (GNNs) are well suited to this representation because they produce an embedding for each node while also making use of the structure of the graph - this makes them a useful choice for pose grouping, where both joint type and the relationships between joints provide information about which person a keypoint belongs to.

This section introduces the message-passing framework that underlies modern GNN architectures. It then describes the graph convolutional network (GCN) as a standard spectral formulation before moving to graph attention networks (GATs). Finally, the section introduces GATv2, which addresses a limitation of the original GAT formulation and provides the attention mechanism used throughout the experiments in this chapter.

3.1.1 The message-passing framework

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph with N nodes and an edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. Each node $i \in \mathcal{V}$ has an initial feature vector $\mathbf{h}_i^{(0)} \in \mathbb{R}^{d_0}$, while an edge $(i, j) \in \mathcal{E}$ may also contain an edge feature $\mathbf{e}_{ij}$. The neighbourhood of node i is defined as $\mathcal{N}(i) = \{j : (i, j) \in \mathcal{E}\}$.

Most modern GNN architectures can be described using the general message-passing framework [71] - at each layer, a node receives information from its neighbours, combines these messages using a permutation-invariant operation, and uses the result to update its representation. Formally,

$$\mathbf{m}_{i \leftarrow j}^{(l+1)} = \mathrm{MSG}^{(l)}\left(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}\right), \tag{3.1}$$

$$\mathbf{h}_i^{(l+1)} = \mathrm{UPDATE}^{(l)}\left(\mathbf{h}_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \mathbf{m}_{i \leftarrow j}^{(l+1)}\right), \tag{3.2}$$

where $\bigoplus$ represents a permutation-invariant aggregation operation such as a sum, mean, or maximum. The functions $\mathrm{MSG}^{(l)}$ and $\mathrm{UPDATE}^{(l)}$ are learned functions at layer l - different choices for the message, aggregation, and update operations lead to different GNN architectures.

GCN (Section 3.1.2) uses linear messages with degree-normalised aggregation followed by a nonlinear update. GAT (Section 3.1.3) instead learns attention weights for the neighbouring nodes, allowing their contributions to depend on the features being processed. GraphSAGE [72] uses neighbourhood aggregation together with a feedforward update and was designed with inductive learning in mind.

The message-passing formulation highlights two properties that are important for the rest of this chapter. First, much of the inductive bias of a GNN comes from how neighbouring information is aggregated and weighted rather than from the general message-passing structure itself. Second, the number of layers determines how far information can travel through the graph - after L message-passing layers, a node can contain information from its L-hop neighbourhood. For the relatively small skeleton graphs considered here, where joints are separated by only a few hops, this also limits the amount of network depth required.

3.1.2 Graph convolutional networks (GCN)

The GCN architecture introduced by [73] is derived from a first-order approximation of spectral graph convolution. Given an adjacency matrix $A \in \mathbb{R}^{N \times N}$ and degree matrix $D = \mathrm{diag}(\sum_j A_{ij})$, the symmetrically normalised adjacency matrix with added self-loops is

$$\tilde{A} = \tilde{D}^{-1/2} (A + I) \tilde{D}^{-1/2}, \tag{3.3}$$

where $\tilde{D} = \mathrm{diag}(\sum_j (A+I)_{ij})$ is the degree matrix after the self-loops have been added. A GCN layer then updates the node representations according to

$$H^{(l+1)} = \sigma\left(\tilde{A} H^{(l)} W^{(l)}\right), \tag{3.4}$$

where $H^{(l)} \in \mathbb{R}^{N \times d_l}$ contains the node features at layer l, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a learned weight matrix, and $\sigma(\cdot)$ is a pointwise nonlinear activation function.

There are two properties of Equation (3.4) that make attention-based aggregation more suitable for the pose-grouping problem considered here. First, the relative edge weights are fixed once $\tilde{A}$ has been constructed - the contribution of a neighbour is therefore determined by the graph structure and node degrees rather than being learned from the features of the nodes involved.

Second, this treatment does not distinguish between the different semantic roles of skeleton connections - for example, information passed between the left shoulder and left elbow may have a different importance from information passed between the left shoulder and the head. A pose-grouping network should be able to learn these differences rather than assigning weights based only on graph degree - the attention-based aggregation described next provides a way to make each neighbour's contribution dependent on the features being processed.

3.1.3 Graph attention networks (GAT and GATv2)

GAT [74] replaces the fixed normalisation used by GCN in Equation (3.3) with a learned attention weight for each edge. For a pair of nodes (i, j), where $j \in \mathcal{N}(i)$, the unnormalised attention score is

$$e_{ij}^{\text{GAT}} = \text{LeakyReLU}\Big(\mathbf{a}^\top \left[W\mathbf{h}_i \,\|\, W\mathbf{h}_j\right]\Big), \tag{3.5}$$

where $W \in \mathbb{R}^{d' \times d}$ is a shared linear projection, $\mathbf{a} \in \mathbb{R}^{2d'}$ is a learned attention vector, and $[\cdot\|\cdot]$ denotes concatenation. The attention scores are normalised across the neighbourhood of node i using a softmax,

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \tag{3.6}$$

and the new node representation is calculated as an attention-weighted sum of the projected neighbouring features,

$$\mathbf{h}_i' = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W\mathbf{h}_j\right). \tag{3.7}$$

Multi-head attention repeats Equations (3.5)–(3.7) using independent parameters for each attention head - the resulting outputs are then either concatenated or averaged.

Although GAT allows neighbour contributions to be learned, its original attention formulation has an important limitation identified by [15] - since $\mathbf{a}^\top$ is applied after the transformed node features have been concatenated, the attention score can be written as

$$\mathbf{a}_1^\top W\mathbf{h}_i + \mathbf{a}_2^\top W\mathbf{h}_j, \tag{3.8}$$

where $\mathbf{a} = [\mathbf{a}_1; \mathbf{a}_2]$. For a fixed query node i, the first term does not depend on the neighbour j, so the relative ranking of neighbours by attention score is the same for every query node. Brody et al. [15] refer to this behaviour as *static attention*.

This limitation is particularly relevant to pose grouping, where the importance of a neighbouring keypoint should depend on the keypoint receiving the message - for example, the relationship between a left shoulder and a right shoulder may be useful when deciding whether they belong to the same person, while the same left shoulder should not necessarily receive similar importance when considered from a distant joint belonging to another person.

GATv2 [15] addresses the static-attention limitation by changing the order of operations in the attention function:

$$e_{ij}^{\text{GATv2}} = \mathbf{a}^\top \, \text{LeakyReLU}(W\left[\mathbf{h}_i \,\|\, \mathbf{h}_j\right]). \tag{3.9}$$

Placing the nonlinearity inside the interaction between the two node representations prevents the score from separating into independent query and neighbour terms, so the ranking of neighbouring nodes can depend on the query node itself. GATv2 provides this more expressive form of dynamic attention without increasing the parameter count relative to GAT [15].

For this reason, GATv2 is used as the attention primitive for the architectures evaluated in this chapter. References to GCN or the original GAT in Chapters 4 and 5 are therefore used for background or baseline comparisons rather than for the main working network.

3.2 PRELIMINARIES — CLUSTERING AND LEARNED GROUPING

Once an embedding has been produced, pose grouping becomes a clustering problem over a finite set of points in the embedding space. This section introduces the main clustering methods used throughout the remainder of the chapter.

The first is k-means, which provides the main clustering baseline and is also used as the read-out for the final PG-GAT model. Deep embedded clustering is then introduced as an approach that jointly learns a representation and its cluster assignments. Slot attention provides an iterative alternative in which features compete for assignment to a set of learned slots. Graph partitioning instead treats grouping as an affinity problem between pairs of nodes. Finally, Newman modularity and its differentiable DMoN formulation provide an approach based directly on the community structure of the graph.

Each of these methods is defined formally in the following subsections - this allows the later methodology to apply them directly to the pose embeddings without repeating their underlying formulation.

3.2.1 *k*-means

Given a set of points $\mathcal{Z} = \{\mathbf{z}_1, \dots, \mathbf{z}_N\} \subset \mathbb{R}^d$ and a target number of clusters k, k-means [75] aims to find cluster centres $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k \in \mathbb{R}^d$ and an assignment function $c : \{1, \dots, N\} \to \{1, \dots, k\}$ that minimise the total squared distance between each point and its assigned centre:

$$\mathcal{L}_{k\text{-means}}(\{\boldsymbol{\mu}_j\}, c) = \sum_{i=1}^{N} \|\mathbf{z}_i - \boldsymbol{\mu}_{c(i)}\|_2^2. \tag{3.10}$$

A standard way of solving this objective is Lloyd's algorithm [76] - it alternates between two steps: each point is first assigned to its nearest current cluster centre, and each centre is then updated to the mean of the points assigned to it. These steps are repeated until the assignments stop changing or the objective no longer improves. The algorithm converges to a local minimum of Equation (3.10), but the final solution can depend on how the initial centres are selected. The k-means++ method [77] improves this initialisation by selecting new centres with probability proportional to their squared distance from the nearest centre already chosen - this provides a $\mathcal{O}(\log k)$ approximation guarantee in expectation and generally gives a more stable starting point.

In this work, k-means is used both as the main clustering read-out and as a comparison point for the learned grouping methods - after an embedding has been produced by the baseline GAT, PG-GAT, hyperbolic PG-GAT, or TriGAT, the person assignments for each image are recovered using k-means with $k = K$. The value of K is either provided by an oracle or estimated from the embeddings using the procedure described in Section 3.12.4.

Before clustering, the embeddings are L2-normalised so that they lie on the unit hypersphere - under this normalisation, squared Euclidean distance is directly related to cosine similarity. The clustering objective in Equation (3.10) is therefore consistent with the cosine-based contrastive objective used to train the embedding network.

3.2.2 Deep embedded clustering (DEC)

Deep Embedded Clustering (DEC) [16] combines representation learning and clustering within a single differentiable objective. For a fixed number of clusters k, DEC maintains a set of cluster centres $\{\boldsymbol{\mu}_j\}_{j=1}^{k} \subset \mathbb{R}^d$ as learnable parameters in the same space as the input embeddings. Each embedding $\mathbf{z}_i$ is assigned to these clusters using a Student's t-distribution with one degree of freedom [78]:

$$q_{ij} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|_2^2\right)^{-1}}{\sum_{j'} \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|_2^2\right)^{-1}}, \tag{3.11}$$

where q_{ij} represents the soft assignment of embedding $\mathbf{z}_i$ to cluster j. From these assignments, DEC constructs a sharpened target distribution p_{ij} by squaring the assignment probabilities and renormalising them:

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'} q_{ij'}^2 / f_{j'}}, \quad f_j = \sum_i q_{ij}, \tag{3.12}$$

where f_j represents the soft frequency of cluster j - this sharpening increases the influence of confident assignments while the frequency term reduces the effect of clusters that already contain a large amount of probability mass.

Training minimises the Kullback–Leibler divergence between the sharpened target distribution and the current soft assignments:

$$\mathcal{L}_{\text{DEC}} = \sum_{i,j} p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right). \tag{3.13}$$

Because q_{ij} depends on both the embeddings and the cluster centres, gradients can be propagated into the centres as well as any upstream embedding network. DEC was originally introduced as a fully unsupervised method, meaning that no ground-truth cluster labels are used during training - instead, the model learns by repeatedly fitting its current soft assignments to the sharpened distribution derived from those same assignments.

Section 3.6.1 applies this formulation to the pose embeddings used in this work and considers why this unsupervised self-training behaviour is structurally inadequate for multi-person pose grouping.

3.2.3 Slot attention

Slot Attention [17] is an iterative competitive attention mechanism originally developed for unsupervised object-centric representation learning. Given a set of N input features $\{\mathbf{z}_i\}_{i=1}^{N} \subset \mathbb{R}^d$ and a fixed number of slots K, the method maintains slot vectors $\mathbf{s}_1, \ldots, \mathbf{s}_K \in \mathbb{R}^d$ - these are initialised by sampling from a learned Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$, with the distribution parameters learned together with the rest of the network.

At each of T iterations, the layer-normalised slots are projected into queries $\mathbf{q}_j = \mathbf{W}_q\mathbf{s}_j$, while the normalised input features are projected into keys $\mathbf{k}_i = \mathbf{W}_k\mathbf{z}_i$ and values $\mathbf{v}_i = \mathbf{W}_v\mathbf{z}_i$. Attention is then calculated using a softmax across the slots:

$$A_{ji} = \frac{\exp\left(\mathbf{q}_j^\top \mathbf{k}_i/\sqrt{d}\right)}{\sum_{j'} \exp\left(\mathbf{q}_{j'}^\top \mathbf{k}_i/\sqrt{d}\right)}, \tag{3.14}$$

where d is the dimension of the slot and feature vectors. Unlike standard attention, where the softmax is commonly applied across the input features, Slot Attention normalises across the competing slots - each input therefore contributes most strongly to the slot with which it has the greatest query-key agreement, while stronger assignment to one slot reduces its contribution to the others. This competition is what allows the slots to separate different parts of the input.

The attention weights are subsequently normalised over the inputs and used to combine the value vectors into an update for each slot. A gated recurrent unit (GRU) [79] combines this update with the previous slot state - the GRU controls how much of the new information is incorporated at each iteration, allowing the slot representations to develop gradually rather than being replaced after each attention step. After T iterations, the final slot representations are compared with the input features to obtain per-input, per-slot scores, with the highest scoring slot providing the final hard assignment.

For the pose-grouping experiments in this work, the Slot Attention head is trained using Hungarian-matched cross-entropy as described in Section 3.6.2. This differs from the unsupervised reconstruction objective used in the original Slot Attention formulation - the original method uses a decoder to reconstruct the input image from the learned slots, whereas no such decoder is required for pose grouping. The reconstruction objective is therefore replaced with direct supervision from the ground-truth person labels - this keeps the competitive attention mechanism while adapting its training signal to the grouping task considered here.

3.2.4 Graph partitioning by edge classification

Graph partitioning approaches clustering differently from methods that predict cluster assignments directly - instead, a classifier predicts whether pairs of nodes belong to the same cluster, and the final partition is recovered from the resulting affinity graph. A classical formulation of graph partitioning is the normalised cut [60], which seeks a partition $(\mathcal{V}_1, \dots, \mathcal{V}_K)$ that minimises

$$\mathrm{NCut}(\mathcal{V}_1, \dots, \mathcal{V}_K) = \sum_{k=1}^{K} \frac{\mathrm{cut}(\mathcal{V}_k, \overline{\mathcal{V}_k})}{\mathrm{vol}(\mathcal{V}_k)}, \tag{3.15}$$

where $\mathrm{cut}(\mathcal{V}_k, \overline{\mathcal{V}_k})$ is the sum of the edge weights crossing the boundary of cluster $\mathcal{V}_k$, and $\mathrm{vol}(\mathcal{V}_k) = \sum_{i \in \mathcal{V}_k} d_i$ is its volume. The discrete normalised-cut problem is computationally difficult, but a spectral relaxation allows an approximate solution to be obtained from the eigenvectors of the graph Laplacian [61].

The learned graph-partitioning method used in this work is based on [18] - rather than solving the spectral problem directly, it learns an affinity score between pairs of nodes. For two input embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$, a pair feature is constructed using their difference and element-wise product, providing information about both the relative position of the two embeddings and dimensions in which they respond similarly. A multi-layer perceptron then maps the pair representation to a scalar same-cluster logit. Training uses binary cross-entropy with the ground-truth same-person or different-person label.

During inference, the predicted affinities are thresholded to form a binary graph. Person groups are then recovered as connected components, implemented using a union-find procedure. The adaptation to pose grouping is described in Section 3.6.3. An important difference from DEC and slot attention is that the number of clusters K does not have to be supplied to the partitioning step - instead, the number of people is determined by the number of connected components produced by the predicted affinity graph.

3.2.5 Modularity and deep modularity networks (DMoN)

The clustering methods above operate primarily on learned embeddings. Modularity-based methods instead make direct use of the structure of the graph. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with adjacency matrix $A \in \{0,1\}^{N\times N}$ and node degrees $d_i = \sum_j A_{ij}$, Newman modularity [59] measures how much more connectivity exists within the proposed communities than would be expected under a random-graph baseline.

For a partition represented by $\mathbf{S} \in \{0,1\}^{N\times K}$, where $S_{ik} = 1$ when node i belongs to cluster k, modularity is defined as

$$Q(\mathbf{S}) = \frac{1}{2m}\mathrm{Tr}\big(\mathbf{S}^\top \mathbf{B}\mathbf{S}\big), \quad \mathbf{B} = \mathbf{A} - \frac{\mathbf{d}\mathbf{d}^\top}{2m}, \tag{3.16}$$

where $m = \frac{1}{2}\sum_{ij} A_{ij}$ is the total number of edges. The term $\mathbf{d}\mathbf{d}^\top/(2m)$ forms the *null model*, which gives the expected connectivity between two nodes under a random graph that preserves the degree sequence - a useful community therefore contains more internal connectivity than would be expected under this baseline.

The choice of null model is important for the pose-grouping problem - a degree-based model is well suited to graphs where connectivity itself carries useful structural information, such as co-authorship or protein-protein interaction networks. The assumption is misaligned with the kNN keypoint graph used in this work (Section 3.3.2), where edges are created mainly from spatial proximity - two nearby keypoints can have a strong graph connection even when they belong to different people.

DMoN [19] makes the modularity objective differentiable by replacing the discrete indicator matrix with a soft assignment matrix $\mathbf{S} \in [0,1]^{N\times K}$ - these assignments are produced by a GNN encoder followed by a per-node softmax over the K clusters. Regularisation terms are then added to discourage degenerate partitions. The resulting objective is

$$\mathcal{L}_{\text{DMoN}} = -\frac{1}{2m}\mathrm{Tr}\big(\mathbf{S}^\top \mathbf{B}\mathbf{S}\big) + \lambda_{\text{ortho}} \left\| \frac{\mathbf{S}^\top \mathbf{S}}{\|\mathbf{S}^\top \mathbf{S}\|_F} - \frac{\mathbf{I}}{\sqrt{K}} \right\|_F + \lambda_{\text{cluster}} \left(\frac{\sqrt{K}}{N} \|\mathbf{s}_{\text{vol}}\|_2 - 1 \right), \tag{3.17}$$

where the first term maximises modularity, the orthogonality term encourages separation between the learned clusters, and the collapse-prevention term discourages empty or degenerate cluster assignments through the cluster-volume vector $\mathbf{s}_{\text{vol}} = \mathbf{S}^\top \mathbf{1}_N$ - the coefficients λ_{ortho} and λ_{cluster} control the relative contribution of these regularisation terms.

DMoN was investigated after the learned grouping heads of Sections 3.2.2–3.2.4 failed to improve on k-means. Unlike those methods, DMoN uses the graph topology itself as the main clustering signal rather than relying only on the geometry of the learned embedding. Section 3.7 describes the ten SA-DMoN variants used to test whether modifying the null model makes this objective better suited to pose grouping - the corresponding experiments in Chapter 4 show that none of the tested modifications provides a useful improvement on the kNN keypoint graph.

3.3 PROBLEM FORMULATION

This section defines the multi-person pose grouping problem and introduces the notation used throughout the remainder of the chapter. The work follows the *bottom-up* approach to multi-person pose estimation, where keypoints are first detected across the complete image and are then grouped into individual person skeletons - this differs from *top-down* methods, which first detect each person using a bounding box and then apply a single-person pose estimator within each detected region.

Bottom-up methods are particularly useful in crowded scenes, where person bounding boxes can overlap and keypoints from nearby people may be difficult to separate using the detection boxes alone. The focus of this work is specifically on the grouping stage that follows keypoint detection - an off-the-shelf single-image keypoint detector [3] produces a set of two-dimensional keypoint detections, with each detection assigned one of the seventeen COCO joint types [1]. The grouping problem is then to partition these detections into separate person skeletons without requiring access to the original image.

Keeping detection and grouping separate is the central design choice in this work - it allows the grouping method to operate on the outputs of an existing detector rather than depending on its internal feature maps. This differs from more tightly integrated approaches such as DEKR [8] and ED-Pose [12], where detection and grouping are learned as part of the same architecture - the practical consequences of this separation are considered further in Section 3.12.

3.3.1 Inputs and outputs

Consider an image containing K people. The detector produces N keypoint detections, collected into the set

$$\mathcal{X} = \{(\mathbf{p}_i, t_i, v_i)\}_{i=1}^{N}, \tag{3.18}$$

where $\mathbf{p}_i \in [0,1]^2$ is the normalised image-plane position of detection i, $t_i \in \mathcal{T} = \{1,\dots,T\}$ is its joint type, with $T = 17$ for COCO, and $v_i \in \{0,1\}$ is a visibility flag indicating whether a confident keypoint was detected at that location. The grouping task is to predict a labelling

$$\hat{\mathbf{y}} \in \{1,\dots,K\}^N, \tag{3.19}$$

where each detection is assigned to one of the K people in the image - two detections i and j should therefore receive the same label if and only if they belong to the same person.

During training, the ground-truth person assignments $\mathbf{y}^\star$ are known. At inference time, however, the number of people K may also be unknown and must be estimated using the procedure described in Section 3.12.4. The main evaluations use an oracle value for K so that grouping performance can be measured separately from errors introduced by estimating the number of people - the fully autonomous pipeline, where K is estimated at inference time, is described in Section 3.12 and evaluated separately.

3.3.2 The skeleton graph

The grouping problem is represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ constructed from the detected keypoints. Each node $i \in \mathcal{V}$ corresponds to one detection in $\mathcal{X}$, giving $|\mathcal{V}| = N$. A detection is represented by a continuous feature vector $\mathbf{x}_i$, whose exact contents depend on whether the depth feature is included - Section 3.4.3 defines both versions.

The input to the first layer of the embedding network combines these continuous features with a learned representation of the joint type:

$$\mathbf{h}_i^{(0)} = \left[\mathbf{x}_i \,\middle\|\, \mathrm{emb}_{\mathcal{T}}(t_i)\right] \in \mathbb{R}^{d_0}, \tag{3.20}$$

where $\mathrm{emb}_{\mathcal{T}} : \{1,\dots,T\} \to \mathbb{R}^{d_t}$ is a learnable lookup table for the seventeen COCO joint types, with $d_t = 32$ in the baseline configuration and $d_t = 64$ in the sweep-selected architecture used for the headline results - keeping the type embedding separate from the spatial features allows the network to learn the meaning of a joint type independently of its position in the image. A nose and a left knee, for example, remain different joint types regardless of where they are detected.

Edges are constructed using a k-nearest-neighbour rule based on image-plane distance - this gives each node a local neighbourhood while avoiding a fully connected graph in which every keypoint attends to every other keypoint. The neighbourhood size depends on the embedding dimension - configurations with output dimensions below 256, including the baseline and the models used before the architecture sweep, use $k = 8$. The 256-dimensional architecture selected by the sweep and used for the headline results uses $k = 16$ - this larger neighbourhood was introduced together with the larger output dimension.

For each configuration, the same value of k is used during training and evaluation, avoiding a train–test difference in graph construction. The neighbourhood size itself was not independently included in the hyperparameter sweep, which is noted as a limitation in Chapter 6. The resulting kNN edge set is directed and contains no self-loops - each node connects to its k nearest neighbours, and an edge is reciprocated only when two nodes each select the other as a neighbour.

Two properties of this graph are important for the methodology that follows. First, its edges represent *spatial proximity*, not person identity - two nearby keypoints can therefore be connected even when they belong to different people. For example, two left-shoulder detections from overlapping people may form an edge despite requiring different final person assignments, as illustrated in Figure 3.1. The embedding network must learn to resolve these cases rather than treating every graph edge as evidence of a shared identity.

Second, the joint type t_i is known for every node - this means that the network can make use of structural information about the human skeleton without requiring any additional information from the detector at inference time. The PG-GAT modifications introduced in Section 3.8 are designed specifically to make use of this observed joint-type information.

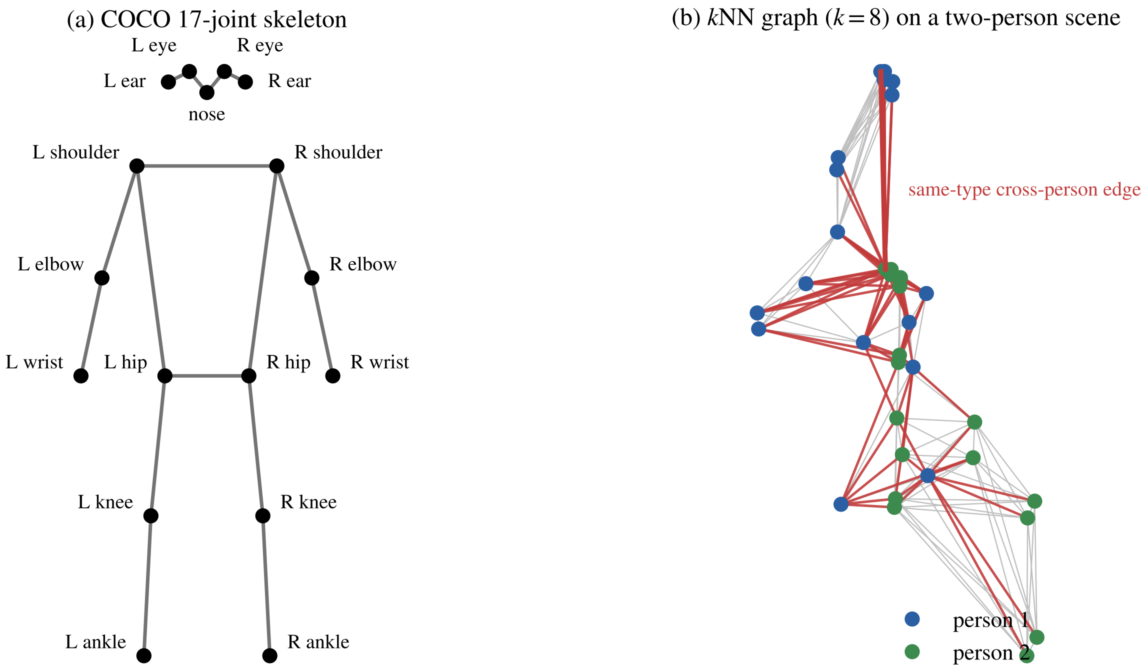

Figure 3.1. The COCO 17-joint skeleton (a) and a representative kNN graph with $k = 8$ on a two-person synthetic scene (b). Nodes represent joint detections. Skeleton edges in (a) represent anatomical connections, while the kNN edges in (b) represent spatial proximity in the image plane. Cross-person edges are shown in red and within-person edges in grey. The highlighted same-type cross-person edge shows an example that the embedding stage of Section 3.5 must learn to separate.

3.3.3 The three sub-problems

The pose-grouping problem is divided into three main parts: learning an embedding, grouping the resulting embeddings, and estimating the number of people in the image.

Embedding: The first task is to produce a representation $\mathbf{z}_i \in \mathbb{R}^d$ for every detected keypoint - the embedding should place keypoints belonging to the same person close together while keeping keypoints from different people separated. During training, a contrastive loss uses the ground-truth person labels to pull or push pairs of embeddings according to whether they belong to the same person. The embedding network is the main component varied throughout the experiments, with PG-GAT, SA-DMoN, hyperbolic, and TriGAT configurations changing only how these representations are produced.

Grouping: Given the embeddings $\{\mathbf{z}_i\}$ and the number of people K, the next task is to recover the person assignments $\hat{\mathbf{y}}$. The headline read-out used in this work is k-means with $k = K$, applied to the L2-normalised embeddings - this restriction is deliberate. The grouping problem is treated as an unconstrained clustering problem in which most of the discriminative work is expected to happen in the embedding network, with the clustering stage providing a simple read-out. Grouping methods that apply additional structural constraints during this final stage are left as a direction for future work in Chapter 6.

K estimation: The final task is to estimate the number of people K directly from the learned representations, allowing the complete pipeline to operate without an oracle value. Section 3.12.4 describes a small MLP head trained for this purpose - jointly training this head with the embedding network reduces the quality of the learned embedding. The K-estimation head is therefore trained separately using a frozen embedding network.

3.3.4 Evaluation metric

The primary evaluation metric used in this work is Pose Grouping Accuracy (PGA). PGA measures the proportion of keypoints assigned to the correct person after matching the predicted cluster labels to the ground-truth person labels - this matching step is required because the cluster identifiers produced by k-means have no fixed meaning. For example, predicted cluster 1 may correspond to ground-truth person 2, without the clustering itself being incorrect.

The correspondence between predicted clusters and ground-truth people is found using the Hungarian algorithm [80] - the Hungarian algorithm solves the assignment problem in polynomial time by finding the one-to-one mapping between two sets that minimises a given cost. Here, it is applied to the overlap between predicted clusters and ground-truth person identities, giving the relabelling that produces the greatest agreement between them.

Given predicted labels $\hat{\mathbf{y}}$ and ground-truth labels $\mathbf{y}^\star$, let $\pi^\star : \{1,\dots,K\} \to \{1,\dots,K\}$ denote the optimal cluster-to-person mapping obtained through Hungarian assignment [80]. PGA is then defined as

$$\mathrm{PGA}(\hat{\mathbf{y}},\mathbf{y}^\star) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\pi^\star(\hat{\mathbf{y}}_i) = \mathbf{y}_i^\star], \tag{3.21}$$

where $\mathbb{1}[\cdot]$ is the indicator function - the resulting score is simply the fraction of detected joints assigned to the correct person after the cluster labels have been aligned.

The same PGA calculation is used for both synthetic and COCO experiments, so results across the two settings use a consistent grouping metric - what changes is the distribution of the input data, rather than the definition of the evaluation measure. Further details on the metric, the datasets used for each evaluation, and the W&B sweep procedure used for hyperparameter selection are provided in Section 3.13.

3.4 DATASET PIPELINE

Every method in this chapter is trained using synthetic data with known person labels and is evaluated on both synthetic and COCO data - using the two data sources makes it possible to measure performance on the distribution used for training, while also testing how well the learned grouping behaviour transfers to real images.

This section describes the synthetic data generator, the adapter used for COCO, and the shared preprocessing pipeline that converts both data sources into the same input format. It also explains the decision to remove the depth channel before the main experiments were carried out, so that the headline results use the same final input representation.

3.4.1 Synthetic generator

The synthetic dataset is generated using Blender [81]. A configurable number of mannequin meshes is placed into each 3D scene, with the camera position, viewing angle, and lighting varied procedurally between scenes. Each generated sample contains a colour image together with per-person annotations for the seventeen COCO keypoints - a joint is stored in the form $[x,y,z,v]$, where (x,y) gives its image-plane position, z is the Euclidean distance from the camera to the joint in metres, and $v \in \{0,1,2\}$ gives its visibility state. Camera pose, lighting, person placement, and joint visibility are varied by the synthetic generator - exact joint depth is also available from the generated 3D scene, although this feature is removed from the final input representation as described in Section 3.4.4.

Visibility is determined using a Blender raycast against the scene geometry - this allows a joint that is present but occluded by another person or an object to be marked as $v = 1$, rather than treating it as missing with $v = 0$. Joints that fall outside the image use the sentinel coordinate $(-1,-1)$ - this avoids confusing an out-of-frame joint with the valid image coordinate $(0,0)$.

The final synthetic dataset contains 400 training scenes, 100 validation scenes, and 100 test scenes. The number of people in each scene is sampled uniformly from $K \in \{1,2,3,4,5\}$ - this ensures that the K-estimation model is trained across the full range of person counts that it is expected to predict at inference time. It also gives the grouping models examples with different levels of crowding rather than concentrating the training data around a single person count - representative scenes are shown in Figure 3.2.

Files follow the naming convention `<guid>_<n>.png` and `<guid>_<n>.json`, where `<n>` records the number of people in the scene - this allows samples to be filtered by person count without first reading the annotation file. The same value is also stored as `num_people` in the image metadata for use when calculating dataset statistics.

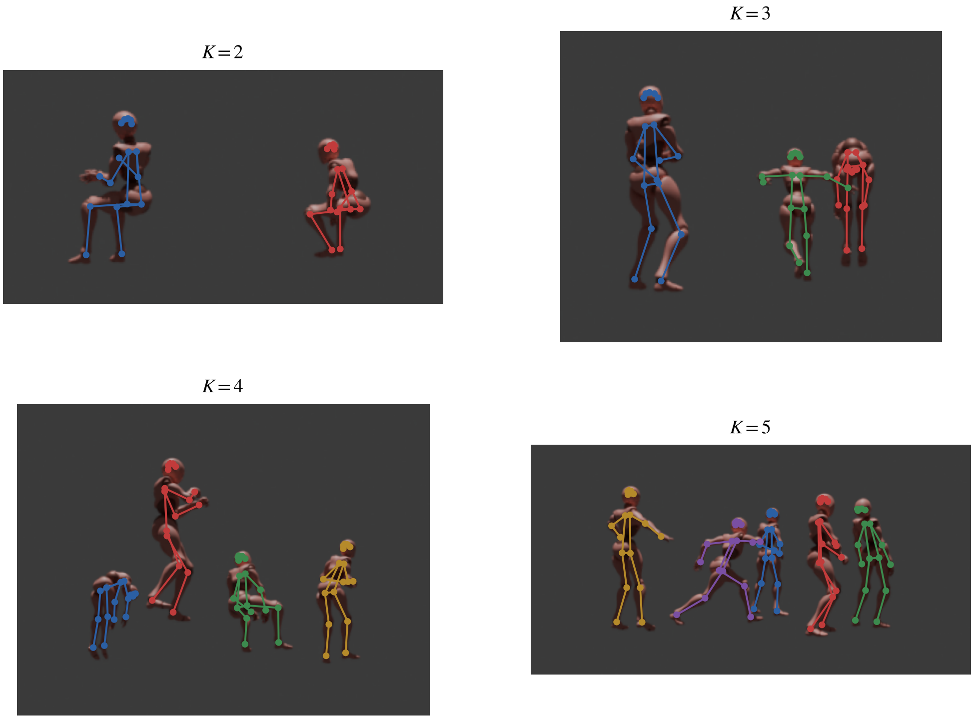

Figure 3.2. Representative scenes from the synthetic dataset for person counts $K \in \{2,3,4,5\}$. Each panel shows a rendered Blender image with the seventeen ground-truth keypoints overlaid and colour-coded by person identity.

3.4.2 COCO adapter

For evaluation on real images, the COCO 2017 person-keypoint annotations [1] are loaded using the `pycocotools` API. Images containing at least one annotated person are retained. COCO uses three visibility values ($0 =$ not labelled, $1 =$ labelled but occluded, and $2 =$ visible), which are mapped directly to the same visibility convention used by the synthetic dataset.

Unlike the synthetic data, COCO does not provide depth information - an initial version of the data pipeline therefore estimates depth using MiDaS [82], a pre-trained monocular depth model that produces a relative depth map from a single RGB image. The DPT-Hybrid variant is used as a compromise between model capacity and inference cost - the model is loaded once when the COCO adapter is created, and the estimated depth value at each annotated keypoint location is sampled from the resulting depth map and stored in the z component of the keypoint vector.

This gives each COCO keypoint the same $[x, y, z, v]$ representation as the synthetic data, allowing both sources to pass through the same downstream preprocessing pipeline - the important difference is that the synthetic z value is obtained directly from the 3D scene, whereas the COCO value is an estimated relative depth.

Depth was initially retained in this way to avoid changing the network input between synthetic training and COCO evaluation - however, further experiments showed that the depth channel was not suitable for the final setup. Section 3.4.4 describes this analysis and the resulting decision to remove depth from all methods reported in the main experiments.

3.4.3 Unified pipeline

Both dataset adapters return the same per-image structure: an image tensor with shape $[3, H, W]$, a list of per-person keypoint tensors with shape $[17, 4]$, an image identifier, and the number of people in the image. A shared `PoseDataset` is used for both data sources so that the same preprocessing steps are applied regardless of whether a sample comes from the synthetic generator or COCO.

Each image is resized to fit within `target_size` $\times$ `target_size` while preserving its original aspect ratio - it is then padded symmetrically to reach the exact target dimensions. The corresponding keypoint coordinates are scaled and shifted by the same amounts so that they remain aligned with the transformed image - the z and v values are left unchanged, and joints with $v = 0$ are not transformed because they do not represent a valid image location.

A custom collate function is used by the dataloader - image tensors can be stacked normally, but the keypoint annotations are kept as variable-length per-image lists. This is necessary because different images can contain different numbers of people, which means the annotation tensors cannot be handled directly by PyTorch's standard batching behaviour.

The preprocessor then converts each image in the batch into a PyTorch Geometric graph [83] - joints with $v = 0$ are removed before graph construction rather than being included as nodes with empty or zero-valued features. This prevents missing joints from contributing artificial messages during graph propagation. Each remaining joint becomes a node with the raw preprocessed feature vector

$$\mathbf{x}_i = [x_{\text{norm}},\, y_{\text{norm}},\, z_{\text{norm}},\, v_{\text{norm}}] \in \mathbb{R}^4, \tag{3.22}$$

where x_{norm} and y_{norm} are normalised to $[0,1]$ using the image dimensions - the depth value z_{norm} is min-max normalised over the valid joints within each graph, while v_{norm} maps the three visibility values to $\{0.0, 0.5, 1.0\}$.

The per-graph depth normalisation places both the Blender depth and the MiDaS-estimated COCO depth into the same numerical range, even though their original scales have different meanings. The resulting $\mathbf{x}_i$ is the preprocessed feature vector used throughout the rest of the chapter - the actual layer-zero GAT input $\mathbf{h}_i^{(0)}$, defined in Equation (3.20), concatenates the learned joint-type embedding with this vector.

Each graph also stores the joint-type indices separately for use by the type embedding, together with the ground-truth person labels required by the contrastive loss during training. Graph edges are constructed using k-nearest neighbours in $(x_{\text{norm}}, y_{\text{norm}}, z_{\text{norm}})$ space - as described in Section 3.3.2, $k = 8$ is used for output dimensions below 256, while the 256-dimensional configuration uses $k = 16$.

3.4.4 Removing the depth channel

The depth channel is removed from the methods used in the main evaluations - this decision follows from a diagnostic experiment reported in Chapter 4, and the reasoning is described here because it affects the input representation used throughout the rest of the methodology.

When depth is included, k-means clustering on the contrastive GAT embeddings reaches a synthetic-validation PGA of 0.9990 - at this point the synthetic grouping problem is almost completely solved. The Blender renderer provides exact metric depth, so people standing at different distances from the camera can often be separated using the z coordinate with very little additional work required from the GAT - this produces an unrealistically strong synthetic baseline and leaves little room to determine whether the learned grouping heads or SA-DMoN provide a meaningful improvement.

The situation is different for COCO - since COCO does not contain ground-truth depth, the z value is estimated using MiDaS [82]. MiDaS provides relative rather than metric depth, and its scale can vary between images - the depth channel therefore introduces an additional difference between the synthetic training data and the real COCO data. A model trained using exact Blender depth may learn relationships that do not transfer cleanly to the estimated depth values available on COCO.

For this reason, depth is removed from the main experiments - this also has the practical advantage of making the grouping method depend only on information that is already available from a standard 2D keypoint detector, without requiring an additional depth-estimation model.

The change is controlled by a `use_depth` flag in the preprocessor configuration. When depth is disabled, the feature vector from Equation (3.22) becomes

$$\mathbf{x}_i = [x_{\text{norm}},\ y_{\text{norm}},\ v_{\text{norm}}] \in \mathbb{R}^3, \tag{3.23}$$

and the kNN graph is constructed using only $(x_{\text{norm}}, y_{\text{norm}})$ - the input dimension of the first GAT layer is reduced accordingly through Equation (3.20). All results in Chapter 4 from the no-depth baseline onwards use this configuration unless a result is explicitly identified as a depth-enabled diagnostic.

3.5 BASELINE: STANDARD GAT WITH CONTRASTIVE LOSS

The baseline used for all later comparisons is a standard graph attention network trained on the synthetic dataset using a contrastive loss - at inference, the output embeddings are L2-normalised and grouped using k-means. This gives a relatively simple starting point and sets the bar that any change to the embedding network or grouping method must clear.

The baseline also fixes the main training and evaluation procedure used throughout the experiments - unless stated otherwise, later methods inherit the same optimiser, training schedule, and clustering protocol. This keeps the comparisons consistent and means that improvements can be linked more directly to the architectural or methodological change being tested, rather than to changes in the surrounding training setup.

3.5.1 Architecture

The baseline network takes the graph $\mathcal{G}$ constructed in Section 3.3.2 and produces an embedding $\mathbf{z}_i \in \mathbb{R}^d$ for each node, with $d = 128$. The input to the first GAT layer is the representation $\mathbf{h}_i^{(0)}$ defined in Equation (3.20) - in the default no-depth configuration, this combines the three preprocessed features $\mathbf{x}_i \in \mathbb{R}^3$ from Equation (3.23) with a learned 32-dimensional joint-type embedding. The resulting input dimension is therefore $d_0 = 35$ - for the earlier depth-enabled diagnostic, the four features of Equation (3.22) give $d_0 = 36$.

Message passing is performed by two GATv2 layers [15], each using four attention heads and a hidden dimension of 64. GATv2 is used instead of the original GAT [74] because its attention mechanism allows the ranking of neighbouring nodes to depend on the query node - as discussed in Section 3.1.3, the original GAT is limited by its static attention formulation, while GATv2 changes the order of the linear projection and nonlinearity to produce dynamic attention [15].

This distinction is useful for pose grouping because not every neighbour should contribute in the same way - a keypoint may need to treat a nearby joint of the same type differently from an anatomically related joint of another type. The importance of a neighbour therefore depends on both nodes involved in the interaction - for this reason, GATv2 is kept as the attention primitive throughout the methods developed in this chapter.

After the two message-passing layers, a linear projection maps each node representation to the 128-dimensional embedding space - the output is then L2-normalised, placing each embedding on the unit hypersphere $\mathbb{S}^{d-1}$. This matches the cosine-based contrastive objective used during training and also gives the downstream k-means stage a bounded representation space. The complete baseline architecture is summarised in Table 3.1.

Component	Setting
Attention primitive	GATv2 [15]
Number of GAT layers	2
Number of attention heads	4
Hidden dimension	64
Output (embedding) dimension	128
Joint-type embedding dimension	32
Layer normalisation	yes
Output normalisation	L2 (unit hypersphere)
Dropout	0.1

Table 3.1. Baseline GAT architecture used as the comparison point for the methods evaluated in this chapter - methods inherit these settings unless a parameter is explicitly changed or included in a later sweep. The architecture sweep used to select the final PG-GAT configuration is described in Section 3.8.5.

3.5.2 Contrastive loss

The embedding network is trained using a margin-based contrastive loss [64] defined on cosine similarity. For a graph containing N keypoints with ground-truth person labels $\mathbf{y}^\star \in \{1,\ldots,K\}^N$, the cosine similarity between two L2-normalised embeddings is

$$\cos(\mathbf{z}_i, \mathbf{z}_j) = \mathbf{z}_i^\top \mathbf{z}_j. \tag{3.24}$$

Training pairs are divided into two sets - positive pairs contain keypoints belonging to the same person,

$$\mathcal{P}^+ = \{(i,j) : \mathbf{y}_i^\star = \mathbf{y}_j^\star,\ i \neq j\}, \tag{3.25}$$

while negative pairs contain keypoints belonging to different people,

$$\mathcal{P}^- = \{(i,j) : \mathbf{y}_i^\star \neq \mathbf{y}_j^\star\}. \tag{3.26}$$

The contrastive objective is then

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{|\mathcal{P}^+|} \sum_{(i,j)\in\mathcal{P}^+} \big(1 - \cos(\mathbf{z}_i, \mathbf{z}_j)\big) \ + \ \frac{1}{|\mathcal{P}^-|} \sum_{(i,j)\in\mathcal{P}^-} \max\big(0,\ \cos(\mathbf{z}_i, \mathbf{z}_j) - \gamma\big). \tag{3.27}$$

The first term pulls embeddings belonging to the same person towards a cosine similarity of 1 - positive pairs keep contributing to the loss however close they already are. The second term penalises different-person pairs whose similarity exceeds the margin γ - once a negative pair has been pushed below γ it no longer contributes, which keeps the gradient focused on the negative pairs that are still too close together.

The margin is set to $\gamma = 0.5$ for the experiments in this work - this value is later tested as part of the 24-run Bayesian loss sweep described in Section 3.8.5, which confirms it as near-optimal. The same contrastive formulation is kept across the main embedding experiments, allowing changes in performance to be attributed more directly to the network architecture rather than to a different training objective.

The symbol γ is used for the contrastive margin throughout this chapter - this avoids confusion with m, which is already used in Section 3.2.5 to denote the total number of graph edges.

3.5.3 Training schedule and grouping read-out

The baseline model is trained for 50 epochs using the 400-scene synthetic training split with a batch size of 4. Optimisation uses AdamW [84] with an initial learning rate of 10^{-3} and weight decay of 10^{-4} - a cosine-annealing schedule [85] reduces the learning rate to 10^{-5} over the course of training. Gradient norms are clipped at 1.0 to reduce the effect of occasional large updates, particularly from batches containing more densely populated scenes.

The model is evaluated every five epochs on the 100-scene synthetic validation split using the PGA metric defined in Equation (3.21) - rather than using the final training epoch automatically, the checkpoint with the highest validation PGA is kept for evaluation.

At inference, the trained network is applied once to each image to produce the set of node embeddings $\{\mathbf{z}_i\}_{i=1}^N$ - these embeddings are already L2-normalised and are grouped using k-means with $k = K$. The scikit-learn implementation [86] is used with ten random initialisations and a fixed random seed - the resulting cluster labels are matched to the ground-truth person labels using the Hungarian assignment, after which PGA is calculated using Equation (3.21).

Chapter 4 reports two versions of this baseline - the first is the depth-enabled configuration using the four-dimensional node features of Equation (3.22). This is kept as a diagnostic result because, as discussed in Section 3.4.4, exact synthetic depth makes the grouping task unusually easy - the second is the no-depth configuration using Equation (3.23). This becomes the main baseline for the experiments that follow.

Unless stated otherwise, the methods from Section 3.6 onward inherit this no-depth setting together with the 50-epoch training schedule and optimiser configuration described above - the architecture settings in Table 3.1 are the manually selected baseline values used by the learned-head and SA-DMoN experiments.

PG-GAT later performs a separate Bayesian architecture search, described in Section 3.8.5, which selects a larger three-layer network with a 256-dimensional output - the two-layer baseline is therefore the bar that any modification of the embedding network or grouping head must clear at matched compute, rather than the final architecture used by PG-GAT.

3.6 LEARNED GROUPING HEADS

Three learned grouping heads are added to the baseline GAT embedding network described in Section 3.5 - each tests a different way of making the grouping stage itself learnable, rather than relying on k-means to recover the structure already present in the embedding space.

The general formulation of each method is introduced in Section 3.2 - this section focuses on how the three heads are adapted for pose grouping, how they are trained together with the GAT embedding network, and how their final person assignments are obtained at inference time.

The three approaches provide alternatives to the simple k-means read-out, but none produces an improvement in the main evaluations reported in Chapter 4 - these results change the direction of the investigation. Rather than adding further complexity to the grouping stage, the later experiments focus on improving the embedding itself, leading to the PG-GAT architecture introduced in Section 3.8.

3.6.1 Deep embedded clustering on pose embeddings

The DEC head [16] treats the final grouping step as an unsupervised clustering problem on top of the GAT embeddings. During training, it uses K cluster centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^d$, where K is the number of people in the current scene - since the person count changes between images, a single fixed set of cluster centres cannot be used across the whole dataset. Instead, the centres are initialised separately for each scene using k-means++ [77] on the current GAT embeddings - this is the main adaptation needed to apply DEC to the pose-grouping setting, where the required number of clusters varies from one image to another.

Given embeddings $\{\mathbf{z}_i\}$ and cluster centres $\{\boldsymbol{\mu}_k\}$, DEC first calculates a soft assignment using a Student's t-distribution with one degree of freedom [78]:

$$q_{ik} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_k\|_2^2\right)^{-1}}{\sum_{k'} \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{k'}\|_2^2\right)^{-1}}, \tag{3.28}$$

where q_{ik} represents the soft assignment of keypoint i to cluster k. A sharpened target distribution is then calculated as

$$p_{ik} = \frac{q_{ik}^2 / f_k}{\sum_{k'} q_{ik'}^2 / f_{k'}}, \quad f_k = \sum_i q_{ik}. \tag{3.29}$$

Squaring the assignment probabilities gives more weight to confident assignments, while the f_k term compensates for differences in cluster size. The DEC objective is the Kullback–Leibler divergence between the sharpened target and the current assignment,

$$\mathcal{L}_{\text{DEC}} = \sum_{i,k} p_{ik} \log \left(\frac{p_{ik}}{q_{ik}} \right), \tag{3.30}$$

and is added to the contrastive loss from Equation (3.27) - the target distribution $\mathbf{p}$ is calculated using `torch.no_grad` and is therefore treated as fixed during each optimisation step. Gradients flow through $\mathbf{q}$ and back into the GAT embeddings, encouraging the current soft assignments to move towards the sharper target.

An important difference between DEC and the other learned heads is that DEC does not use the ground-truth person labels directly in its own clustering loss - the GAT is still supervised through the contrastive loss, but the DEC component only has its current assignments from which to learn. As a result, it can strengthen an assignment that is already roughly correct, but it has no direct supervision telling it when an initial assignment is wrong - this limitation is corroborated by the experimental results reported in Section 4.2.

3.6.2 Slot attention on pose embeddings

The slot attention head [17] takes a different approach from DEC by using the ground-truth person labels during training. For each scene, K slot vectors $\mathbf{s}_1, \ldots, \mathbf{s}_K \in \mathbb{R}^d$ are initialised by sampling from a learned Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$ - the parameters of this distribution are learned jointly with the GAT embedding network.

The slots are refined over several iterations using competitive attention over the GAT embeddings - at each iteration t, the layer-normalised slots are used to form queries, while the layer-normalised keypoint embeddings provide the keys and values. Unlike standard attention, the softmax is applied *across the slots* - this means that the slots compete for each keypoint rather than independently attending to the same inputs. The attention matrix is then normalised over the inputs and used to form an update for each slot.

A GRU cell [79] combines each update with the slot state from the previous iteration, followed by a residual feed-forward block - after $T = 7$ iterations, the final slot representations are used to calculate a logit for every joint-slot pair,

$$\ell_{ik} = \mathbf{z}_i^\top \mathbf{W}_{\text{key}} \mathbf{s}_k. \tag{3.31}$$

As with k-means, the slot identifiers have no fixed relationship to the ground-truth person identifiers - the training loss therefore uses Hungarian matching before calculating cross entropy. The optimal slot-to-person mapping $\pi^\star$ is found from the mean prediction probability of each slot for each ground-truth person - cross entropy is then calculated after applying this mapping to align the slot labels with the person labels. This allows the network to learn person assignments without requiring, for example, slot 1 to always represent person 1.

At inference, the same iterative slot refinement is performed, but the ground-truth labels and Hungarian matching are no longer required for the prediction itself - each keypoint is assigned to the slot with the largest final logit, giving the predicted person partition.

The iteration count and learning-rate schedule were chosen from preliminary isolation tests carried out before the experiments reported in Chapter 4 - seven iterations were used instead of the standard three because the additional refinement gave more stable assignments in this setting. Training uses a cosine learning-rate schedule from 10^{-3} to 10^{-5}, which was also more stable than the constant learning rate tested during the preliminary experiments.

3.6.3 Graph partitioning on pose embeddings

The graph partitioning head approaches the grouping problem as binary pair classification rather than directly predicting one of K clusters. For each pair of keypoints (i, j), a pairwise MLP $\phi_{\text{pair}} : \mathbb{R}^{2d} \to \mathbb{R}$ predicts a single logit indicating whether the two joints belong to the same person - its input combines the difference between the two embeddings with their element-wise product:

$$\phi_{\text{pair}}\big([\mathbf{z}_i - \mathbf{z}_j \,\|\, \mathbf{z}_i \odot \mathbf{z}_j]\big) \in \mathbb{R}. \tag{3.32}$$

This pair representation follows the construction used in graph-based face clustering [18] - the difference term captures the relative position and direction of the two points in embedding space, while the element-wise product provides information about features that are active in both embeddings.

The head is trained using binary cross entropy over all $\binom{N}{2}$ possible keypoint pairs in a scene - this creates a class imbalance because different-person pairs become increasingly common as the number of people grows. For example, a synthetic scene with five complete 17-joint skeletons contains approximately 680 positive pairs and 2890 negative pairs - to stop the larger negative class from dominating the loss, positive examples are weighted using

$$\texttt{pos\_weight} = \min\left(10, \frac{n_{\text{neg}}}{n_{\text{pos}}}\right). \tag{3.33}$$

At inference, the predicted pairwise affinities are thresholded at 0.5 - pairs above the threshold form edges in a binary affinity graph, and the final person groups are obtained from its connected components using union-find. In this formulation, a sequence of positive connections can therefore place several joints into the same predicted person group.

One useful property of this approach is that the number of people K does not need to be supplied at inference time - instead, the predicted number of people is simply the number of connected components in the affinity graph. This makes graph partitioning attractive for the autonomous pipeline considered in Section 3.12, where K would otherwise need to be estimated separately - as shown by the results in Chapter 4, however, this practical advantage does not lead to better transfer performance on the real-data evaluation.

3.6.4 Joint training schedule

All three learned heads are trained jointly with the baseline GAT for 50 epochs on the synthetic training split - the optimiser, learning-rate schedule, batch size, and other training settings follow Section 3.5.3. This keeps the training setup consistent with the baseline and makes the effect of adding each head easier to compare.

For each method, the total training objective combines the contrastive loss from Equation (3.27) with the loss belonging to that particular grouping head:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contrastive}} + \lambda_{\text{head}}\mathcal{L}_{\text{head}}, \tag{3.34}$$

where $\lambda_{\text{head}} = 1.0$ for all three methods - this value was retained from the initial isolation experiments. Early tests did not show a useful improvement from varying the head weight, so a larger hyperparameter search was not carried out - the available experimental budget was instead focused on the later PG-GAT investigation, where the changes to the embedding network proved more promising.

Validation and checkpoint selection use the same procedure as the baseline in Section 3.5.3, with PGA used as the validation metric - at inference, each method uses its own grouping head rather than falling back to the baseline k-means read-out. DEC and slot attention obtain the final assignments from their respective forward passes, while the graph partitioning method uses the connected components of its thresholded affinity graph as the predicted person groups.

3.7 SA-DMON — SKELETON-AWARE DEEP MODULARITY NETWORKS

The three learned grouping heads from Section 3.6 all fail to improve on plain k-means when applied to embeddings from the same GAT backbone. This section considers a different approach. The GAT backbone is retained - but the grouping stage is replaced with a Deep Modularity Network (DMoN) [19], where the training objective is based on a differentiable relaxation of Newman modularity [59].

A vanilla DMoN configuration is tested first, followed by nine skeleton-aware variants that modify different parts of the modularity formulation. Despite these changes, all ten configurations plateau within a relatively narrow performance range and remain below the k-means baseline. The experiments point to a structural problem with the modularity objective in this setting - its null model is misaligned with a graph whose edges are constructed from spatial proximity between keypoints.

Together with the learned-head experiments of Section 3.6, these results provide further evidence that changing the grouping head is not the main source of improvement. Instead, the quality of the embedding produced by the network is the binding constraint.

3.7.1 Vanilla DMoN

Given the keypoint graph $\mathcal{G}$ from Section 3.3.2 - with adjacency matrix $A \in \{0,1\}^{N\times N}$ - DMoN predicts a soft cluster-assignment matrix $\mathbf{S} \in [0,1]^{N\times K}$. Each row of $\mathbf{S}$ sums to one, so that S_{ik} represents the soft assignment of node i to cluster k. The assignments are produced by applying a single linear projection to the GAT embeddings followed by a softmax over the K clusters.

The standard DMoN objective contains three terms [19]:

$$\mathcal{L}_{\text{DMoN}} = -\frac{1}{2m}\text{Tr}\big(\mathbf{S}^\top \mathbf{B}\mathbf{S}\big) + \lambda_{\text{ortho}} \left\| \mathbf{S}^\top\mathbf{S}/\|\mathbf{S}^\top\mathbf{S}\|_F - \mathbf{I}/\sqrt{K} \right\|_F + \lambda_{\text{cluster}} \left(\tfrac{\sqrt{K}}{N} \|\mathbf{s}_{\text{vol}}\|_2 - 1 \right), \quad (3.35)$$

where

$$\mathbf{B} = \mathbf{A} - \frac{\mathbf{d}\mathbf{d}^\top}{2m} \quad (3.36)$$

is the modularity matrix, $\mathbf{d}$ contains the node degrees, and $m = \frac{1}{2}\sum_{ij} A_{ij}$ is the total number of edges. The first term encourages partitions with high modularity. The orthogonality term discourages highly unbalanced assignments, while the term weighted by λ_{cluster} helps prevent solutions in which one or more clusters collapse and receive no nodes.

Two further terms are added to the DMoN objective. The first is a supervised cross-entropy term, computed against the ground-truth person labels $\mathbf{y}^\star$ after Hungarian matching of clusters to people, with weight $\lambda_{\text{supervised}} = 1$. This keeps the predicted clusters aligned with person identity, rather than allowing the network to follow only the partition preferred by the modularity objective. The second is a type-exclusivity term with weight $\lambda_{\text{type}} = 1$, which penalises a cluster that accumulates more than one joint of the same type. In the implementation, the spectral term itself also carries a weight, with $\lambda_{\text{spectral}} = 0.1$ unless a configuration below changes it.

This vanilla configuration forms the starting point for the SA-DMoN experiments reported in Chapter 4. Its main limitation comes from the structure of the graph itself. Modularity rewards groups of nodes that are more densely connected than expected under its null model, while the keypoint graph is constructed from spatial proximity. As a result, nearby joints are encouraged to remain together even when they belong to different people. Pose grouping often requires exactly the opposite behaviour. This mismatch motivates the skeleton-aware modifications to the null model introduced in the next subsection.

3.7.2 SA-DMoN v1: a skeleton-aware null model

The v1 variant replaces the standard degree-based null model $\mathbf{N}_{\text{deg}} = \mathbf{d}\mathbf{d}^\top/(2m)$ with one that includes both spatial proximity and the known structure of the COCO skeleton. The spatial component is defined using a Gaussian kernel over the normalised keypoint positions

$$P_{ij} = \exp\Bigl(-\frac{\|\mathbf{p}_i - \mathbf{p}_j\|_2^2}{2\sigma^2}\Bigr), \tag{3.37}$$

where σ controls the spatial bandwidth and is treated as a hyperparameter below.

The skeleton component is represented by a fixed 17×17 joint-type affinity matrix T, constructed using breadth-first search over the COCO skeleton graph. Joint types that are directly connected in the skeleton are assigned an affinity of $T_{ab} = 1.0$. Pairs separated by two hops receive 0.5, and three-hop pairs receive 0.25. Joint types more than three hops apart - or otherwise topologically disconnected - receive an affinity of zero.

For nodes i and j with joint types a and b, respectively, the spatial and skeleton terms are combined as

$$R_{ij} = T_{ab} \cdot P_{ij}. \tag{3.38}$$

The resulting matrix is rescaled so that its total mass matches the edge mass of the original graph,

$$\tilde{R}_{ij} = R_{ij} \left(\frac{2m}{\sum_{ij} R_{ij}} \right). \tag{3.39}$$

The pose-specific modularity matrix is then

$$\mathbf{B}_{\text{pose}} = \mathbf{A} - \tilde{\mathbf{R}}, \tag{3.40}$$

which replaces **B** in the DMoN loss of Equation (3.35). The remaining parts of the model are left unchanged, including the assignment head, the orthogonality and collapse-prevention terms, and the supervised cross-entropy and type-exclusivity terms.

The bandwidth σ proved to be the main sensitive parameter in this formulation. Several versions were therefore tested to determine whether the behaviour of the null model was mainly caused by how this bandwidth was selected. Each configuration below corresponds to one of the SA-DMoN ablations reported in Chapter 4.

1. **Learnable, unconstrained.** The bandwidth is parameterised as $\sigma = \exp(\xi)$, with ξ initialised to $\log 0.2$, and optimised with the rest of the network. This allows the model to choose its own spatial scale.
2. **Learnable, clamped.** The same learnable parameterisation is used - but σ is clamped to the range $[0.05, 0.5]$ during the forward pass. This prevents the bandwidth from diverging, as observed in the unconstrained configuration.
3. **Learnable, clamped, $\lambda_{\text{spectral}} = 1$.** This uses the clamped bandwidth while increasing the weight of the spectral term so that it has the same weighting as the supervised loss. The purpose is to test whether the modularity signal becomes useful when it has a larger influence on training.
4. **Fixed at the per-graph median pairwise distance.** Here, σ is recalculated for each scene as the median pairwise keypoint distance using `torch.no_grad`. This gives a bandwidth that adapts to the spatial scale of the scene without allowing the optimiser to change it directly.
5. **Spectral term removed.** The spectral weight is set to $\lambda_{\text{spectral}} = 0$. The skeleton-aware null model is still calculated for diagnostic purposes - but it contributes no modularity gradient during training. This configuration separates the effect of the supervised loss from the modularity-based objective.

These five v1 configurations form the methodological basis of SA-DMoN Experiments 4–8 reported in Chapter 4.

3.7.3 SA-DMoN v2: decoupled adjacency and entropy type loss

The motivation for the v2 variants comes from a limitation observed in the v1 formulation. The spatial adjacency $\mathbf{A}$ and the spatial component of the null model $\mathbf{P}$ are both strongly determined by keypoint distance. As a result, the two terms in $\mathbf{B}_{\text{pose}} = \mathbf{A} - \tilde{\mathbf{R}}$ can partially cancel around the selected bandwidth, weakening the modularity signal available during training.

The v2 formulation tests two changes intended to address this problem. The first separates the adjacency used by the modularity loss from the geometric graph used for message passing. The second replaces the threshold-based type-exclusivity loss used in v1 with a smoother entropy-based objective. The two changes are independent and can also be used together, giving four configurations in total.

Fix 1: feature-based adjacency: Instead of using the geometric kNN adjacency from Section 3.3.2 inside the spectral loss, a new adjacency is constructed from the current GAT embeddings. This feature graph is formed using k-nearest neighbours in embedding space with $k = 8$. The original geometric graph is still used by the GAT for message passing - only the adjacency supplied to the modularity objective is changed. Since the embedding itself changes during training, the feature-based graph is reconstructed on each forward pass.

This separates the two roles played by the graph. Message passing still uses spatial proximity to define which keypoints exchange information, while the modularity objective operates on neighbourhoods learned in the embedding space, rather than another distance-based version of the same spatial graph.

Fix 2: entropy type loss: The second change replaces the v1 type-exclusivity term with a soft entropy-based objective

$$\mathcal{L}_{\text{type}}^{\text{v2}} = -\sum_{i,k} S_{ik} \log p_{ik}^{\text{type}}, \tag{3.41}$$

where p_{ik}^{type} is the estimated probability that cluster k contains the joint type t_i, calculated from the current soft assignment statistics. Compared with the threshold-based ReLU formulation used in v1, this gives a smoother gradient as the assignments change. The trade-off is that the sharper discrete hinge of the earlier loss is no longer present.

Configurations: Four v2 configurations are evaluated in Chapter 4. The first uses feature-based adjacency with $\lambda_{\text{spectral}} = 1.0$. The second uses the same feature adjacency but reduces the spectral weight to $\lambda_{\text{spectral}} = 0.1$ - matching the weighting used in v1. The third keeps the original spatial adjacency and changes only the supervised term to the entropy type loss. The final configuration combines both changes - using feature-based adjacency together with the entropy-based type objective.

3.7.4 Training schedule

All ten SA-DMoN variants are trained jointly with the GAT on the synthetic training split. These include the vanilla DMoN baseline, the five v1 configurations described above, including the no-spectral diagnostic, and the four v2 configurations. Training otherwise follows the optimiser settings given in Section 3.5.3.

Depending on the behaviour of the head loss, each configuration is trained for either 100 or 150 epochs. A 100-epoch schedule is used when the loss has clearly plateaued by that point. For configurations where the head loss is still changing at 100 epochs, training is extended to 150 epochs to allow additional time for convergence. This avoids discarding a variant simply because its grouping objective converges more slowly than the baseline contrastive model.

Despite the longer training schedules, none of the SA-DMoN configurations are able to improve improve on *k*-means applied to the baseline GAT embeddings. The experimental results are reported in Section 4.3. The broader reason for this behaviour - particularly the mismatch between the modularity objective and the structure of the pose-grouping graph - is discussed in Section 5.3.1.

3.8 PG-GAT — POSE GROUPING GRAPH ATTENTION NETWORK

This section introduces PG-GAT, the main method developed in this work. PG-GAT builds on the baseline GATv2 embedding network from Section 3.5, but modifies the attention mechanism to make better use of the known structure of the human skeleton.

Three changes are introduced to the baseline attention mechanism. Each one targets a particular limitation observed in the earlier experiments, and each can be enabled independently, so that its effect can be tested through ablation. The aim is to improve the embedding itself, rather than add further complexity to the grouping head, which follows directly from the learned-head and SA-DMoN results described in the previous sections.

PG-GAT is trained in two stages. The network is first pre-trained on the synthetic dataset and is then fine-tuned on COCO to reduce the synthetic-to-real domain gap. The architecture and training choices are not treated as fixed assumptions - the main architectural settings are tested through a Bayesian sweep, the three skeleton-aware changes are evaluated through isolation ablations, and the COCO fine-tuning configuration is selected through a separate hyperparameter sweep. Together, these experiments provide the basis for the final PG-GAT configuration used in the headline evaluations.

3.8.1 Design intuition

The motivation for PG-GAT follows from the results of Sections 3.6 and 3.7. Three different learned grouping heads (DEC, slot attention, and graph partitioning), together with ten SA-DMoN variants, were unable to improve on plain k-means applied to the standard GAT embeddings. Since these methods change the grouping stage while keeping the underlying embedding network largely fixed, their repeated failure rules out the grouping head as the binding constraint and identifies the embedding network as the dominant axis of variation.

PG-GAT therefore changes the embedding stage, rather than introducing another grouping head. The final read-out remains the same k-means procedure used by the baseline. Instead, the modifications described below change how the attention mechanism processes relationships between keypoints in the graph. This keeps the comparison with the baseline relatively direct - the main difference is how the embedding is produced, rather than how it is clustered afterwards.

Standard GATv2 attention [15] does not explicitly distinguish between different categories of edges. The same attention mechanism is used for a left-shoulder-left-elbow edge, a left-shoulder-right-shoulder edge, or a left-shoulder-right-ankle edge. For pose grouping, however, these relationships have different meanings. Two detections of the same joint type cannot belong to the same valid person skeleton, while skeletally adjacent joint types provide useful evidence that two detections may belong together. Other pairs, particularly those that are distant in both the image and the skeleton topology, generally provide less direct grouping information.

PG-GAT introduces three modifications that make these differences available to the attention mechanism, while retaining the GATv2Conv primitive used by the baseline. Each modification can be enabled or disabled independently, allowing its contribution to be measured in the ablation experiments of Chapter 4. The complete architecture is illustrated in Figure 3.3.

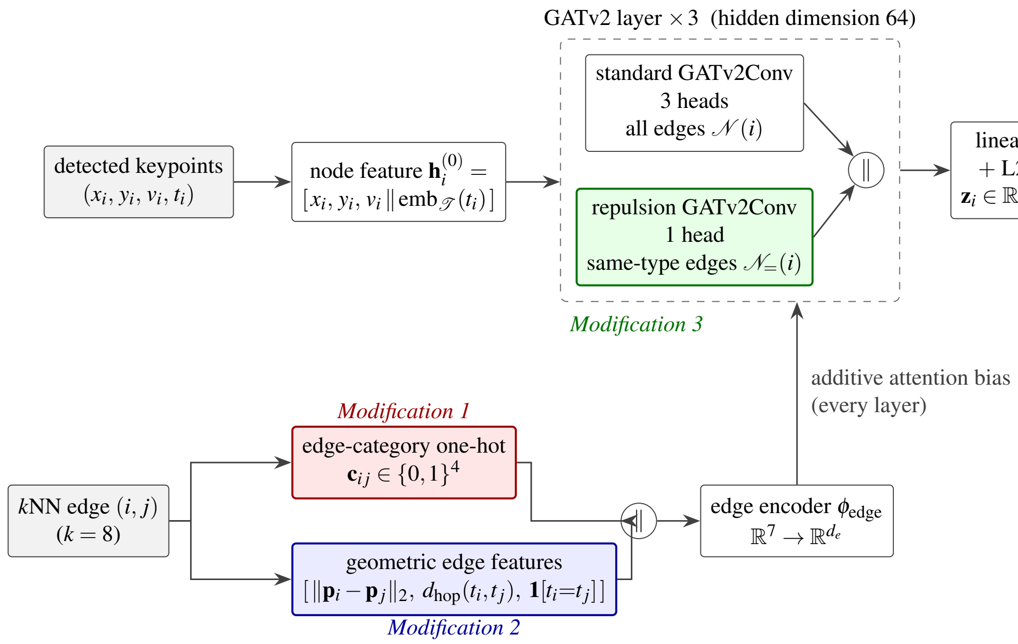

Figure 3.3. The PG-GAT architecture. Input node features contain the keypoint position, visibility flag, and a learned joint-type embedding. Modification 1 assigns each edge to one of four categories and supplies the resulting one-hot representation to the GATv2 attention through a learned additive bias (red). Modification 2 adds three geometric features to this edge representation: L2 distance, skeleton hop distance, and a same-type indicator (blue). Modification 3 separates the multi-head attention into a standard sub-layer operating over the full neighbourhood and a repulsion sub-layer restricted to same-type edges (green).

3.8.2 Modification 1: type-pair attention bias

Every edge $(i, j) \in \mathcal{E}$ is assigned to one of four mutually exclusive categories according to the joint types (t_i, t_j):

- SAME-TYPE - $t_i = t_j$. Two detections of the same joint type cannot belong to the same valid person skeleton, so this category provides direct evidence that the nodes should be separated.
- SKELETAL-NEIGHBOUR - (t_i, t_j) forms a directly connected pair in the COCO skeleton graph, such as a shoulder and elbow. These edges provide a strong structural cue because anatomically adjacent joints from the same person tend to have consistent spatial relationships.
- SAME-LIMB - (t_i, t_j) belongs to the same anatomical limb chain but the two joint types are not directly adjacent, for example a shoulder and wrist. This category provides a weaker,

longer-range structural relationship.

- CROSS-BODY - all remaining joint-type pairs. These edges have no direct local relationship in the skeleton topology, and therefore carry a weak or no structural prior.

The category is encoded as a one-hot vector $\mathbf{c}_{ij} \in \{0,1\}^4$ and passed through a small edge encoder ϕ_{edge},

$$\mathbf{e}_{ij}^{\text{cat}} = \phi_{\text{edge}}(\mathbf{c}_{ij}) \in \mathbb{R}^{d_e}. \tag{3.42}$$

The resulting feature is supplied to the GATv2 layer through the `edge_dim` mechanism of the PyTorch Geometric `GATv2Conv` implementation [83]. This introduces a learned edge-dependent contribution to the attention pre-activation, which can be written as

$$\alpha_{ij}^{(l)} \propto \exp\Big(\mathbf{a}^{(l)\top} \text{LeakyReLU}\big(\mathbf{W}^{(l)}[\mathbf{h}_i^{(l)} \,\|\, \mathbf{h}_j^{(l)}] + \mathbf{W}_e^{(l)} \mathbf{e}_{ij}^{\text{cat}}\big)\Big), \tag{3.43}$$

where $\mathbf{a}^{(l)}$, $\mathbf{W}^{(l)}$, and $\mathbf{W}_e^{(l)}$ are learned parameters of layer l. The edge category can therefore influence the attention assigned to a neighbour, while retaining the standard GATv2 attention mechanism. In particular, the network can learn different responses to same-type, skeletal-neighbour, same-limb, and cross-body edges, rather than requiring their significance to be inferred from the node features alone.

An earlier implementation instead used custom scatter-based attention with separate projection parameters for each edge category. This version was numerically unstable during training, and reached only 0.60 synthetic PGA. The custom attention implementation was therefore replaced by the standard `GATv2Conv` operator with explicit edge features. This restored stable training, while retaining the intended type-dependent attention mechanism.

3.8.3 Modification 2: skeleton-relative position encoding

The baseline GAT receives position only through the node features. Consequently, relative spatial relationships between two connected keypoints are not represented explicitly in the attention calculation - they must instead be inferred from the endpoint node representations. The second PG-GAT modification makes this information directly available to the attention mechanism by adding three geometric features to each edge:

- the Euclidean distance between the endpoint positions, $\|\mathbf{p}_i - \mathbf{p}_j\|_2$;
- the skeleton hop distance $d_{\text{hop}}(t_i, t_j)$, defined as the shortest-path distance between joint types t_i and t_j on the COCO skeleton graph and normalised to $[0,1]$;

- a same-type indicator $\mathbb{1}[t_i = t_j] \in \{0,1\}$.

These three values are concatenated with the four-dimensional category encoding introduced in Modification 1, giving the edge descriptor

$$\mathbf{g}_{ij} = \left[\mathbf{c}_{ij} \,\|\, \|\mathbf{p}_i - \mathbf{p}_j\|_2 \,\|\, d_{\text{hop}}(t_i, t_j) \,\|\, \mathbb{1}[t_i = t_j]\right] \in \mathbb{R}^7. \tag{3.44}$$

The descriptor is passed through the same edge encoder ϕ_{edge} used for the categorical representation,

$$\mathbf{e}_{ij}^{\text{geom}} = \phi_{\text{edge}}(\mathbf{g}_{ij}), \tag{3.45}$$

and the resulting feature enters the GATv2 attention calculation through the same `edge_dim` mechanism described above. Modification 2 therefore changes the information supplied to the attention layer, without requiring a different message-passing operator.

Of the three added features, the skeleton hop distance provides the most direct description of anatomical structure. It tells the network how closely related two joint types are in the skeleton topology, independently of their proximity in the image. A shoulder–elbow pair, for example, has a hop distance of one, and represents a plausible local skeletal relationship. A shoulder–ankle pair has a much larger topological distance, so similar image-plane proximity does not carry the same grouping evidence. The attention mechanism can use this distinction when deciding how strongly information should pass between the two nodes.

3.8.4 Modification 3: same-type repulsion heads

The third modification reserves part of the multi-head attention capacity for distinguishing same-type keypoints that belong to different people. In a standard GAT layer with H heads, every head attends over the same neighbourhood. The signal associated with separating, for example, two left-elbow detections is therefore learned by heads that must also model the relationships between anatomically different joints. PG-GAT separates these two roles by introducing a dedicated repulsion sub-layer.

The layer is divided into a standard `GATv2Conv` operating over the full neighbourhood and a smaller repulsion `GATv2Conv` restricted to same-type edges. Let the standard sub-layer contain H_{std} heads and attend to $\mathcal{N}(i)$, while the repulsion sub-layer contains H_{rep} heads and attends only to

$$\mathcal{N}_{=}(i) = \{j \in \mathcal{N}(i) : t_j = t_i\}, \tag{3.46}$$

with $H_{\text{std}} + H_{\text{rep}} = H$. The outputs of the two sub-layers are concatenated along the head dimension,

$$\mathbf{h}_i^{(l+1)} = \left[\mathrm{GATv2Conv}_{\text{std}}(\mathbf{h}^{(l)}, \mathcal{N}(i)) \,\big\|\, \mathrm{GATv2Conv}_{\text{rep}}(\mathbf{h}^{(l)}, \mathcal{N}_{=}(i))\right], \tag{3.47}$$

so that the total number of attention heads remains equal to that of the corresponding standard layer.

The repulsion sub-layer receives only edges connecting detections of the same joint type. It therefore does not need to divide its attention capacity between anatomical grouping and same-type discrimination. Under the contrastive objective of Equation (3.27), these heads receive a focused training signal for separating same-type detections belonging to different people. In practical terms, the sub-layer is specialised for questions such as which person each of several left-elbow detections belongs to, while the standard heads continue to model the wider skeletal neighbourhood.

The default PG-GAT configuration assigns one of the four attention heads to this repulsion pathway, giving $H_{\text{rep}} = 1$ and $H_{\text{std}} = 3$. The head allocation is evaluated as an axis of the loss sweep described in Section 3.8.5, which varies the number of repulsion heads over $\{0, 1, 2\}$.

3.8.5 Composing the modifications and training

The three PG-GAT modifications can be enabled independently through boolean flags in `SAGATConfig`[1]: `use_type_pair_attention`, `use_position_encoding`, and `use_repulsion_heads`. This makes it possible to disable each mechanism independently and measure its contribution without changing the rest of the network. The PG-GAT configuration used for the headline evaluations enables all three modifications. Their individual and combined effects are reported in Table 4.4 of Chapter 4.

Architecture selection: The PG-GAT architecture is not inherited directly from the baseline in Section 3.5. Instead, six architectural hyperparameters (number of layers, number of attention heads, hidden dimension, output dimension, joint-type embedding dimension, and dropout) are selected through a 32-run Bayesian sweep. The search space spans relatively small configurations through to the practical memory limit at batch size 4 on the experimental hardware. Synthetic validation PGA from Equation (3.21) is used as the selection metric.

[1]The implementation retains the legacy class name `SAGATConfig`; the method is referred to as PG-GAT throughout this document.

Hyperband early termination [87] is used to remove weak configurations from the search. The first pruning decision is made after the fifth validation point, corresponding to epoch 25. This reduces the cost of clearly underperforming runs, while still allowing configurations with slower initial convergence to develop. The sweep selects a three-layer network with four attention heads, hidden dimension 64, output dimension 256, joint-type embedding dimension 64, and dropout 0.1. This configuration is used for the PG-GAT results reported in Chapters 4 and 5.

A separate loss and optimisation sweep varies the contrastive margin, number of repulsion heads, learning rate, and weight decay. The search returns settings close to those already used by the baseline, including the contrastive margin of $\gamma = 0.5$. This provides an additional check that the improvement obtained with PG-GAT is not primarily the result of retuning the contrastive objective or optimiser.

Two-stage training: PG-GAT is trained in two stages. The first stage uses the 400-scene synthetic training split from Section 3.4.1 and follows the optimiser settings of Section 3.5.3. The resulting model is then fine-tuned on the COCO [1] `train2017` split for a smaller number of epochs and at a reduced learning rate. The purpose of this second stage is to adapt the embedding to real COCO detections, while retaining the person-separation structure learned during synthetic pre-training.

The fine-tuning schedule is selected through a separate 12-run Bayesian sweep over the number of epochs, learning rate, and weight decay. The search is centred around the previously used fine-tuning recipe of 20 epochs at a learning rate of 10^{-4}. The selected configuration uses 15 epochs, together with the learning rate and weight decay chosen by the sweep. This checkpoint produces the headline PG-GAT results in Chapter 4 and is recorded under the W&B alias `meng-headline`.

Read-out: The grouping stage is unchanged from the baseline. For each image, PG-GAT produces L2-normalised per-keypoint embeddings, and k-means is run with $k = K$ to recover the person assignments, following Section 3.5.3. PG-GAT therefore changes the representation on which grouping is performed, while deliberately keeping the final read-out simple.

3.8.6 Variants explored after PG-GAT

PG-GAT as described in this section is the final embedding network used for the headline result in Chapter 4. Three additional variants are evaluated to test whether particular aspects of the design can be improved. These variants are treated as controlled extensions of PG-GAT, rather than as separate solutions to the grouping problem.

PG-GAT v2, described in Section 3.9, tests whether the coordinate-based node representation is missing useful appearance information. Pre-trained visual features extracted with a MobileNetV2 [22] backbone are added to the positional input to determine whether local image appearance provides identity information that cannot be recovered from keypoint position and type alone.

The hyperbolic PG-GAT variant of Section 3.10 instead changes the geometry of the output space. The Euclidean embedding is replaced with an embedding on a Lorentz-model hyperbolic manifold, motivated by the lower-distortion representations that hyperbolic spaces can provide for tree-structured data. Since the human skeleton has a tree-like topology, this variant tests whether that geometric property translates into better or more compact pose-grouping embeddings.

Finally, TriGAT in Section 3.11 changes the basic unit represented by the graph. Rather than embedding individual joints, it constructs representations from triplets of connected joints. The angle at the central joint provides a rotation-invariant description of local articulation that is not explicitly available to the standard single-joint representation.

The experiments in Chapter 4 show that none of these three extensions provides a clear improvement over the defended PG-GAT configuration. Their methodologies are nevertheless included here because each tests a distinct assumption of the main design, and the results provide evidence for retaining the simpler PG-GAT formulation used in the final pipeline.

3.9 PG-GAT V2 — VISUAL FEATURE AUGMENTATION

This section describes the visual-feature-augmented PG-GAT variant, referred to as PG-GAT v2. The purpose of the variant is to test whether local appearance information provides useful identity cues beyond the position, visibility, and joint-type information used by the standard PG-GAT model. Pre-trained image features are extracted from the input image and sampled at each detected keypoint, then incorporated into the node representation before graph message passing.

The variant leaves the remaining PG-GAT pipeline unchanged, so that the effect of the added visual information can be isolated. Its methodology is included here to define the model evaluated in Chapter 4.

The experiments show that the additional visual features do not improve on the standard PG-GAT configuration - the reasons for this result are considered in Section 5.2.5.

3.9.1 Motivation and design constraints

The standard PG-GAT model of Section 3.8 performs grouping without direct access to the source image. Its input consists of the preprocessed keypoint features and joint-type information, while the RGB image is used only by the upstream detector. As a result, local appearance cues around each detected joint are discarded before the grouping stage.

Before constructing a learned visual-feature variant, a lightweight feasibility study was performed. Features from a pre-trained MobileNetV2 [22] were sampled at each detected keypoint and added to the representation used for clustering. In the simplest configuration, these features improved COCO grouping accuracy by approximately six percentage points over the positions-only input. However, the gain decreased as the surrounding grouping configuration became stronger, falling below one percentage point in the best configurations tested.

This diminishing improvement suggested that the visual features contain some useful identity information, but that much of it becomes redundant once the grouping representation is already strong. The PG-GAT v2 methodology below therefore gives the visual pathway substantially more capacity than the initial feasibility study. This provides a stronger test of whether appearance information can contribute meaningfully when it is learned jointly with the PG-GAT embedding, rather than simply being appended to the input of a clustering procedure. The resulting Chapter 4 experiment can therefore distinguish a genuinely limited visual contribution from a failure caused by an underpowered visual-feature integration.

3.9.2 Visual feature extraction and caching

The visual backbone is MobileNetV2 [22], pre-trained on ImageNet [88]. Features are taken from layer 17, which produces a 320-channel feature map at $1/32$ of the input image resolution. This layer was selected based on the preceding feasibility study. Features from shallower layers, which primarily capture lower-level texture and edge information, reduced grouping performance across the configurations tested, whereas the deeper semantic features were the only ones to provide a measurable improvement.

The MobileNetV2 backbone is kept frozen throughout the PG-GAT v2 experiments. Its parameters are therefore not updated by the grouping loss. This keeps the experiment focused on whether a fixed, pre-trained visual representation contains useful information for pose grouping, rather than introducing the additional effect of jointly fine-tuning a large image backbone.

For each COCO image, MobileNetV2 is evaluated once to produce the 320-channel feature map. The feature map is then bilinearly sampled at each ground-truth keypoint location, producing a visual descriptor

$$\mathbf{u}_i \in \mathbb{R}^{320} \tag{3.48}$$

for keypoint i. Bilinear sampling is used because the feature map has a spatial resolution of only $1/32$ of the input image, so a keypoint location will not generally coincide exactly with a feature-map location.

Feature extraction is performed as a preprocessing step over the COCO `train2017` and `val2017` splits, rather than during training. The resulting per-keypoint descriptors are stored on disk in PyTorch Geometric `Data` objects, with the visual descriptor attached as a node-level feature. The resulting cache contains 54,564 training samples and 2,258 validation samples.

Caching removes the MobileNetV2 forward pass from the PG-GAT v2 training loop. The frozen visual features therefore need to be computed only once, reducing the cost of subsequent experiments, and allowing different visual-fusion configurations to be evaluated without repeating feature extraction.

3.9.3 Architecture and input fusion

The PG-GAT v2 network extends the PG-GAT architecture of Section 3.8 by adding a visual-feature projection at the input stage. Each cached MobileNetV2 descriptor $\mathbf{u}_i \in \mathbb{R}^{320}$ is first projected to 32 dimensions using a learned linear layer, followed by layer normalisation [89] and a GELU activation [90]:

$$\mathbf{u}_i^{\text{proj}} = \text{GELU}(\text{LayerNorm}(\mathbf{W}_v\mathbf{u}_i + \mathbf{b}_v)), \qquad \mathbf{W}_v \in \mathbb{R}^{32\times 320}. \tag{3.49}$$

The resulting vector $\mathbf{u}_i^{\text{proj}} \in \mathbb{R}^{32}$ is concatenated with the positional and joint-type representation used by PG-GAT, before the first graph-attention layer. The visual pathway therefore adds local appearance information to the existing node representation, without changing the graph construction or the later message-passing stages. All three skeleton-aware modifications introduced in Section 3.8 are retained. PG-GAT v2 is consequently an additive extension of PG-GAT, rather than a replacement for any of its structural components.

The trainable PG-GAT v2 network contains 197,280 parameters. Approximately 27,000 additional parameters arise from the visual projection block of Equation (3.49) and the corresponding increase in the input dimension of the first GAT layer. The remaining 170,560 parameters are inherited from the standard PG-GAT model.

The frozen MobileNetV2 feature extractor contributes a further approximately 2.2×10^6 parameters. Although these parameters are not updated during PG-GAT v2 training, they remain part of the inference pipeline, and are therefore included when comparing the total model size against alternatives such as the 28.6×10^6-parameter HigherHRNet model. The complete parameter-count comparison is reported in Section 4.10.

3.9.4 Training schedule and the extended run

PG-GAT v2 is trained from scratch using the cached COCO training features and the contrastive objective of Equation (3.27). No learned grouping head is introduced - at evaluation time the resulting embeddings are clustered using the same k-means read-out as the standard PG-GAT model. The experiment therefore isolates the effect of adding visual information to the embedding network.

Two training schedules are evaluated in Chapter 4. The first trains PG-GAT v2 for 20 epochs, with an initial learning rate of 10^{-3} and cosine annealing to 10^{-5}. The second extends this training to 50 epochs by resuming from the 20-epoch checkpoint and training for a further 30 epochs, with the initial learning rate reduced to 5×10^{-4}. The extended schedule is included to test whether the visual pathway simply requires more optimisation time before its additional capacity becomes useful. The results in Chapter 4 show that the additional 30 epochs improve PGA by only a few thousandths, indicating that the comparison with standard PG-GAT is not explained by premature termination of training.

The optimiser, weight-decay, and gradient-clipping settings otherwise follow Section 3.5.3. Validation is performed every two epochs on the cached COCO validation set, and the checkpoint with the highest COCO validation PGA is retained for final evaluation.

COCO validation is used rather than synthetic validation because the synthetic pipeline does not contain cached MobileNetV2 descriptors. Generating equivalent features would require applying the ImageNet-pre-trained backbone to the rendered synthetic images, introducing an additional synthetic-to-real appearance-domain effect into an experiment intended to isolate the contribution of visual features.

Restricting model selection to COCO therefore keeps the experiment focused on whether those features improve grouping on the real-image distribution for which they were introduced. Synthetic transfer is reported separately in Section 4.11 for completeness.

3.10 HYPERBOLIC PG-GAT

This section describes a hyperbolic variant of PG-GAT in which the Euclidean output space $\mathbb{R}^d$ is replaced by a Lorentz-model hyperbolic manifold $\mathbb{H}^d$ with curvature -1. The remainder of the PG-GAT architecture is retained, allowing the experiment to isolate the effect of changing the geometry of the learned representation.

The motivation follows from the structure of the data. A human skeleton forms a shallow kinematic tree, and a multi-person scene can be viewed as a forest of such trees. Hyperbolic spaces are well suited to representing hierarchical and tree-structured data because their volume grows exponentially with radius, matching the exponential growth of a tree more closely than the polynomial volume growth of Euclidean space. Consequently, tree structures can be represented with lower distortion in low-dimensional hyperbolic spaces than in comparable Euclidean spaces [68, 69].

This motivates a direct experimental question for pose grouping. If part of the representational capacity of Euclidean PG-GAT is used to encode the hierarchical relationships between joints in the skeleton, then placing the final embeddings on a hyperbolic manifold may provide a more natural geometry for those relationships. The hyperbolic variant therefore changes the embedding space while leaving the skeleton-aware message-passing machinery unchanged, providing a controlled test of whether the geometric prior itself improves grouping quality or embedding efficiency.

3.10.1 Architecture

The internal message-passing layers of the hyperbolic variant remain Euclidean and retain all three PG-GAT modifications introduced in Section 3.8. Only the final embedding geometry is changed. The Euclidean output of the PG-GAT stack is mapped onto the Lorentz hyperboloid

$$\mathbb{H}^d = \left\{ \mathbf{x} \in \mathbb{R}^{d+1} : \langle \mathbf{x}, \mathbf{x} \rangle_{\mathcal{L}} = -1,\, x_0 > 0 \right\}, \tag{3.50}$$

where the Minkowski inner product is

$$\langle \mathbf{x}, \mathbf{y} \rangle_{\mathcal{L}} = -x_0 y_0 + \sum_{k=1}^{d} x_k y_k. \tag{3.51}$$

This hybrid Euclidean-internal, hyperbolic-output design is deliberate. It isolates the effect of the embedding geometry while leaving the message-passing architecture unchanged. A fully hyperbolic PG-GAT would instead require replacing the linear transformations and aggregation operations throughout the network with manifold-aware counterparts, such as Möbius operations and Fréchet-mean aggregation. Such a model would test several architectural changes at once, and would therefore make it difficult to attribute any difference in grouping performance specifically to the output geometry. The Lorentz model is used instead of the Poincaré ball because of its greater numerical stability for learned hyperbolic representations [70]. The implementation uses the `geoopt` library [91].

Before projection onto the manifold, the Euclidean output $\mathbf{v}_i \in \mathbb{R}^d$ is multiplied by a learnable positive tangent-scale parameter τ. The scaled vector is then mapped from the tangent space at the origin onto the Lorentz hyperboloid using the exponential map,

$$\mathbf{x}_i^{\mathbb{H}} = \mathrm{expmap}_0(\tau \mathbf{v}_i)\,. \tag{3.52}$$

The scale is initialised as

$$\tau_0 = \frac{1}{\sqrt{d}}. \tag{3.53}$$

This initialisation controls the magnitude of the tangent vectors before the exponential map. Preliminary measurements showed that it places initial pairwise geodesic distances approximately in the range $[0.6, 1.0]$ for the embedding dimensions considered here. Without this scaling, the Euclidean output norm is approximately proportional to $\sqrt{d}$, and exponential mapping produces initial pairwise hyperbolic distances on the order of 20. At that scale, the contrastive margin $\gamma = 0.5$ provides little useful separation pressure because most negative pairs already lie far beyond the margin. The tangent scaling therefore keeps the initial hyperbolic geometry in a range where the contrastive objective supplies a meaningful gradient.

3.10.2 Hyperbolic contrastive loss

The Euclidean contrastive objective of Equation (3.27) is defined in terms of cosine similarity between L2-normalised embeddings. Once the output is moved onto the Lorentz manifold, cosine similarity is no longer the natural metric. The hyperbolic variant therefore replaces it with the Lorentz geodesic distance

$$d_{\mathbb{H}}(\mathbf{x}, \mathbf{y}) = \operatorname{arccosh}\left(-\langle \mathbf{x}, \mathbf{y} \rangle_{\mathcal{L}}\right), \tag{3.54}$$

which is non-negative and equals zero when the two points coincide. Using the same positive and negative pair sets $\mathcal{P}^+$ and $\mathcal{P}^-$ defined in Section 3.5.2, the hyperbolic contrastive objective is

$$\mathcal{L}_{\mathbb{H}} = \frac{1}{|\mathcal{P}^+|} \sum_{(i,j) \in \mathcal{P}^+} d_{\mathbb{H}}\left(\mathbf{x}_i^{\mathbb{H}}, \mathbf{x}_j^{\mathbb{H}}\right) + \frac{1}{|\mathcal{P}^-|} \sum_{(i,j) \in \mathcal{P}^-} \max\left(0, \gamma_{\mathbb{H}} - d_{\mathbb{H}}\left(\mathbf{x}_i^{\mathbb{H}}, \mathbf{x}_j^{\mathbb{H}}\right)\right). \tag{3.55}$$

The first term pulls embeddings belonging to the same person toward zero geodesic distance. The second pushes embeddings belonging to different people apart until their distance exceeds the hyperbolic margin $\gamma_{\mathbb{H}}$. The margin is set to $\gamma_{\mathbb{H}} = 2.0$, approximately twice the initial mean pairwise distance produced by the tangent-scale initialisation of Equation (3.52). A separate margin is required because the Euclidean value $\gamma = 0.5$ is defined on cosine similarity, and therefore has no equivalent interpretation on the Lorentz distance scale.

Computing the full pairwise distance matrix requires an additional numerical safeguard. Floating-point error in the batched Minkowski inner products can move the argument of arccosh slightly outside its valid domain, most notably for self-distances whose theoretical argument is exactly one. The implementation ensures that tensors produced through `.expand()` are made contiguous before the pairwise operations, and applies `torch.nan_to_num` to the resulting distance matrix as a final safeguard against non-finite values. These operations do not change the intended loss - they prevent numerical artefacts in the Lorentz-distance calculation from propagating through training.

3.10.3 Grouping read-out

The downstream k-means read-out of Section 3.5.3 operates in Euclidean space, and therefore cannot be applied directly to points on the Lorentz manifold. Before clustering, each hyperbolic embedding is mapped back to the tangent space at the origin using the logarithmic map,

$$\mathbf{v}_i^{\tan} = \operatorname{logmap}_0\left(\mathbf{x}_i^{\mathbb{H}}\right), \qquad \operatorname{logmap}_0 : \mathbb{H}^d \to T_{\mathbf{0}}\mathbb{H}^d. \tag{3.56}$$

The tangent representation contains the additional Lorentz time coordinate introduced by the hyperboloid model. This coordinate is discarded, leaving a d-dimensional spatial vector, which is then L2-normalised before being supplied to the same k-means procedure used by Euclidean PG-GAT.

The tangent space provides a locally Euclidean approximation to the curved manifold. Geometrically, it can be understood as the flat space that touches the manifold at the chosen reference point and locally approximates its geometry. Mapping the embeddings to the tangent space therefore permits standard Euclidean clustering while retaining, during training, the geodesic structure imposed by the hyperbolic contrastive loss.

This approximation is most faithful near the origin. The tangent-scale parameter τ introduced in Equation (3.52) keeps the learned representations within a comparatively small region of the manifold, where distortion introduced by the tangent-space projection is correspondingly limited. The procedure nevertheless constitutes a deliberate compromise - training and pairwise supervision use the Lorentz geometry, whereas the final partition is recovered from its Euclidean tangent-space approximation. This makes the hyperbolic variant directly compatible with the standard PG-GAT k-means read-out, while avoiding the additional methodological change of introducing a manifold-specific clustering algorithm.

3.10.4 Two pre-registered hypotheses and the dimension axis

The hyperbolic variant is evaluated against two hypotheses specified before the corresponding experiments were run. The first tests for a quality advantage at matched representational capacity:

H1: A hyperbolic PG-GAT embedding matches or exceeds the grouping performance of its Euclidean counterpart at the same output dimension.

The second tests whether the geometric prior permits a more compact representation:

H2: A hyperbolic PG-GAT embedding matches the grouping performance of the Euclidean model at a substantially smaller output dimension.

The distinction is important because hyperbolic geometry need not improve the maximum achievable PGA to be useful. A model that preserves Euclidean grouping quality with a substantially lower-dimensional embedding would constitute an efficiency gain even if H1 is not supported.

The initial evaluation uses two controlled single-run ablations. H1 is tested at output dimension $d = 128$, comparing Euclidean and hyperbolic embeddings at matched dimensionality. H2 is tested at $d = 32$, a four-fold reduction relative to the 128-dimensional comparison configuration and an eight-fold reduction relative to the $d = 256$ sweep-selected PG-GAT architecture used for the headline results. These experiments provide the direct tests of the two pre-specified hypotheses.

A subsequent eight-run Bayesian sweep over output dimension, number of message-passing layers, and hidden dimension tests whether the single-run conclusions persist across a broader region of the hyperbolic architecture space. The sweep is therefore used as defensibility evidence rather than as the source of the hypotheses - the hypotheses and their primary comparisons are fixed independently of the sweep outcome.

Finally, a separate Option B experiment fine-tunes the $d = 32$ hyperbolic model on COCO. This experiment tests whether any low-dimensional efficiency advantage observed after synthetic pre-training survives adaptation to the real-image distribution, rather than being specific to the synthetic evaluation regime.

Unless explicitly stated otherwise, the hyperbolic experiments inherit the PG-GAT architecture components, training procedure, optimiser, and evaluation protocol defined in Sections 3.5.3 and 3.8.5. The embedding geometry, its associated contrastive distance, and the output dimension under test are therefore the principal experimental variables.

3.11 TRIGAT — TRIPLET-BASED PRIMITIVE

This section describes TriGAT, a variant of PG-GAT in which the fundamental graph primitive is changed from an individual keypoint to a triplet of anatomically connected joints. The motivation is articulation. When two people occupy similar regions of the image, absolute keypoint position provides a weak identity cue, whereas the relative configuration of connected joints may remain discriminative. Triplets such as shoulder–elbow–wrist and hip–knee–ankle encode a local articulation pattern through the angle at the central joint, providing information that is not represented explicitly by a single-joint node.

In particular, the pivot angle of a three-joint chain is invariant to global in-plane translation and rotation. It therefore provides a local descriptor of pose configuration that is complementary to the absolute positional features consumed by PG-GAT. This motivates the TriGAT hypothesis - representing connected joint triplets as graph nodes should expose articulation as a first-class feature to the attention mechanism, and thereby improve identity separation in difficult scenes where people overlap spatially.

TriGAT tests this hypothesis by changing the unit on which message passing operates, while retaining the broader PG-GAT embedding-and-clustering framework. The resulting triplet representations must ultimately be converted back to per-joint embeddings, so that grouping can be evaluated with the same k-means read-out and PGA metric used throughout the chapter. This conversion is an important part of the variant, because a joint may participate in multiple triplets and therefore receive multiple candidate identity signals.

The experiments reported in Chapter 4 do not support the triplet hypothesis. Rather than treating that outcome as evidence against articulation information in general, the analysis distinguishes the information available to the triplet representation from the information lost when triplet-level predictions are converted back to joint-level assignments. The following subsections therefore specify the triplet construction, its geometric features, the TriGAT message-passing architecture, and the joint-level read-out required to make the comparison with PG-GAT controlled and interpretable.

3.11.1 Triplet enumeration and features

The COCO skeleton topology defines 19 canonical triplet types, indexed by ordered joint-type triples (t_A, t_B, t_C) for which (t_A, t_B) and (t_B, t_C) are both edges of the COCO skeleton graph. The middle joint B acts as the pivot of the triplet. For each person in a scene, at most one instance of each canonical triplet is constructed. A triplet is omitted whenever any of its three constituent joints is missing or invisible, yielding approximately 12–19 valid triplets per person depending on visibility.

Each triplet is represented by a continuous feature vector that combines the absolute position of its pivot with the relative geometry of its two incident bones:

$$\mathbf{f}_{ABC} = [\mathbf{p}_B,\ \Delta_{AB},\ \Delta_{CB},\ \cos\theta,\ \sin\theta,\ \|\Delta_{AB}\|_2,\ \|\Delta_{CB}\|_2] \in \mathbb{R}^{10}, \tag{3.57}$$

where $\mathbf{p}_B \in [0,1]^2$ is the normalised image-plane position of the pivot and

$$\Delta_{AB} = \mathbf{p}_A - \mathbf{p}_B, \qquad \Delta_{CB} = \mathbf{p}_C - \mathbf{p}_B \tag{3.58}$$

are the displacement vectors from the pivot to the two endpoints. The articulation angle $\theta = \angle(A,B,C)$ is represented by the pair $(\cos\theta, \sin\theta)$ rather than by the scalar angle itself. This representation avoids the discontinuity at the angular wrap-around boundary while retaining the orientation information of the local articulation. The final two components encode the lengths of the two bones, providing scale information that is discarded by the angle alone.

In addition to these continuous geometric features, each canonical triplet type has a learnable embedding

$$\mathbf{e}^{\text{tri}}_{(t_A,t_B,t_C)} \in \mathbb{R}^{16}. \tag{3.59}$$

The layer-zero TriGAT representation is therefore

$$\mathbf{h}^{(0)}_{ABC} = \left[\mathbf{f}_{ABC} \,\|\, \mathbf{e}^{\text{tri}}_{(t_A,t_B,t_C)}\right] \in \mathbb{R}^{26}. \tag{3.60}$$

The triplet-type embedding serves the same role as the joint-type embedding in Section 3.3.2 - it allows the network to distinguish geometrically similar configurations occurring at different anatomical locations. A shoulder-elbow-wrist triplet and a hip-knee-ankle triplet may exhibit similar local angles, for example, but their structural meanings within the skeleton are different. TriGAT therefore supplies both the measured articulation and the anatomical identity of the articulation to the message-passing network.

3.11.2 Triplet graph and embedding network

The triplet graph $\mathcal{G}^{\mathrm{tri}} = (\mathcal{V}^{\mathrm{tri}}, \mathcal{E}^{\mathrm{tri}})$ contains one node for every valid triplet constructed in the scene. With full visibility, a scene containing K people therefore contains approximately $19K$ triplet nodes, which is comparable in scale to the $17K$ nodes of the corresponding joint graph. Missing or invisible joints reduce this count according to the triplet-construction rule described above.

Edges are constructed by k-nearest neighbours in the normalised image-plane position of the triplet pivots, with $k = 8$. Thus, for triplet nodes $u = (A, B, C)$ and $v = (A', B', C')$, neighbourhood membership is determined by the Euclidean distance $\|\mathbf{p}_B - \mathbf{p}_{B'}\|_2$. This preserves the spatial-graph construction used by the joint-level model while changing only the primitive represented by each node.

A GATv2 message-passing stack is applied to $\mathcal{G}^{\mathrm{tri}}$, using the same depth, hidden width, and attention-head configuration as the corresponding PG-GAT configuration. The network maps each triplet representation $\mathbf{h}^{(0)}_{ABC}$ to a 128-dimensional output embedding,

$$\mathbf{z}^{\mathrm{tri}}_{ABC} \in \mathbb{R}^{128}, \qquad \left\|\mathbf{z}^{\mathrm{tri}}_{ABC}\right\|_2 = 1, \tag{3.61}$$

so that the resulting triplet embeddings lie on the unit hypersphere.

Training uses the same margin-based cosine contrastive objective as Equation (3.27), but the positive and negative pair sets are constructed over triplet nodes rather than individual joints. Each triplet inherits the ground-truth person identity of its pivot joint B. Consequently, two triplets form a positive pair when their pivots belong to the same person and a negative pair otherwise. The loss therefore encourages all local articulation descriptors belonging to one person to occupy a common region of the embedding space while separating triplets associated with different people.

This construction keeps the supervision and embedding objective aligned with the joint-level PG-GAT comparison: the principal change is the information represented by a graph node. TriGAT supplies the attention network with an explicit local articulation descriptor, whereas PG-GAT must infer comparable relationships through message passing between individual joint nodes.

3.11.3 Voting from triplet clusters to joint labels

At inference, the triplet embeddings are first grouped using k-means with $k = K$, producing one cluster label for each triplet. These triplet-level assignments must then be converted back to per-joint labels so that the result can be evaluated using the same PGA metric as the joint-level models.

A joint can appear in several triplets, and each of those triplets may receive a different cluster identifier from k-means. A deterministic voting rule is therefore used to produce the final joint assignment. Triplets in which the joint acts as the pivot receive twice the voting weight of triplets in which the same joint appears as one of the two outer nodes. This gives greater influence to the triplet whose articulation is centred on that joint. If the weighted vote is tied, the lowest-numbered cluster identifier is used as the tie-break.

After voting, the predicted cluster identifiers are aligned with the ground-truth person labels using the Hungarian matching procedure described in Equation (3.21). PGA is then calculated in the same way as for the other grouping methods in this chapter.

The voting stage is also the main weakness of the TriGAT design. A triplet-level error does not remain local to a single representation - because one triplet contains three joints, an incorrect triplet assignment can influence the final labels of several joints. Small errors in the triplet embedding can therefore be amplified when the predictions are converted back to the joint level. The experiments in Chapter 4 show this effect clearly, and Section 5.2.4 discusses the resulting failure mode in more detail.

3.11.4 Training schedule

TriGAT is trained for 50 epochs on the synthetic training split using the optimiser, learning-rate schedule, weight decay, and gradient-clipping settings of Section 3.5.3. The training budget is matched to the corresponding PG-GAT experiment so that the comparison reported in Chapter 4 does not confound the change in graph primitive with additional optimisation time.

At inference, the trained network produces one embedding for each valid triplet in the scene. These embeddings are partitioned using k-means with $k = K$, following the same oracle-person-count convention used by the joint-level PG-GAT evaluations. The resulting triplet-level cluster assignments are then converted to per-joint labels using the weighted voting procedure of Section 3.11.3, after which Hungarian matching and PGA are computed according to Equation (3.21). This preserves the common evaluation endpoint used throughout the chapter while isolating the effect of replacing individual joints with articulation-aware triplets as the message-passing primitive.

3.12 INTEGRATION: DETECTION, K ESTIMATION, AND THE END-TO-END PIPELINE

The preceding sections develop PG-GAT under a controlled evaluation setting in which the detected keypoints and the number of people K are provided independently of the grouping model. This section closes the resulting inference loop by replacing those assumptions with the components required for an operational bottom-up pipeline. It describes the upstream keypoint detector that supplies PG-GAT with detections, the K-estimation head that predicts the number of people from the learned representation, and the end-to-end evaluation protocol used to produce the detector-independent result reported in Chapter 4.

The resulting pipeline consists of three distinct stages: keypoint detection, PG-GAT embedding, and grouping. The detector first produces a set of keypoint coordinates and joint-type labels. PG-GAT consumes these detections without access to the source image and maps each keypoint to an identity-discriminative embedding. The number of people is then estimated from the learned representation, allowing k-means to recover the final per-person partition without access to the ground-truth K. Figure 3.4 shows this arrangement.

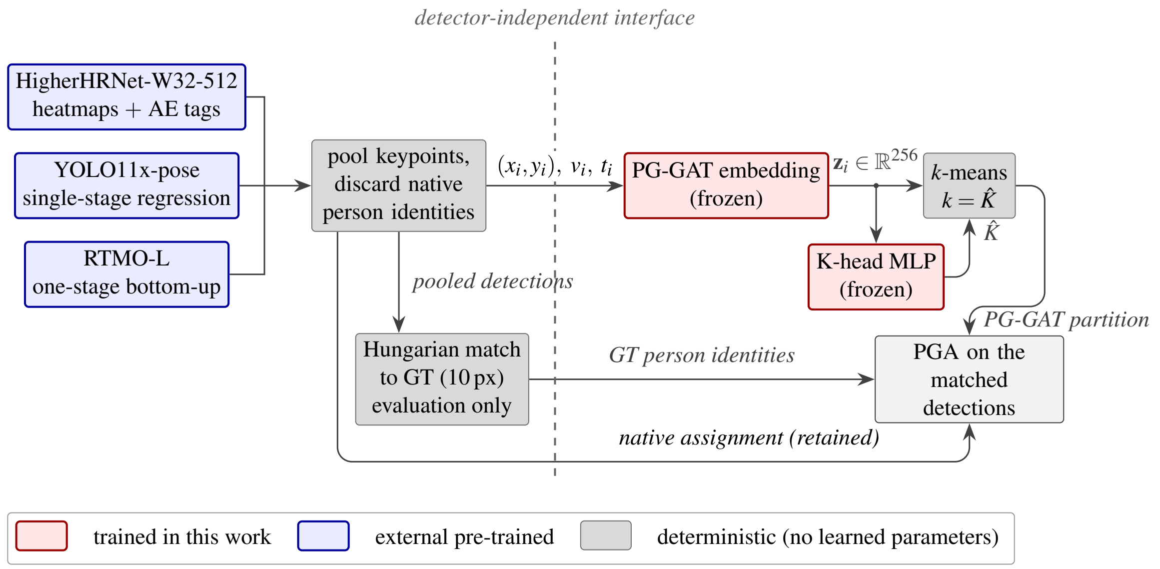

Figure 3.4. The detector-independent interface of the end-to-end pipeline. Each of the three detectors evaluated in Chapter 4 supplies keypoints - their native person assignments are discarded, and only keypoint coordinates, visibility flags, and joint types cross the interface (dashed line) into the frozen PG-GAT network. The frozen K-head predicts $\hat{K}$ from the embeddings and k-means produces the partition. Ground truth enters only for evaluation - detections are matched to annotations by Hungarian assignment at 10 pixels, and PGA is computed on the matched set for both the PG-GAT partition and the detector's retained native assignment.

This separation is central to the detector-independent claim of the method. PG-GAT is trained as a grouping module rather than as part of a joint detection-and-grouping network - its interface consists only of detected keypoints and their joint types. Consequently, the upstream detector can be replaced without modifying the PG-GAT architecture or jointly retraining the embedding network. The end-to-end experiment in this section tests how well that modularity survives the transition from annotated keypoints to the imperfect detections produced by a real bottom-up pose estimator.

3.12.1 Keypoint detection backbone

The detector used in every end-to-end evaluation in Chapter 4 is HigherHRNet-W32 at an input resolution of 512 pixels [3, 2]. The model is integrated through the `simple-HigherHRNet` wrapper using the publicly available COCO-trained weights. The W32 configuration is the smallest HigherHRNet variant used here and provides a practical bottom-up detector against which the additional cost and grouping performance of PG-GAT can be measured.

HigherHRNet is chosen deliberately, rather than replaced by a more recent pose estimator such as DEKR [8] or ED-Pose [12]. Its associative-embedding (AE) mechanism [7] provides a canonical joint-trained reference for bottom-up keypoint grouping. In addition to predicting keypoint heatmaps, AE predicts a scalar tag at each detected keypoint location. During training, tags belonging to the same person are encouraged to be similar, and tags belonging to different people are encouraged to separate. At inference, these learned tags provide the identity signal used to assemble the detected joints into per-person poses.

This comparison places the modularity claim of PG-GAT under a stronger test than comparison against another detached grouping method. The AE tag representation is learned jointly with the HigherHRNet detector on the COCO training distribution, whereas PG-GAT receives only the detached keypoint candidates after detection. PG-GAT must therefore recover identity structure without access to the detector's internal image features or to the joint optimisation that produced the detections.

HigherHRNet is frozen for all experiments - its weights are not fine-tuned, its AE head is not replaced, and the detector is not retrained jointly with PG-GAT. The network produces per-joint-type heatmaps, after which its standard non-maximum-suppression procedure extracts candidate keypoints. The candidate set supplied to PG-GAT is the same candidate set available to HigherHRNet's native AE grouping, ensuring that differences in the grouping comparison are not caused by different detection inputs.

For evaluation, detected keypoints are matched to annotated COCO keypoints using Hungarian assignment subject to a maximum image-plane distance of 10 pixels. Each successfully matched detection inherits the ground-truth person identity of its matched annotation. PGA is then computed using Equation (3.21) for the PG-GAT and AE grouping outputs over this common set of matched detections. Restricting the comparison to the same detected-and-matched keypoints isolates the grouping stage from the upstream localisation error and gives both methods the same evaluation denominator.

3.12.2 Detector-swap protocol

The modular design of PG-GAT implies that the upstream keypoint detector can be replaced without retraining the grouping module. The detector-swap experiment tests this property empirically, rather than treating detector independence as a consequence of the interface alone. Two additional detectors from different architectural families are evaluated: YOLO11x-pose [20], a single-stage instance-regression model, and RTMO-L [21], a one-stage bottom-up pose estimator. Together with HigherHRNet, these models provide three substantially different sources of keypoint detections on which to test the same frozen grouping network.

Both YOLO11x-pose and RTMO-L return keypoints already assembled into per-person skeletons. To reproduce the ungrouped input assumed by PG-GAT, this native identity information is removed before the grouping network is invoked. All keypoints above the detector-specific confidence threshold are pooled into a single detection set, while retaining only their coordinates, joint types, and confidence information. The pooled detections are then matched to ground-truth keypoints using the same per-joint-type Hungarian assignment with a maximum 10-pixel distance used in Section 3.12.3. Consequently, the evaluation protocol is unchanged when the upstream detector is swapped.

The discarded per-person assignments are retained separately for evaluation. On the same matched subset of detections, these assignments are scored against the ground-truth person identities to obtain each detector's *native grouping* accuracy. This provides a detector-specific reference - PG-GAT is compared not only across different detection distributions but also against the grouping information produced natively by the detector that generated those keypoints.

For every detector-swap experiment, PG-GAT uses the same frozen headline checkpoint and the same frozen K-estimation head used in the HigherHRNet evaluation. No parameters are updated, and PG-GAT receives no detector feature maps, instance assignments, or information about the detector's training procedure. The swap evaluation is therefore zero-shot with respect to both YOLO11x-pose and RTMO-L - only the detected keypoint set and its observed joint types cross the interface between detector and grouping module.

The RTMO evaluation uses the publicly released `body7` weights, which are trained on a multi-dataset corpus rather than exclusively on COCO. Its native-grouping result therefore benefits from a broader training distribution than the COCO-only detector references. This difference does not affect the zero-shot PG-GAT protocol, but it is treated as a qualification when the native-grouping results are compared in Section 4.9.5.

3.12.3 Pose grouping accuracy on detected keypoints

The end-to-end protocol computes PGA according to Equation (3.21) on the subset of detections that can be matched to ground-truth keypoints. Detections whose nearest valid assignment exceeds the 10-pixel matching threshold are excluded from both the numerator and denominator. The resulting metric therefore measures grouping quality conditional on successful detection, rather than conflating grouping errors with detector recall or localisation failure.

For the HigherHRNet evaluation, two grouping mechanisms are applied to the same matched detections. The first is HigherHRNet's native associative-embedding grouping, which provides the joint-trained reference and reaches a PGA of 0.9847 under this protocol. The second is the PG-GAT embedding of Section 3.8, followed by k-means clustering. PG-GAT is evaluated both with oracle K and with K predicted by the estimator described below. Because the underlying detection set and evaluation denominator are identical, the difference between the AE and PG-GAT results isolates grouping quality from upstream detection effects.

3.12.4 K estimation

An autonomous grouping pipeline cannot assume that the number of people K is known in advance. Three approaches to estimating K were investigated. The first provides a classical non-parametric baseline, the second exposes an undesirable interaction between count prediction and representation learning, and the third is the decoupled design used in the final pipeline.

Eigengap: The first approach estimates K spectrally from the learned embeddings. An affinity matrix is constructed using an RBF kernel on pairwise embedding distances, from which the graph Laplacian and its ordered eigenvalues are computed. The predicted cluster count is taken from the largest consecutive eigengap [61]. In practice, the contrastive PG-GAT embeddings exhibit a dominant first spectral gap corresponding approximately to the distinction between one connected population and multiple separated groups. This causes the heuristic to favour very small values of K and, in particular, to underpredict scenes containing two people. Eigengap estimation is therefore retained only as a non-learned baseline.

Joint training: The second approach introduces a small MLP that predicts K jointly with the PG-GAT embedding network. For each scene, the node embeddings are summarised by concatenating their mean-pooled representation, max-pooled representation, and the scalar keypoint count normalised by the 17 COCO joint types. The MLP predicts the person count and is trained with an L1 loss against the ground-truth K. Because the head is optimised jointly with PG-GAT, the count-prediction gradient is allowed to propagate into the embedding network.

This configuration achieves effectively perfect K estimation on the synthetic training distribution but degrades the representation needed for grouping. In particular, the global pooling statistics are encouraged to encode scene-level count information, while the contrastive objective requires the individual node embeddings to preserve person-level identity separation. Allowing both objectives to modify the same representation reduces the quality of the contrastive embedding and consequently lowers grouping performance. Joint training is therefore retained as a documented negative result rather than as the final design.

Frozen-backbone training: The final approach separates count estimation from representation learning. A trained PG-GAT checkpoint is loaded, and all embedding-network parameters are frozen. A newly initialised K-head MLP is then trained for 50 epochs on pooled features derived from these fixed embeddings, using the same L1 objective against the ground-truth person count. No K-estimation gradient reaches PG-GAT, and no contrastive objective is required during K-head training. The identity-separation structure of the original embedding is therefore preserved, while the lightweight head learns only the scene-level mapping required to estimate K.

The resulting K head contains 35,009 trainable parameters and converges in under one minute on the synthetic training split. Its prediction accuracy is reported separately in Chapter 4, allowing the autonomous-pipeline result to be decomposed into embedding/grouping error and K-estimation error.

The same separation of objectives is retained for real-data evaluation. Because COCO differs substantially from the synthetic dataset in keypoint visibility, missing detections, and person-density statistics, the COCO K head is trained on embeddings produced from COCO samples rather than reusing the synthetic-trained count head. PG-GAT itself remains frozen during this stage. The adaptation therefore changes only the lightweight count estimator and does not alter the grouping representation whose detector independence is under evaluation.

3.12.5 The autonomous pipeline

The complete autonomous pipeline, illustrated in Figure 3.5, composes the components above into an inference system that consumes a raw image and produces per-person keypoint assignments without requiring the ground-truth person count or person identities. HigherHRNet first detects the candidate keypoints and their joint types. The frozen PG-GAT network maps these detections to identity-discriminative embeddings, the frozen-backbone K-head predicts the number of people $\hat{K}$ from those embeddings, and k-means with $k = \hat{K}$ produces the final partition. Ground-truth annotations enter only after inference, when detected keypoints are matched to annotations for evaluation and PGA is computed according to Equation (3.21).

The autonomous result is reported separately from the oracle-K evaluation in Chapter 4. This distinction separates two sources of error that would otherwise be conflated. In the oracle-K setting, k-means is supplied with the true number of people, so the result measures the quality of the PG-GAT embedding and its resulting partition independently of count-estimation error. This is the setting in which the detector-independence gap of 0.0256 is reported. The autonomous setting instead replaces the oracle with the learned K-head and therefore measures the operational performance of the complete pipeline. Reporting both prevents errors in $\hat{K}$ from obscuring the grouping quality of the embedding itself, while still demonstrating the performance of the system when no oracle information is available.

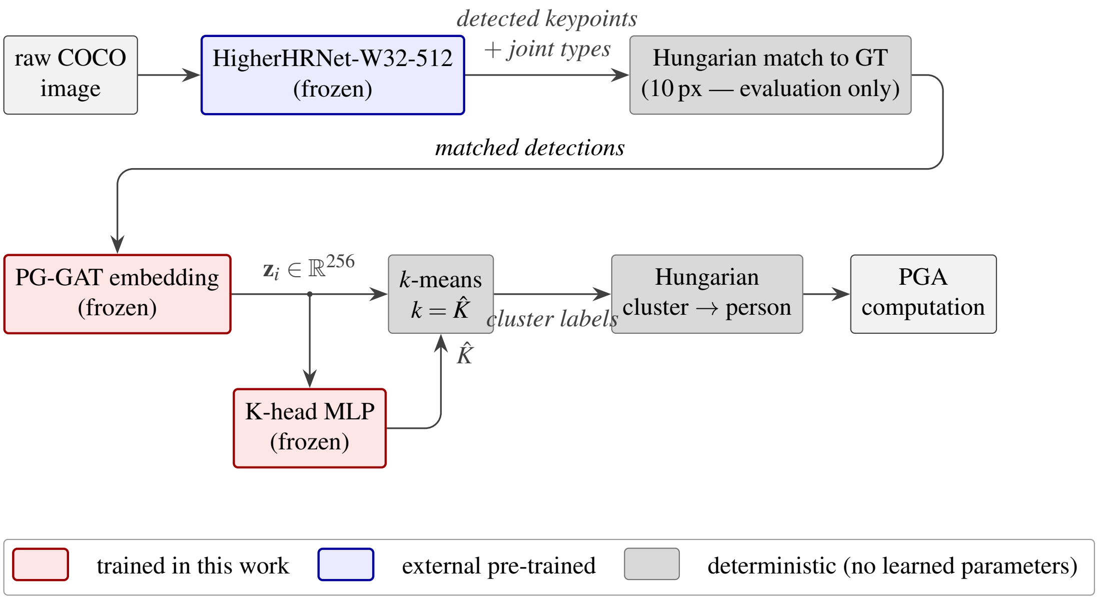

Figure 3.5. The autonomous end-to-end pose-grouping pipeline. HigherHRNet detects candidate keypoints and assigns each a joint type. The frozen PG-GAT network embeds the resulting detection set, the frozen K-head MLP predicts the number of people $\hat{K}$, and k-means with $k = \hat{K}$ produces the final per-person partition. Hungarian matching to ground-truth keypoints is performed only after inference to associate matched detections with person identifiers for PGA evaluation - it is not an input to the grouping pipeline. Components trained in this work and frozen at inference (PG-GAT and the K-head) are shown in red, the external pre-trained detector in blue, and deterministic operations with no learned parameters (k-means and evaluation-only Hungarian matching) in grey.

3.13 EVALUATION PROTOCOL AND HYPERPARAMETER DEFENCE

This section defines the evaluation protocol used for every result in Chapter 4 and records the hyperparameter-selection procedure used for the PG-GAT experiments. The same core evaluation logic is retained across three settings: synthetic evaluation, COCO evaluation with annotated keypoints, and COCO end-to-end evaluation with detected keypoints. The data distribution and source of the keypoints change between these settings, but the grouping read-out, label-alignment procedure, and PGA definition remain fixed. This common protocol makes the reported results directly interpretable as changes in grouping performance rather than changes in the evaluation procedure.

Hyperparameter selection is treated as part of the experimental methodology rather than as an informal implementation detail. The principal PG-GAT architectural, optimisation, loss, and fine-tuning choices are supported by Bayesian sweeps over explicitly defined search spaces, with the selection metric and stopping procedure fixed for each sweep. The resulting configurations are then evaluated under the same protocol as the corresponding baselines and ablations. This separates the search used to select a configuration from the evaluations used to characterise its behaviour.

Reproducibility is maintained through experiment logging - every result reported in Chapter 4 is associated with a recorded run and its complete configuration, including model hyperparameters, optimiser settings, dataset split, checkpoint-selection criterion, and evaluation parameters. Consequently, the numerical comparisons in the results chapter can be traced to concrete experimental runs rather than to unlogged manual configurations. The subsections below specify the dataset-specific evaluation regimes and the Bayesian sweep protocol used to select the final PG-GAT configuration.

3.13.1 Datasets and splits

Four evaluation views are reported across three data regimes, all using Pose Grouping Accuracy (PGA) from Equation (3.21) as the grouping metric.

Synthetic validation: The synthetic validation set consists of the 100 scenes defined in Section 3.4.1. Synthetic-validation PGA is used as the model-selection criterion during synthetic training - the checkpoint with the highest validation PGA is retained for subsequent evaluation. The same metric is used as the optimisation target for the Bayesian architecture and loss sweeps described below.

Synthetic test: The held-out synthetic test set contains a further 100 scenes that are not used for optimisation or checkpoint selection. Synthetic-test PGA is reported alongside validation performance to measure generalisation beyond the split used for model selection.

COCO val with ground-truth keypoints: The oracle-keypoint evaluation uses the COCO [1] `val2017` person-keypoint annotations rather than detections from an upstream pose estimator. Images are first filtered to contain at least one person with labelled keypoints, yielding 2,346 images. Of these, 2,307 contain sufficient valid keypoints to form a scoreable grouping instance, and the ground-truth-keypoint evaluations are reported over these 2,307 scenes.

The retained set contains both single-person and multi-person images. Single-person scenes are valid inputs, but require no non-trivial identity separation and therefore raise the absolute PGA. For transparency, Chapter 4 additionally reports a stricter diagnostic restricted to `min_people` ≥ 2, containing 1,248 multi-person images. The full filtered set remains the headline COCO evaluation because the corresponding HigherHRNet reference results are evaluated on the full distribution - the multi-person subset is reported alongside it to expose performance when an actual grouping decision is required.

COCO val end-to-end: The end-to-end evaluation replaces annotated keypoints with detections from the upstream pose estimator. Candidate images are selected using a stricter pycocotools filter requiring at least one person with at least three visible keypoints. This retains 2,258 of the 5,000 `val2017` images. An image contributes to a detector-specific PGA result only when at least two detections can subsequently be matched to ground-truth keypoints under the protocol of Section 3.12.3. Because the detections differ between models, the final number of scoreable scenes is detector-dependent: 2,165 for HigherHRNet, 2,196 for YOLO11x-pose, and 2,121 for RTMO-L.

For the primary end-to-end experiment, the keypoints are produced by HigherHRNet-W32-512 as described in Section 3.12.1. The same detected keypoints are grouped once by HigherHRNet's native associative-embedding mechanism and once by PG-GAT, allowing their difference to be attributed to the grouping stage rather than to detection recall. The detector-swap evaluations repeat the same protocol with YOLO11x-pose and RTMO-L as described in Section 3.12.2. These end-to-end experiments are the evaluations that directly exercise the detector-independent interface of PG-GAT.

3.13.2 Run identification and the W&B logging layer

Weights & Biases (W&B) [92] is used as the experiment-tracking and provenance layer for the results reported in Chapter 4. For each training run, W&B records the model configuration, training and validation metrics, source-code state, and associated output artifacts under a persistent run identifier. Every run cited in Chapter 4 belongs to the `multiperson-pose-grouping` project and is assigned an eight-character run identifier. The same identifier is used as the corresponding local output-directory identifier in the outputs/<name>/<run_id>/ hierarchy and is recorded in the experimental evidence table referenced from Chapter 4. This provides a common key linking a reported result to its dashboard record, local outputs, configuration, and checkpoint.

Three properties of this logging scheme are important for reproducibility. First, whenever a run produces a checkpoint that improves its validation criterion, the checkpoint is uploaded as a W&B artifact.

Successive versions are tracked through the auto-updated `latest` alias, while a post-hoc promotion script (`promote_best.py`) assigns the project-level `best` alias after comparing the relevant runs. Named aliases including `champion-arch`, `champion-loss`, `champion-hyperbolic`, and `meng-headline` identify the canonical artifacts associated with the principal architecture, loss, hyperbolic, and final-model results. These aliases provide stable names for the selected checkpoints, while the underlying artifact versions retain their full provenance.

Second, the complete run configuration is serialised when the W&B run is initialised. Architectural hyperparameters, optimiser settings, dataset options, random seeds, and experiment-specific switches are therefore recoverable from the experiment record independently of the local output directory. This makes the logged run, rather than a later verbal reconstruction, the source of truth for determining which configuration produced a reported result or whether a particular setting was evaluated.

Third, every run used to support a numerical result in Chapter 4 is tagged `dissertation-final`, together with a tag identifying its experimental role, such as `ablation`, `sweep`, or `finetune`. The common tag provides a single queryable subset of the W&B project containing the experimental evidence used by the dissertation. Consequently, each reported result can be traced from the chapter table to a run identifier, from that identifier to its complete configuration and metric history, and from the run to the exact checkpoint artifact used for evaluation.

3.13.3 Bayesian sweep methodology

Each principal PG-GAT hyperparameter underlying the headline model is supported by a sweep over a predefined operating range. Four sweeps are used across Section 3.8.5 and Section 3.10.4: an architecture sweep, a loss-and-regularisation sweep, a hyperbolic-architecture sweep, and a COCO fine-tuning sweep. Although their search spaces differ, the four sweeps follow the same experimental protocol.

Search strategy: All four sweeps use Bayesian optimisation through the W&B sweep agent. Hyperband early termination [87] is used where applicable to stop configurations that remain uncompetitive after the specified number of validation iterations. At the available budgets of 12–32 runs per sweep, Bayesian optimisation provides a sample-efficient alternative to exhaustive grid search, while early termination limits the compute spent on configurations that are unlikely to become competitive.

This combination keeps each search within the compute budget available on the experimental hardware.

Selection metric: The architecture, loss-and-regularisation, and hyperbolic sweeps are ranked by synthetic-validation PGA from Equation (3.21). This is also the metric used during synthetic training for checkpoint selection, so the criterion used to rank sweep configurations is consistent with the criterion used to retain the best checkpoint within each run.

The fine-tuning sweep applies the same principle after the synthetic-pre-trained architecture has been fixed - only the fine-tuning variables are allowed to change, while the architecture selected by the preceding sweep remains constant. The separation between architecture selection and fine-tuning is deliberate. It prevents improvements caused by adaptation to the COCO distribution from being attributed retrospectively to an architectural choice.

COCO oracle-keypoint and end-to-end results are computed in a separate evaluation pass on the selected checkpoints. They are therefore reported as downstream evaluations rather than additional optimisation objectives for the architecture, loss, and hyperbolic searches.

Search-space sizing: The bounds of every searched variable are fixed in the corresponding W&B sweep configuration and reproduced with the sweep results in Chapter 4. The ranges are chosen to cover the practically relevant operating region, from compact configurations comparable to those used in prior GNN-on-pose work to the largest configurations permitted by the batch-size-4 memory budget of the experimental hardware. Values outside these predefined ranges are not evaluated - the reported optimum should therefore be interpreted as the best configuration found within the stated search space rather than as a global optimum over all possible PG-GAT architectures.

Post-sweep evaluation: After each sweep, the selected configuration is associated with a canonical `champion-<sweep>` artifact alias. The corresponding checkpoint is then passed through the common evaluation pipeline to obtain the downstream COCO metrics required by Chapter 4. The resulting provenance chain is

$$\text{sweep run} \longrightarrow \text{checkpoint artifact} \longrightarrow \texttt{champion}\ \text{alias} \longrightarrow \text{reported evaluation.} \tag{3.62}$$

Each stage is represented in the W&B experiment record and, where applicable, in the corresponding local output directory.

3.13.4 The headline provenance

The headline PG-GAT checkpoint is the product of a sequential model-selection procedure rather than a single isolated training run. The architecture sweep first selects the number of message-passing layers, attention-head count, hidden dimension, output dimension, joint-type embedding dimension, and dropout. With that architecture fixed, the COCO fine-tuning sweep searches the fine-tuning duration, learning rate, and weight decay. The resulting model therefore has a two-stage provenance - architecture selection followed by fine-tuning selection.

The loss-and-regularisation sweep provides complementary evidence for the contrastive margin and repulsion-head configuration used by the model, while the hyperbolic sweep performs the corresponding architecture search for the hyperbolic variant. These searches do not alter the provenance of the headline Euclidean checkpoint - they test whether important alternative settings within their respective design axes produce a stronger configuration.

The canonical artifact produced by the final fine-tuning stage is identified by the W&B alias `meng-headline`, and its source run is identified by `b5noyg7t`. The results reported for the headline PG-GAT model in Section 4.6.2 are derived from this checkpoint and can therefore be traced from the reported metric to a specific artifact, run configuration, and training history.

This provenance is the basis of the hyperparameter-defensibility claim made in this work. The principal architectural and optimisation choices that determine the headline PG-GAT checkpoint are either selected or validated through explicit logged searches over stated ranges rather than being justified retrospectively from the final test result.

CHAPTER 4 EXPERIMENTAL RESULTS

This chapter presents the experimental results for the methods developed in Chapter 3, following the order in which the investigation progressed. Each section records the experimental setup, reports the corresponding quantitative results, and then interprets those results in the context of the pose-grouping problem. This structure preserves the experimental progression from the initial baseline and learned grouping heads through to PG-GAT and its subsequent variants.

Negative results are retained rather than omitted. The learned grouping heads, SA-DMoN variants, visual-feature augmentation, and TriGAT are therefore reported together with the measurements that led to their rejection and an analysis of their observed failure modes. Where differences between methods are small, they are treated as such rather than presented as conclusive improvements. The aim of the chapter is therefore not only to identify the strongest configuration but also to show the experimental evidence that motivated each change in direction and ultimately led to the final PG-GAT design.

4.1 BASELINE: STANDARD GAT

This section reports the headline results for the standard GAT baseline of Section 3.5 on both synthetic and COCO data, with and without the depth channel. These experiments establish the reference point against which the methods in the remainder of the chapter are evaluated.

Two results are particularly important for the experiments that follow. First, the depth-enabled model achieves near-perfect grouping on the synthetic data, indicating that exact rendered depth makes the synthetic task too easy to provide a useful basis for comparing grouping methods. Second, removing depth improves performance on COCO despite making the synthetic task more difficult. This result exposes the mismatch between the metric depth available during synthetic training and the estimated relative depth available on real images. On this basis, the no-depth configuration is adopted for all subsequent experiments unless stated otherwise.

4.1.1 Setup

Two baseline configurations are evaluated, both using the architecture specified in Table 3.1. The network consists of two GATv2 layers with four attention heads, a hidden dimension of 64, an output dimension of 128, and a 32-dimensional learnable joint-type embedding. The final node embeddings are L2-normalised onto the unit hypersphere before clustering.

Both configurations follow the training protocol of Section 3.5.3. Training runs for 50 epochs on the 400-scene synthetic training split, using AdamW with an initial learning rate of 10^{-3}, cosine annealing to 10^{-5}, weight decay of 10^{-4}, and a batch size of 4. Gradient norms are clipped at 1.0. The only experimental variable between the two runs is whether depth is included in the node features. The depth-on configuration uses the four-dimensional feature vector of Equation (3.22), whereas the no-depth configuration uses the three-dimensional feature vector of Equation (3.23).

Synthetic-validation PGA is measured on the 100-scene validation split and provides the checkpoint-selection signal during training. The selected checkpoint is then evaluated on the 2,307 scoreable COCO `val2017` scenes to measure zero-shot transfer from synthetic to real data. This COCO evaluation uses ground-truth keypoint locations, which isolates grouping performance from errors introduced by an upstream detector. Results using detected keypoints are considered separately in the end-to-end evaluation of Section 4.9.

4.1.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (k-means, GT kp)	W&B run
Standard GAT, depth on	0.9990	0.8366	`su8rmvti`
Standard GAT, depth off	**0.8783**	**0.8841**	`jkd1ln4a`

Table 4.1. Baseline GAT results with and without the depth channel. The depth-on configuration is retained as a diagnostic experiment, while the depth-off configuration establishes the baseline used for comparison with the subsequent methods in this chapter. Both results are traceable to their W&B run identifiers in the `multiperson-pose-grouping` project.

4.1.3 Interpretation

Three observations follow from Table 4.1.

1. The depth-on synthetic result leaves almost no useful headroom: The depth-on configuration reaches a synthetic-validation PGA of 0.9990, corresponding to an error rate of only 0.1% at the individual joint-assignment level. At this operating point, there is effectively no meaningful room for subsequent methods to demonstrate an improvement on the synthetic validation set. Small changes in the score would be difficult to distinguish from run-to-run or clustering variation rather than a substantive improvement in the embedding. The depth-enabled synthetic setting is therefore unsuitable as the primary measurement regime for comparing the methods developed in this chapter. The result is retained as a diagnostic, but no subsequent method uses depth unless explicitly stated otherwise.

2. Depth substantially weakens synthetic-to-COCO transfer: The limitation becomes clearer when the same checkpoint is evaluated on COCO. During synthetic training, the depth channel contains metric depth produced directly by the Blender renderer. For COCO, the corresponding channel is obtained from the relative monocular depth estimated by MiDaS [82], as described in Section 3.4.2. Although both signals are normalised before entering the network, they originate from substantially different measurement processes and have different error characteristics. The model can therefore exploit a highly reliable synthetic cue that does not transfer cleanly to the real-image domain.

This mismatch is reflected in the results. PGA falls from 0.9990 on synthetic validation to 0.8366 on COCO, a difference of 0.1624, or 16.24 percentage points. More importantly, removing the depth channel raises COCO PGA from 0.8366 to 0.8841, an absolute improvement of 0.0475. This result motivates the depth-removal decision of Section 3.4.4 - all subsequent methods operate without depth, so that their performance does not depend on a cue whose quality differs substantially between the synthetic and real domains.

3. The no-depth result establishes the working baseline: Without depth, the standard GAT achieves a COCO validation PGA of 0.8841 using k-means on ground-truth keypoints over the 2,307-scene evaluation set. This is the working baseline for the remainder of the chapter. Subsequent learned grouping heads and embedding modifications are evaluated against this result under the common protocol established in Chapter 3.

Removing depth does make the synthetic task considerably harder - synthetic-validation PGA decreases from 0.9990 to 0.8783, a drop of 0.1207. At the same time, COCO performance increases from 0.8366 to 0.8841. The two changes are consistent with the intended effect of the intervention. Exact rendered depth had made the synthetic problem unusually easy while providing a representation that transferred poorly to the real-image setting. Removing it produces a more difficult synthetic grouping problem but a stronger real-data baseline. The no-depth configuration is therefore used as the comparison point for the experiments that follow.

4.2 FAILED LEARNED GROUPING HEADS

This section reports the results for the three learned grouping heads introduced in Section 3.6: DEC, slot attention, and graph partitioning. Each head is trained jointly with the standard GAT backbone of Section 3.5, using the same no-depth input configuration and matched optimisation settings. Their performance is compared against the k-means read-out of the no-depth baseline from Section 4.1.

None of the three learned heads improves on the corresponding k-means baseline. Although the heads approach the grouping problem in different ways, their results point in the same direction - replacing the clustering read-out does not address the main source of error in the pipeline. This provides the first experimental indication that the quality of the embedding, rather than the choice of grouping head, is the more important constraint. The investigation therefore shifts first to the graph-structural objective explored by SA-DMoN in Section 4.3, and subsequently to modifying the embedding network itself through PG-GAT in Section 4.4.

4.2.1 Setup

All three learned heads use the no-depth node features of Equation (3.23) and are trained jointly with the GAT backbone for 50 epochs. Unless stated below, the optimiser, learning-rate schedule, batch size, and other training settings are identical to those of the baseline in Section 3.5.3. The head-specific loss is added to the contrastive embedding loss with weight 1.0 in each case. Preliminary isolation experiments explored alternative head-loss weights, but none produced a configuration that exceeded the baseline. Further tuning of this coefficient was therefore not pursued, and the remaining experimental budget was directed toward the later PG-GAT investigation.

The principal head-specific settings are as follows.

- **DEC.** The Student's t distribution uses one degree of freedom ($\alpha = 1$), following the original

DEC formulation of Xie et al. [16]. This setting was also numerically stable at the embedding scale produced by the baseline GAT. The target distribution is refreshed every 100 optimisation steps - refreshing it at every step produced cluster collapse during the preliminary isolation experiments. Because the number of people varies between scenes, a single set of cluster centres cannot be shared across the dataset. The centres are therefore re-initialised for each scene using k-means++, as described in Section 3.6.1.

- **Slot attention.** The slot-attention head uses seven refinement iterations rather than the three iterations used in the original formulation. Preliminary tests with three iterations produced unstable slot assignments late in training, whereas seven iterations combined with a decaying learning rate gave more stable convergence. The learning rate follows the same cosine schedule from 10^{-3} to 10^{-5} used by the surrounding training pipeline. These settings were fixed before the comparative experiments reported below.
- **Graph partitioning.** Pairwise affinity probabilities are thresholded at 0.5 during inference before connected components are extracted. To account for the imbalance between same-person and different-person pairs, the binary cross-entropy loss uses $\texttt{pos\_weight} = \min(10, n_{\text{neg}}/n_{\text{pos}})$. For a fully visible five-person synthetic scene, different-person pairs outnumber same-person pairs by approximately 4 : 1. The adaptive weight therefore prevents the substantially larger negative class from dominating the loss, while the upper bound of 10 prevents unusually imbalanced scenes from producing excessively large positive-class weights.

4.2.2 Per-head results

Configuration	Synth val PGA	COCO val PGA (k-means, GT kp)	W&B run
Standard GAT (baseline, depth off)	0.8783	0.8841	`jkd1ln4a`
GAT + DEC	0.8608	0.8798	`5q0p9it9`
GAT + slot attention	0.9025	0.8254	`2kq3i8yv`
GAT + graph partitioning	0.9714	0.8366	`s3qr04xm`

Table 4.2. Synthetic-validation and COCO transfer results for the three learned grouping-head experiments compared with the no-depth GAT baseline. The COCO column evaluates k-means on the embeddings produced after joint training with each head, rather than the head's own grouping output. This separates changes in embedding quality from the performance of the learned read-out itself. The corresponding native COCO read-outs are lower for slot attention (0.7986) and graph partitioning (0.7514), as discussed in the text. None of the learned-head configurations exceeds the baseline COCO PGA of 0.8841.

4.2.3 Interpretation

The three rows of Table 4.2 show different behaviours, but none produces an improvement that transfers to COCO. Together, they motivate the shift away from increasingly complex grouping heads and toward modifications of the embedding network.

DEC: sharpening provides little additional signal: DEC reaches a synthetic-validation PGA of 0.8608, which is 0.0175 below the baseline. On COCO, k-means applied to the jointly trained embeddings reaches 0.8798, only 0.0043 below the baseline value of 0.8841. The relatively small change on COCO suggests that the DEC objective has only a limited effect on the representation learned by the contrastive GAT.

This behaviour is consistent with the limitation identified in Section 3.6.1. The DEC loss does not use ground-truth person labels directly. Instead, its target distribution is constructed from the current soft assignments and encourages already-confident assignments to become sharper. In this application, per-scene k-means++ initialisation already provides reasonable centres when the GAT embedding is well separated, leaving relatively little for the DEC objective to improve. Conversely, when the initial partition places keypoints from different people together, the self-generated target contains no external identity information that would explicitly correct that assignment. Controlled experiments on Gaussian-blob embeddings at a comparable joint count show the same behaviour, with DEC plateauing at approximately 0.76 despite additional optimisation steps. The result therefore provides no evidence that adding DEC improves either the baseline embedding or its COCO transfer.

Slot attention: a synthetic improvement that does not transfer: Slot attention reaches a synthetic-validation PGA of 0.9025, an improvement of 0.0242 over the no-depth baseline. It is therefore the only learned-head experiment in Table 4.2 to improve on the baseline synthetic score. The improvement does not carry over to COCO. The slot head itself reaches a PGA of 0.7986, while applying k-means to its jointly trained embeddings recovers 0.8254. Both are below the baseline COCO result of 0.8841.

The difference between the synthetic and COCO results indicates that the additional supervision has encouraged a representation that is useful on the training distribution but less robust under transfer. One possible contributor is the absence of an explicit joint-type exclusivity constraint in the slot mechanism. Slots compete for keypoints according to their learned representations, but the assignment procedure does not directly encode facts such as two detections of the same joint type belonging to different people. This constraint becomes particularly useful when spatial or embedding evidence is ambiguous. The present experiment does not isolate that factor independently - however, it is treated as a motivation for the type-aware PG-GAT experiments rather than as a demonstrated cause of the transfer failure.

Graph partitioning: the largest synthetic gain and the largest transfer loss: Graph partitioning produces the strongest synthetic result of the three learned-head experiments, reaching 0.9714 PGA compared with 0.8783 for the baseline, an absolute improvement of 0.0931. Its behaviour on COCO is markedly different. The partition head's own read-out reaches only 0.7514, 0.1327 below the baseline, while k-means applied to the jointly trained embeddings recovers to 0.8366 but still remains 0.0475 below the baseline.

The strong synthetic result is consistent with the dense supervision provided by binary classification over all joint pairs. Unlike a direct K-way assignment, every same-person and different-person pair contributes a training signal, and the connected-components read-out does not require the number of people to be supplied in advance. The large reduction on COCO, however, shows that this synthetic advantage does not survive the domain shift. The thresholded affinity read-out is particularly sensitive to changes around the decision boundary - incorrectly accepting even a small number of cross-person edges can merge otherwise correct components. The fact that k-means on the same embeddings improves the COCO score from 0.7514 to 0.8366 supports the conclusion that the learned read-out contributes part of the failure. At the same time, the recovered score remains below the 0.8841 baseline - showing that joint training with the partition loss has also produced an embedding that transfers less effectively.

Cross-cutting finding: improving the read-out is not sufficient: Despite their different objectives, none of the three learned heads produces a COCO improvement over the standard GAT followed by k-means. DEC changes the baseline only slightly, while slot attention and graph partitioning produce substantial synthetic improvements that reverse under transfer. In the latter two cases, returning to k-means at evaluation time does not recover the original baseline performance, indicating that the auxiliary head losses also alter the underlying representation.

These experiments do not prove that no learned grouping head could improve pose grouping. They do show that three substantially different learned alternatives fail to improve the baseline under the evaluation protocol used here, despite two of them producing clear gains on synthetic data. This shifts the experimental focus toward the representation itself. Sections 4.3–4.4 therefore investigate whether changing the information encoded by the graph and its embedding network, while retaining the simple k-means read-out, produces improvements that survive the synthetic-to-COCO transfer.

4.3 SA-DMON

This section reports the results for the ten SA-DMoN configurations introduced in Section 3.7. The experiments begin with vanilla DMoN and then evaluate five v1 configurations based on the skeleton-aware spatial null model. These vary the treatment of the spatial bandwidth σ and include a diagnostic configuration in which the spectral term is removed entirely. Four v2 configurations then test whether the result changes when the adjacency used by the modularity objective is decoupled from the geometric message-passing graph, when the type-exclusivity term is replaced by the entropy-based formulation, or when both changes are applied together.

Across these changes, the result remains largely unchanged. All ten configurations converge to synthetic-validation PGA values in a narrow range of approximately 0.77–0.79, below the no-depth GAT baseline of 0.8783. Neither changing the bandwidth treatment nor modifying the adjacency and type-loss formulations produces a configuration that breaks out of this range. The consistency of the result across the two SA-DMoN generations suggests that the limitation is not associated with a single choice of σ or one particular implementation of the auxiliary losses. Instead, the experiments provide evidence that the modularity objective itself is poorly aligned with the spatial keypoint graph used for pose grouping. On this basis, modularity-driven grouping is not pursued further, and the investigation moves toward modifying the embedding network directly.

4.3.1 Setup

All ten SA-DMoN configurations use the standard GAT backbone of Section 3.5 and otherwise retain the optimiser and training settings of Section 3.5.3. The experimental variables are confined to the DMoN head and, for the v2 variants, the adjacency matrix supplied to the spectral objective. This keeps the underlying embedding architecture fixed while testing whether changes to the modularity formulation improve the resulting grouping.

The v1 configurations are trained for 150 epochs. This longer schedule was retained for variants whose head losses were still changing at epoch 100, so that an apparent plateau was not caused simply by terminating training too early. The v2 configurations are trained for 100 epochs, after preliminary runs showed that their head losses had already converged within this interval. The difference in training length therefore reflects the observed convergence behaviour of the two groups rather than a change in the optimisation objective.

The spectral-loss coefficient $\lambda_{\text{spectral}}$ varies between configurations and is reported with the corresponding results below. The orthogonality and collapse-prevention terms are both assigned a fixed weight of 0.1, following the DMoN formulation of Tsitsulin et al. [19]. The supervised cross-entropy term retains a weight of 1.0 throughout. Consequently, the ablation isolates the treatment of the modularity signal while keeping the remaining loss weights and GAT training configuration fixed.

4.3.2 Headline numbers

ID	Configuration	Synth val PGA	COCO val PGA	W&B run
	Standard GAT baseline (depth off)	0.8783	0.8841	jkd1ln4a
Exp 3-bis	Vanilla DMoN, $\lambda_{\text{spectral}} = 0.1$	0.7879	0.8657	124ykt2c
Exp 4	SA-DMoN v1, learnable σ, unconstrained	0.7872	0.8658	69yfqshy
Exp 5	SA-DMoN v1, learnable σ, clamped $[0.05, 0.5]$	0.7918	0.8649	rvd7qaow
Exp 6	SA-DMoN v1, clamped σ, $\lambda_{\text{spectral}} = 1.0$	0.7892	0.8628	2ok6u2k3
Exp 7	SA-DMoN v1, σ = per-graph median pair-wise distance	0.7769	0.8642	usjwc380
Exp 8	SA-DMoN v1, spectral term removed	0.7889	0.8684	2ucbx3e0
Exp v2a	SA-DMoN v2, feature adjacency only, $\lambda_{\text{spectral}} = 1.0$	0.7849	0.8622	x6v3wfnv
Exp v2b	SA-DMoN v2, feature adjacency only, $\lambda_{\text{spectral}} = 0.1$	0.7925	0.8677	qn7rv98j
Exp v2c	SA-DMoN v2, entropy type loss only	0.7934	0.8660	zz2sj7an
Exp v2d	SA-DMoN v2, feature adjacency + entropy type loss	0.7910	0.8652	02jqs33r

Table 4.3. Results for the ten DMoN and SA-DMoN configurations compared with the no-depth GAT baseline. COCO val PGA is k-means on ground-truth keypoints. Synthetic-validation PGA remains within $[0.7769, 0.7934]$ across all ten configurations, a total range of 0.0165, and every configuration remains below the baseline value of 0.8783. The same pattern is observed on COCO: applying k-means to the jointly trained embeddings produces PGA values between 0.8622 and 0.8684, compared with 0.8841 for the baseline. The DMoN heads' own COCO read-outs are lower still, ranging from 0.8107 to 0.8213. Across the tested configurations, neither the skeleton-aware null model, the treatment of σ, the spectral-loss weight, feature-based adjacency, nor the entropy type loss produces an improvement over the standard GAT baseline.

4.3.3 Interpretation

The ten configurations in Table 4.3 are most informative when considered as a sequence of attempts to change the same underlying objective. The dominant result is not the ranking of individual configurations, but the narrow range in which all ten remain. Three observations follow.

1. The plateau is the main result: Every DMoN and SA-DMoN configuration reaches a synthetic-validation PGA between 0.7769 and 0.7934, giving a total spread of only 0.0165. This remains true despite substantial changes to the modularity formulation. The spatial bandwidth σ is made learnable, constrained, or scene-dependent; the spectral contribution is re-weighted or removed; the adjacency supplied to the modularity objective is changed from geometric to feature-based; and the type-related loss is reformulated. None of these changes moves the result outside a band of approximately two PGA points.

More importantly, the entire band lies below the no-depth GAT baseline of 0.8783. The best SA-DMoN result, 0.7934, remains 0.0849 below the baseline. The same pattern appears after transfer to COCO, where k-means on the jointly trained embeddings remains within $[0.8622, 0.8684]$, compared with 0.8841 for the standard GAT. The consistency of the result across the ten configurations suggests that the limitation is associated with the modularity-based training signal rather than with a single poorly chosen hyperparameter.

2. The bandwidth experiments expose a weakness in the skeleton-aware null model: Experiments 4–7 vary the spatial bandwidth σ through four different treatments: unconstrained learning, constrained learning, constrained learning with increased spectral weight, and a fixed scene-adaptive value based on the median pairwise distance. None produces a useful improvement.

The unconstrained configuration provides the clearest diagnostic. During training, the learned bandwidth grows to approximately 341. From Equation (3.37), increasing σ causes the spatial kernel to approach

$$P_{ij} \to 1, \tag{4.1}$$

so the spatial component of the proposed null model progressively loses its dependence on keypoint distance. In this limit, Equation (3.38) is governed primarily by the joint-type affinity T_{ab} rather than by the intended combination of type and spatial information. The optimiser therefore finds a regime in which the spatial part of the skeleton-aware null contributes very little discrimination.

Constraining σ to $[0.05, 0.5]$ prevents this numerical growth, but the learned value moves to the upper boundary of the allowed interval. This behaviour is consistent with the objective favouring a broader spatial kernel than the imposed range permits. The resulting PGA of 0.7918 is nevertheless only 0.0046 above the unconstrained configuration and remains well below the GAT baseline. Increasing $\lambda_{\text{spectral}}$ from 0.1 to 1.0 reduces PGA from 0.7918 to 0.7892, while replacing the learned bandwidth with the per-scene median pairwise distance reduces it further to 0.7769. Within the bandwidth treatments tested here, there is therefore no evidence of an operating point at which the skeleton-aware null produces a competitive grouping representation.

3. The v2 changes do not break the plateau: The v2 experiments test whether the v1 result is caused by two more specific design choices. The first replaces the geometric adjacency inside the spectral objective with an adjacency recomputed from the current embedding, reducing the direct similarity between the spatially constructed adjacency and spatial null. The second replaces the original type-related term with the smoother entropy formulation described in Section 3.7.

Feature adjacency with $\lambda_{\text{spectral}} = 1.0$ reaches 0.7849 (v2a). Reducing the spectral weight to 0.1 raises this to 0.7925 (v2b), a marginal 0.0007 above the best v1 configuration and far too small to alter the overall conclusion. The entropy type loss alone gives the highest synthetic PGA among the ten experiments at 0.7934 (v2c), while combining feature adjacency and entropy produces 0.7910 (v2d). Their COCO results remain similarly close, between 0.8622 and 0.8677 for the four v2 configurations.

The v2 experiments therefore do not support either proposed change as a solution to the SA-DMoN plateau. Decoupling the spectral adjacency from the geometric graph and reformulating the type loss both alter the objective, but neither produces a meaningful improvement over v1 or approaches the standard GAT baseline. Taken together with the bandwidth experiments, the results support the interpretation that Newman-style modularity is poorly aligned with the spatial-proximity graph used in this pose-grouping formulation. The graph connects joints because they are spatially close, whereas the desired communities are defined by person identity; proximity and community membership are therefore not the same structural signal.

Methodological pivot: The learned-head experiments of Section 4.2 and the SA-DMoN experiments above approach grouping from substantially different directions, yet neither produces an improvement over the standard GAT followed by k-means. This does not rule out learned grouping heads in general, but it provides little evidence that further complexity at the read-out stage is the most productive direction under the present formulation.

The investigation therefore shifts to the representation itself. PG-GAT, evaluated in Section 4.4, retains k-means as the grouping read-out and instead modifies the GAT attention mechanism to expose pose-specific structural information directly during embedding. This change tests the hypothesis suggested by the preceding experiments: that improving the representation available to the grouping stage is more effective than replacing the grouping stage itself.

4.4 PG-GAT — ISOLATION ABLATIONS

This section isolates the three PG-GAT modifications introduced in Section 3.8. Each modification is first evaluated independently on the no-depth GAT baseline, followed by a fourth configuration in which all three are enabled together. The underlying architecture is kept fixed to the settings of Table 3.1, so the differences between rows are limited to the attention mechanisms being tested.

The purpose of this ablation is to determine whether the individual PG-GAT modifications contribute useful signal under matched training conditions and whether their combination provides an additional benefit. It is not intended to establish the best PG-GAT architecture. Architecture depth, width, embedding dimension, head count, and related hyperparameters are investigated separately by the Bayesian sweep in Section 4.5. The results below should therefore be interpreted as an isolation experiment on the baseline architecture rather than as the final PG-GAT performance.

4.4.1 Setup

All four ablation configurations use the synthetic-only training procedure of Section 3.5.3 and the no-depth node features of Equation (3.23). The architecture is fixed to the baseline settings of Table 3.1: two GATv2 layers, four attention heads, hidden dimension 64, output dimension 128, and dropout 0.1. Keeping the layer stack and training schedule fixed ensures that differences between the ablation rows can be attributed to the PG-GAT attention modifications rather than to changes in model capacity or training budget.

The three modifications are controlled independently by the flags defined in Section 3.8.5: `use_type_pair_attention` for the edge-category information of Modification 1, `use_position_encoding` for the geometric edge features of Modification 2, and use_repulsion_heads for the dedicated same-type attention of Modification 3. When repulsion is enabled, one of the four attention heads is assigned to the same-type sub-layer ($H_{\text{rep}} = 1$), leaving three heads for the standard neighbourhood ($H_{\text{std}} = 3$).

Each configuration is trained for 50 epochs. No COCO fine-tuning is performed in this ablation, allowing the effect of the attention modifications to be examined before introducing an additional domain-adaptation stage. Both synthetic and zero-shot COCO results can therefore be used to assess whether a modification improves the learned representation and whether that improvement survives transfer. The effect of explicit COCO fine-tuning is considered separately in Section 4.6.

4.4.2 Headline numbers

Configuration	Type-pair	Pos-enc	Repulsion	Synth val PGA	COCO val PGA	W&B run
Standard GAT (baseline)	—	—	—	0.8783	0.8841	`jkd1ln4a`
PG-GAT, type-pair only	on	off	off	0.8825	0.8819	`8peqkcyh`
PG-GAT, pos-enc only	off	on	off	0.8825	**0.8884**	`ffl2tde3`
PG-GAT, repulsion only	off	off	on	0.8834	0.8785	`gfqkbqnw`
PG-GAT, all three (full)	**on**	**on**	**on**	**0.8896**	0.8868	`r5ekg0zl`

Table 4.4. Isolation ablation of the three PG-GAT attention modifications under matched architecture and training compute. Each modification improves synthetic-validation PGA when enabled independently, with gains of 0.0042, 0.0042, and 0.0051 over the standard GAT baseline for type-pair attention, positional encoding, and repulsion heads, respectively. Enabling all three produces the largest synthetic gain, reaching 0.8896, or 0.0113 above the baseline. Zero-shot COCO transfer is less uniform: positional encoding is the strongest isolated variant at 0.8884, while type-pair attention and repulsion alone fall slightly below the baseline. The full configuration reaches 0.8868, above the baseline but below positional encoding alone. These results motivate retaining the three mechanisms for the subsequent architecture search while treating their transfer behaviour separately from their synthetic contribution.

4.4.3 Interpretation

Four observations from the ablation are relevant to the experiments that follow.

1. Each modification contributes under the matched synthetic setting: The three single-modification configurations produce similar synthetic-validation results, reaching 0.8825, 0.8825, and 0.8834 for type-pair attention, positional encoding, and repulsion, respectively. All three are above the standard GAT baseline of 0.8783, although the individual gains are small. Enabling all three modifications produces the strongest result at 0.8896, which is 0.0062 above the best single-modification configuration and 0.0113 above the baseline.

This behaviour is consistent with the design motivation of Section 3.8.1. The three mechanisms expose different information to the attention layer: categorical joint-pair structure, relative geometric information, and a dedicated same-type neighbourhood. Their combination does not remove the gains observed when the mechanisms are introduced individually, and instead produces the highest synthetic PGA in the ablation. The experiment does not contain the three pairwise combinations, however, so it cannot separately measure interaction effects between the modifications.

2. The isolated gains are small and should not be over-interpreted: The three single-modification results lie within 0.0009 of one another, and their improvements over the baseline range from 0.0042 to 0.0051. These differences are small relative to the per-scene variation observed in PGA on the synthetic evaluation data. The individual rows are therefore not treated as evidence for a meaningful ranking between the three mechanisms.

The more useful result is that no single mechanism accounts for the performance of the full configuration. Each produces a small improvement when introduced independently, while the configuration containing all three gives the largest improvement under the same architecture and training budget. This provides the experimental basis for retaining all three mechanisms in the PG-GAT configuration passed to the subsequent architecture search.

3. Zero-shot COCO transfer is mixed: The synthetic-only models show a different pattern on COCO. Positional encoding is the strongest isolated configuration, reaching 0.8884 compared with 0.8841 for the standard GAT. Type-pair attention reaches 0.8819 and repulsion 0.8785, both slightly below the baseline. The full configuration reaches 0.8868, an improvement of 0.0027 over the baseline, but below the positional-encoding-only result.

The ablation therefore does not support a strong zero-shot COCO advantage for the complete PG-GAT modification set. Its clearest benefit at this stage is on the synthetic representation under matched compute. Section 4.6 shows that the substantially larger COCO improvement appears only after the PG-GAT architecture is scaled using the architecture sweep and subsequently adapted to COCO. A fine-tuned version of each isolated modification would be required to determine how much each mechanism contributes to that final fine-tuned result. Such an experiment was not performed, so no per-modification attribution of the headline COCO gain is claimed.

4. These are baseline-architecture results, not the final PG-GAT configuration. The ablation deliberately retains the two-layer baseline architecture so that each row can be compared with the standard GAT without changing model depth or embedding capacity. The architecture sweep of Section 4.5 subsequently selects a larger PG-GAT configuration with three layers and an output dimension of 256. The COCO fine-tuning stage then adapts that sweep-selected model to the target distribution.

The resulting headline COCO PGA of 0.9715 (Section 4.6) should therefore not be interpreted as the direct continuation of the 0.8896 synthetic ablation result. The two measurements answer different questions. The present ablation tests whether the PG-GAT attention modifications improve the matched baseline architecture, while the later sweeps determine the architecture and fine-tuning recipe used for the final model. Together, these experiments establish the sequence used to construct the headline system - retain the three pose-specific attention mechanisms, select an appropriate architecture around them, and then adapt the resulting embedding network to the COCO distribution.

4.5 PG-GAT — BAYESIAN ARCHITECTURE AND LOSS SWEEPS

The isolation ablation of Section 4.4 evaluates the PG-GAT modifications while keeping the architecture fixed to the baseline configuration. It does not determine whether that architecture provides the most suitable capacity for the modified attention mechanism. Once all three PG-GAT modifications are enabled, the number of layers, hidden dimension, embedding dimension, and other architectural choices may affect how effectively the additional structural information is used.

This section reports two Bayesian sweeps used to select the PG-GAT configuration carried forward to the remaining experiments. The first searches the architecture parameters and determines the layer-stack configuration used by the final PG-GAT model. The second searches the main contrastive-loss and optimisation parameters, to test whether the values inherited from the standard GAT baseline remain appropriate after the attention mechanism is modified.

Both sweeps follow the Bayesian optimisation and Hyperband early-termination procedure described in Section 3.13.3. Synthetic-validation PGA is used as the selection metric throughout. The purpose of these sweeps is therefore distinct from the isolation ablation - the ablation establishes the behaviour of the three PG-GAT modifications at matched baseline capacity, while the sweeps determine the architecture and training configuration used for the PG-GAT centrepiece.

4.5.1 Architecture sweep

Setup: The architecture sweep searches six model-capacity parameters: the number of GAT layers, number of attention heads, hidden dimension, output embedding dimension, joint-type embedding dimension, and dropout rate. The complete search space is given in Table 4.5. The bounds were chosen to cover the range relevant to prior GNN-based pose methods [13, 11], while remaining feasible at batch size 4 on the experimental hardware.

The sweep is allocated a budget of 32 runs and uses Hyperband early termination at the fifth validation evaluation (epoch 25). Configurations performing poorly at that point can therefore be terminated before completing the full training schedule, reducing the cost of exploring unproductive regions of the search space. Bayesian optimisation is implemented through the Weights & Biases sweep agent [92]. The complete sweep is recorded under identifier `wg95w0k0` in the `multiperson-pose-grouping` project.

Axis	Search space
`num_layers`	$\{2,3,4,5\}$
`num_heads`	$\{2,4,8\}$
`hidden_dim`	$\{64,128,256\}$
`output_dim`	$\{32,64,128,256\}$
`joint_embedding_dim`	$\{16,32,64\}$
`dropout`	$\{0.0,0.1,0.2\}$

Table 4.5. Search space for the PG-GAT architecture sweep. The ranges cover compact configurations through to the largest models that remain feasible at batch size 4 on the experimental hardware.

Result: The sweep selects the configuration shown in Table 4.6. Relative to the baseline architecture, the selected model increases the number of GAT layers from two to three and doubles both the output embedding dimension from 128 to 256 and the joint-type embedding dimension from 32 to 64. The number of attention heads remains at four, the hidden dimension remains at 64, and dropout remains at 0.1.

The winning run reaches a synthetic-validation PGA of 0.9634. This is 0.0738 above the 0.8896 obtained by the full three-modification configuration in the baseline-sized isolation ablation of Table 4.4. The comparison shows that the capacity of the architecture surrounding the PG-GAT modifications has a substantial effect on the achievable result. The gain should not, however, be attributed to any single architectural change because the six parameters were varied jointly during the Bayesian search.

The W&B parameter-importance analysis provides some indication of which axes are associated with the sweep outcome. Within the sampled runs, `hidden_dim` receives the largest reported importance (0.639) and has a correlation of -0.80 with synthetic-validation PGA. Larger hidden dimensions are therefore associated with poorer results in the configurations sampled by this sweep, while the winning value of 64 is the smallest value in the searched range. `output_dim` is the only axis with a substantial positive correlation ($+0.50$), and the winning configuration uses the maximum searched value of 256. These statistics describe associations within the jointly sampled configurations rather than isolated causal effects - a controlled one-axis-at-a-time ablation would be required to assign the improvement to a particular parameter.

Figure 4.1 shows the outcome of each run against the six swept parameters. The same broad pattern is visible in the raw results, with higher hidden dimensions tending to occur among the weaker configurations and larger output dimensions more frequently appearing among the stronger ones. Because Bayesian optimisation adapts its sampling according to previous observations, the values on each axis are not sampled equally, and the plots should not be interpreted as balanced factorial ablations.

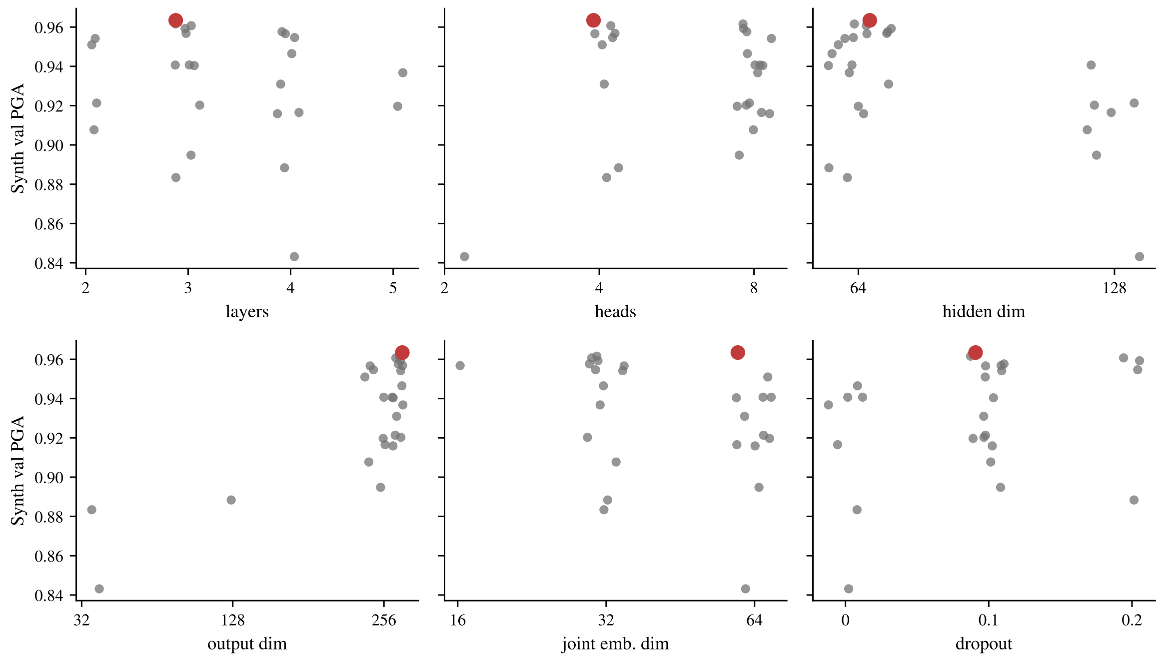

Figure 4.1. Architecture-sweep outcomes for sweep `wg95w0k0`. The best synthetic-validation PGA reached by each of the 32 Bayesian-sampled runs is plotted against the six architecture parameters, with the winning configuration (`champion-arch`, run `08d0ffde`) highlighted in red. Hyperband-terminated runs report the best validation value reached before termination. Because the Bayesian search adapts its sampling during the sweep, the candidate values on each axis do not receive equal coverage.

Axis	Selected value
`num_layers`	3
`num_heads`	4
`hidden_dim`	64
`output_dim`	256
`joint_embedding_dim`	64
`dropout`	0.1
Synth val PGA	0.9634
Source run	`08d0ffde`
W&B alias	`champion-arch`

Table 4.6. Winning configuration from the PG-GAT architecture sweep. This architecture is retained for the subsequent PG-GAT experiments, including the COCO fine-tuning sweep of Section 4.6 and the final headline evaluation.

4.5.2 Loss sweep

Setup: The loss sweep searches four training parameters: the contrastive margin γ of Equation (3.27), the number of repulsion heads H_{rep} from Section 3.8.4, learning rate, and weight decay. The architecture is fixed to the `champion-arch` configuration of Table 4.6, so the search changes the optimisation and loss configuration without introducing additional variation in network depth or width.

The search space is centred on the values inherited from the baseline training procedure: $\gamma = 0.5$, $H_{\text{rep}} = 1$, learning rate 10^{-3}, and weight decay 10^{-4}. The numerical ranges extend approximately one order of magnitude around these defaults, while the repulsion-head search tests alternative allocations of the four attention heads. A total of 24 configurations are evaluated using the same Bayesian optimisation and Hyperband early-termination procedure as the architecture sweep.

Result: The highest-scoring run produced by the loss sweep (`mkmtc8om`, alias `champion-loss`) reaches a synthetic-validation PGA of 0.9617. This is 0.0017 below the 0.9634 obtained by the existing `champion-arch` configuration using the inherited loss and optimiser settings. The difference is small, but no claim of statistical equivalence is made because the configurations were not evaluated over repeated training seeds.

The important result is that the 24-run Bayesian search does not find an alternative loss and optimisation configuration that improves on the inherited settings. The values $\gamma = 0.5$, $H_{\text{rep}} = 1$, learning rate 10^{-3}, and weight decay 10^{-4} are therefore retained. Their use in the final PG-GAT model is no longer based only on inheritance from the baseline - the sweep explicitly searches alternatives around each value and finds no configuration with a higher synthetic-validation PGA. The loss-sweep winner is consequently not used as the downstream checkpoint - the `champion-arch` configuration remains the starting point for the subsequent experiments.

4.5.3 Interpretation

The two sweeps serve different purposes. The architecture sweep searches for additional model capacity and produces a configuration with synthetic-validation PGA 0.0738 above the full PG-GAT configuration evaluated at baseline architecture sizing in Table 4.4. This establishes the three-layer, 256-dimensional configuration used in the remaining PG-GAT experiments.

The loss sweep instead tests whether the main training hyperparameters inherited from the baseline leave an obvious source of improvement unexplored. Across the 24 Bayesian-sampled configurations, none exceeds the 0.9634 obtained using the inherited margin, repulsion-head count, learning rate, and weight decay. The sweep therefore provides evidence for retaining those values rather than replacing them with another sampled configuration.

Taken together, the sweeps provide two different forms of hyperparameter justification. The six architecture parameters are selected directly by the architecture sweep, while the four loss and optimisation parameters are retained after a separate sweep fails to find an improvement over their inherited values. The final PG-GAT configuration is therefore not the result of an unrestricted set of hand-picked choices - each of its principal architecture and training parameters has been searched over an explicitly defined operating range, with the resulting runs and selected checkpoints recorded in W&B.

4.6 COCO FINE-TUNING

This section reports the COCO fine-tuning stage applied to the architecture-sweep-selected PG-GAT model. This stage adapts the synthetically trained embedding network to the COCO distribution and produces the checkpoint used for the headline COCO evaluations reported throughout the remainder of the chapter.

Two fine-tuning configurations are considered. The first uses the legacy 20-epoch recipe retained from the earlier stages of the investigation and provides a reference point for the effect of COCO adaptation. The second is selected through a 12-run Bayesian sweep over the fine-tuning hyperparameters, testing whether the inherited recipe can be improved while keeping the PG-GAT architecture fixed.

The winning sweep configuration provides the final checkpoint used for the headline evaluation. It is recorded in W&B under the alias `meng-headline` and artifact version `pg_gat_finetune_sweep:v20`. The COCO results associated with this checkpoint are therefore treated as the final PG-GAT results in the comparisons that follow.

4.6.1 Setup

Both fine-tuning configurations start from the `champion-arch` checkpoint selected in Section 4.5, which reaches a synthetic-validation PGA of 0.9634 before adaptation. Fine-tuning is performed on the COCO [1] `train2017` person-keypoint annotations, replacing the 400-scene synthetic training split with the approximately 56,000 COCO images retained by the person-keypoint dataset filter. The model architecture is unchanged during this stage.

The training objective also remains unchanged. Fine-tuning uses the contrastive loss of Equation (3.27), AdamW, and a cosine learning-rate schedule. The legacy configuration runs for 20 epochs with an initial learning rate of 10^{-4}, decaying to 10^{-6}, and weight decay 10^{-4}. It provides the inherited fine-tuning recipe against which the subsequent sweep is compared.

The Bayesian fine-tuning sweep searches three parameters: training duration, learning rate, and weight decay. The number of fine-tuning epochs is selected from $\{10, 15, 20, 30\}$, the initial learning rate is sampled log-uniformly from $[10^{-5}, 5 \times 10^{-4}]$, and weight decay is selected from $\{10^{-5}, 10^{-4}, 10^{-3}\}$. The sweep has a total budget of 12 runs and uses Hyperband early termination at the third validation evaluation (epoch 6). All runs begin from the same `champion-arch` checkpoint, so differences between them reflect the fine-tuning recipe rather than changes to the PG-GAT architecture or synthetic pre-training stage.

4.6.2 Headline numbers

Configuration	Synth val	COCO val	COCO E2E	W&B run
PG-GAT (synthetic only, `champion-arch`)	0.9634	0.9010	0.9005	`08d0ffde`
Legacy COCO fine-tune (20 ep)	0.9178	0.9565	0.9430	`ugzbgepn`
Single fine-tune of `champion-arch` (20 ep)	—	0.9695	0.9559	`g0h6pu00`
Fine-tune sweep winner (`meng-headline`)	**0.9634**	**0.9715**	**0.9591**	`b5noyg7t`
HigherHRNet AE (reference)	—	—	0.9847	—

Table 4.7. Effect of COCO fine-tuning on the architecture-sweep-selected PG-GAT. All columns report PGA; COCO val is *k*-means on ground-truth keypoints and COCO E2E uses HigherHRNet detections. The synthetic-only `champion-arch` checkpoint provides the pre-adaptation reference. All COCO fine-tuned configurations improve substantially on its COCO performance, with the Bayesian sweep winner (`meng-headline`) reaching 0.9715 PGA using ground-truth keypoints and 0.9591 in the HigherHRNet-based end-to-end evaluation. HigherHRNet's native associative-embedding grouping reaches 0.9847 on the same end-to-end protocol and is included as the detector-specific reference.

4.6.3 Interpretation

Four observations follow from Table 4.7.

1. COCO fine-tuning produces one of the largest gains in the experimental pipeline: The synthetic-only `champion-arch` checkpoint reaches a COCO PGA of 0.9010 when *k*-means is applied to its embeddings. After COCO fine-tuning, the sweep-selected checkpoint reaches 0.9715, an absolute improvement of 0.0705. The corresponding end-to-end result increases from 0.9005 to 0.9591, a gain of 0.0586.

This improvement is comparable in magnitude to the gain obtained from the architecture sweep, and substantially larger than the changes observed in the individual PG-GAT isolation experiments. The result also agrees with the transfer behaviour observed earlier in the chapter. The synthetic-only PG-GAT already produces a strong representation on its source distribution, but a substantial gap remains when that representation is applied directly to COCO. Fine-tuning on the target distribution removes much of this gap while leaving the PG-GAT architecture and contrastive objective unchanged. The result therefore identifies domain adaptation as a major component of the final COCO performance.

1a. Performance on multi-person scenes: The full COCO evaluation set contains single-person images, for which the grouping decision is trivial. As discussed in Section 3.13.1, these scenes raise the absolute PGA and can obscure performance on the cases in which grouping is actually required. The headline checkpoint is therefore also evaluated on the 1,248-scene subset containing at least two people.

On this subset, the k-means read-out reaches 0.9472 PGA, compared with 0.9715 on the full filtered evaluation set. The difference of 0.0243 shows the effect of moving to the more difficult multi-person evaluation distribution, although it should not be interpreted as a direct causal contribution of the single-person scenes because the composition and weighting of the evaluation set also change. The multi-person result provides the stricter measure of grouping performance, while the full-set value is retained as the headline figure to remain comparable with the HigherHRNet AE reference evaluated on the full distribution.

2. The fine-tune sweep provides a small improvement over the single-run recipe: A single 20-epoch fine-tune of `champion-arch` reaches 0.9695 COCO PGA. The 12-run Bayesian sweep raises this to 0.9715, an absolute difference of 0.0020. This is a small change relative to the overall variation of PGA across images, and repeated-seed experiments were not performed to establish whether the difference is systematic. The 0.0020 gain is therefore not treated as a substantive experimental finding.

The main purpose of the sweep is instead to test the sensitivity of the headline result to the fine-tuning recipe. The inherited schedule is no longer accepted without comparison - epoch count, learning rate, and weight decay are searched over the neighbourhood defined in the previous section. Figure 4.2 shows the resulting landscape. The sampled configurations occupy a relatively narrow band, with approximately 0.011 PGA between the highest and lowest synthetic-validation outcomes. Within the tested range, the final performance is therefore not strongly dependent on one narrowly chosen fine-tuning configuration.

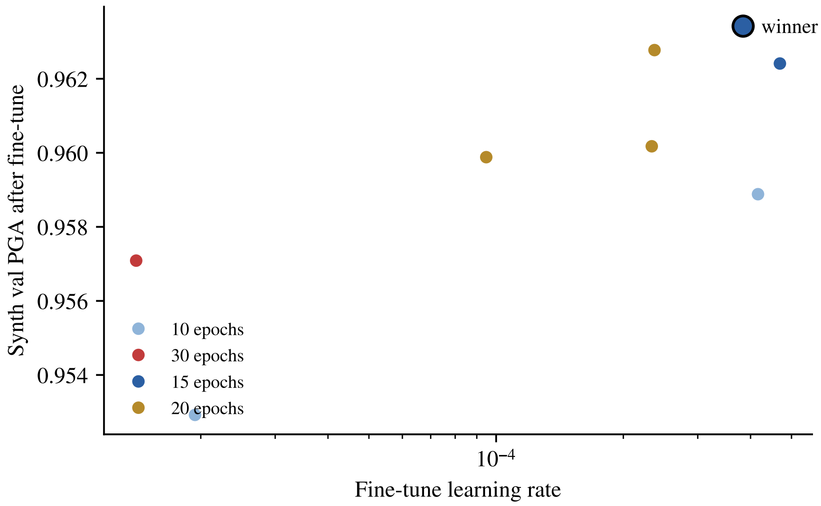

Figure 4.2. Fine-tuning sweep outcomes for sweep `i9vpg4t1` across 12 Bayesian-sampled configurations. Synthetic-validation PGA after COCO fine-tuning is plotted against the initial fine-tuning learning rate, with epoch count indicated by colour. The selected configuration (`meng-headline`, run `b5noyg7t`) is outlined in black. The observed spread across the sampled configurations is approximately 0.011 PGA, indicating relatively low sensitivity to the fine-tuning recipe within the searched neighbourhood.

The sweep winner is promoted to the `meng-headline` alias because it provides the strongest checkpoint found in the search. The small difference from the single 20-epoch fine-tune is retained for completeness rather than presented as an independent performance gain.

3. Synthetic performance is retained after COCO fine-tuning: The final checkpoint reaches 0.9634 PGA when re-evaluated on the synthetic validation set, matching the value obtained by the `champion-arch` checkpoint before fine-tuning. This value was measured by a separate evaluation of the fine-tuned checkpoint rather than copied from the pre-training run. The corresponding held-out synthetic-test PGA is 0.9580.

This result is useful because the improvement on COCO does not coincide with a measurable reduction in source-domain performance under the reported evaluation. The embedding therefore adapts to the COCO distribution while retaining its performance on the synthetic data on which it was originally trained. In this experiment, there is no evidence of catastrophic forgetting at the level measured by

PGA. The final checkpoint can consequently be used for both synthetic and COCO evaluations without maintaining separate domain-specific PG-GAT models.

4. The remaining end-to-end gap to HigherHRNet AE is 0.0256**:** The fine-tuned PG-GAT reaches an end-to-end PGA of 0.9591, compared with 0.9847 for HigherHRNet's native associative-embedding grouping on the same evaluation protocol. The remaining absolute difference is therefore 0.0256. This is the detector-independence gap carried forward into the end-to-end analysis of Section 4.9.

The comparison is demanding because HigherHRNet's AE tags are learned jointly with the detector on COCO, whereas PG-GAT receives detached keypoint detections and performs grouping as a separate module. The observed difference is consistent with the possibility that joint training provides useful detector-specific information, although the present experiment does not isolate joint training as the cause of the remaining gap. Section 5.1.3 returns to this difference when considering the trade-off between detector-specific grouping performance and a modular grouping component that can be applied independently of the upstream detector.

4.7 HYPERBOLIC PG-GAT

This section reports the results of the hyperbolic PG-GAT variant described in Section 3.10. Three individual configurations and a Bayesian hyperparameter sweep are evaluated against the two hypotheses defined in Section 3.10.4. H1 tests whether a hyperbolic embedding can match or exceed the Euclidean PG-GAT at the same output dimension, while H2 tests whether comparable grouping performance can be retained at a substantially smaller embedding dimension.

The results provide no evidence in support of H1. H2 is supported only in the synthetic-only setting, where the lower-dimensional hyperbolic representation remains competitive with the Euclidean model. This advantage does not persist after COCO fine-tuning. The hyperbolic variant is therefore treated as an efficiency trade-off rather than an improvement in grouping quality - it provides evidence that the embedding dimension can be reduced under some conditions but does not replace the Euclidean PG-GAT used for the headline results.

4.7.1 Setup

Three single-run configurations are used to test the two hypotheses. For H1, the hyperbolic PG-GAT is trained with output dimension $d = 128$, matching the Euclidean configuration used for the controlled comparison. For H2, the output dimension is reduced to $d = 32$, giving a four-fold reduction in embedding dimensionality. A third configuration, denoted Option B, starts from the $d = 32$ H2 checkpoint and fine-tunes it on COCO for 20 epochs using the legacy fine-tuning recipe of Section 4.6. This tests whether any efficiency advantage observed at the smaller dimension survives adaptation to the real-data distribution.

All three configurations use the Lorentz-model output projection and hyperbolic contrastive objective described in Section 3.10.2. The tangent-scale parameter τ is initialised to $1/\sqrt{d}$, and the geodesic contrastive margin is fixed at $\gamma_{\mathbb{H}} = 2.0$. These settings are held constant so that the primary experimental variable is the embedding dimension rather than the geometry-specific training parameters.

In addition to the three targeted runs, an eight-run Bayesian sweep is used to test the sensitivity of the hyperbolic model to output dimension and the associated architecture parameters. The sweep is reported separately below and provides broader evidence for the dimension comparison than the two hypothesis-driven single runs alone.

4.7.2 Headline numbers

Configuration	d	Synth val	COCO val	COCO E2E	W&B run
PG-GAT Euclidean (ablation)	128	0.8896	0.8868	—	`r5ekg0zl`
PG-GAT Euclidean (`champion-arch`)	256	0.9634	0.9010	0.9005	`08d0ffde`
PG-GAT Euclidean (`meng-headline`, COCO ft)	256	0.9634	0.9715	0.9591	`b5noyg7t`
H1: Hyperbolic, synthetic only	128	0.8773	0.8919	—	`aox7h3di`
H2: Hyperbolic, synthetic only	32	0.8856	0.8969	—	`g45tdxi5`
Option B: Hyperbolic, COCO fine-tuned	32	0.8900	0.9399	0.9336	`lragtfzu`
Hyperbolic sweep winner (`champion-hyperbolic`)	—	0.8795	0.8876	—	`ua4wdb3p`

Table 4.8. Hyperbolic PG-GAT results against the corresponding Euclidean reference configurations. All columns report PGA; COCO val is k-means on ground-truth keypoints and COCO E2E uses HigherHRNet detections. H1 compares hyperbolic and Euclidean embeddings at $d = 128$, while H2 tests whether useful grouping performance can be retained after reducing the embedding dimension to $d = 32$. The $d = 32$ synthetic-only model remains close to the $d = 128$ Euclidean ablation on synthetic validation and exceeds it on zero-shot COCO (0.8969 versus 0.8868), despite using a four-times-smaller embedding. After COCO fine-tuning, however, the $d = 32$ hyperbolic model reaches 0.9399 COCO PGA and 0.9336 end-to-end PGA, below the corresponding Euclidean headline results of 0.9715 and 0.9591. The hyperbolic result is therefore treated as an embedding-efficiency trade-off rather than an improvement in final grouping quality.

4.7.3 Interpretation

The individual hyperbolic configurations show a consistent pattern. Among the two synthetic-only hypothesis tests, the reduced-dimension H2 configuration performs better than H1, while the COCO-fine-tuned Option B configuration gives the strongest absolute COCO result of the hyperbolic experiments.

1. H2 provides evidence for the efficiency hypothesis, while H1 does not support the quality hypothesis: At the matched output dimension of $d = 128$, H1 reaches a synthetic-validation PGA of 0.8773, compared with 0.8896 for the Euclidean ablation under the corresponding architecture. The hyperbolic representation therefore provides no improvement at matched dimension, and H1 is not supported by this experiment.

Reducing the hyperbolic output dimension to $d = 32$ produces a synthetic-validation PGA of 0.8856, only 0.0040 below the $d = 128$ Euclidean reference. On zero-shot COCO the same configuration reaches 0.8969, compared with 0.8868 for the Euclidean reference. Although repeated-seed experiments were not performed to establish statistical equivalence, the result shows that reducing the embedding dimension by a factor of four does not produce a corresponding loss in grouping performance in the synthetic-only experiment. H2 is therefore supported as an efficiency result rather than as evidence of superior absolute accuracy.

2. The reduced-dimension advantage does not persist after COCO fine-tuning: Option B fine-tunes the $d = 32$ H2 checkpoint on COCO and reaches 0.9399 COCO validation PGA, an improvement of 0.0430 over the H2 zero-shot result of 0.8969. The Euclidean headline model improves from 0.9010 before fine-tuning to 0.9715 afterwards, a gain of 0.0705. The final hyperbolic result therefore remains 0.0316 below the fine-tuned Euclidean headline.

The important observation is that the lower-dimensional hyperbolic model benefits from COCO adaptation, but not enough to retain its competitive position once both approaches are fine-tuned. One possible explanation is that the 32-dimensional representation provides less capacity for adaptation than the 256-dimensional Euclidean headline model, although the present experiments do not isolate embedding dimension from geometry sufficiently to establish this as the cause. The result nevertheless shows that the synthetic-only efficiency finding does not translate into a competitive fine-tuned COCO result.

3. The hyperbolic sweep does not identify a stronger configuration: An eight-run Bayesian sweep over hyperbolic output dimension, network depth, and hidden dimension was conducted under sweep identifier `qpggbsdn`. The selected configuration reaches a synthetic-validation PGA of 0.8795 and a zero-shot COCO PGA of 0.8876. Its synthetic result is below both the matched Euclidean ablation (0.8896) and the targeted $d = 32$ H2 run (0.8856).

None of the configurations sampled by the sweep exceeds the matched Euclidean reference on synthetic validation. The H1 result is therefore not simply contradicted by one manually selected hyperbolic configuration - a broader search over dimension, depth, and width also fails to identify an accuracy advantage within the tested search space.

The sweep does, however, show relatively stable transfer. Its selected configuration changes from 0.8795 on synthetic validation to 0.8876 on zero-shot COCO, while the matched Euclidean ablation changes from 0.8896 to 0.8868. This does not establish that hyperbolic embeddings are generally more robust to domain shift, but it provides some evidence that their main limitation in these experiments is absolute grouping quality rather than a pronounced synthetic-to-COCO degradation.

4. The useful result is efficiency rather than higher accuracy: The hyperbolic experiments therefore support a narrower conclusion than the original quality hypothesis. A $d = 32$ hyperbolic embedding reaches 0.8856 synthetic-validation PGA and 0.8969 zero-shot COCO PGA, compared with 0.8896 and 0.8868, respectively, for the $d = 128$ Euclidean ablation. This is achieved with a four-times-smaller embedding representation.

After COCO fine-tuning, however, the comparison favours the Euclidean model - the hyperbolic Option B configuration reaches 0.9399 compared with 0.9715 for the Euclidean headline, a difference of 0.0316. The hyperbolic variant is therefore not used as the final grouping model. Its contribution is instead an efficiency observation - in the synthetic-only setting, a substantially smaller hyperbolic embedding retains competitive grouping performance. This may be useful in settings where embedding storage or downstream processing cost matters more than maximising COCO PGA. Section 5.2.3 returns to the possible reasons why this advantage does not extend to the fully fine-tuned configuration.

4.8 TRIGAT — TRIPLET-BASED PRIMITIVE

This section reports the results for the triplet-based PG-GAT variant described in Section 3.11. TriGAT replaces the single-joint node representation with the three-joint primitive of Equation (3.57) while retaining the same training schedule and downstream k-means read-out used by PG-GAT. The comparison therefore isolates the effect of changing the graph primitive from individual joints to skeleton triplets.

TriGAT does not improve on the corresponding PG-GAT configuration. Its synthetic-validation performance is lower, and the subsequent diagnostics indicate that the conversion from triplet-level clusters back to joint-level assignments is a major source of error. In particular, mistakes at the triplet level can affect multiple joints through the voting procedure of Section 3.11.3, reducing the benefit of the additional articulation information carried by the triplet features. The results below therefore treat TriGAT as a negative architectural experiment rather than an alternative to the joint-based PG-GAT used for the headline evaluation.

4.8.1 Setup

TriGAT is trained for 50 epochs on the synthetic training split, using the optimiser and learning-rate schedule of Section 3.5.3, matching the PG-GAT training budget. The triplet graph contains one node for each valid skeleton triplet. A fully visible K-person scene therefore contains approximately $19K$ triplet nodes, which is similar in scale to the $17K$ nodes of the corresponding joint graph. Missing or invisible joints reduce the number of valid triplets as described in Section 3.11.1.

At inference, k-means with $k = K$ is applied to the learned triplet embeddings to obtain a cluster assignment for each triplet. These assignments are then converted back to joint-level labels using the voting procedure of Section 3.11.3. A vote from a triplet in which the joint is the pivot receives twice the weight of a vote from a triplet in which it appears as a wing, with ties resolved in favour of the lowest-numbered cluster identifier. The resulting joint-level partition is finally matched to the ground-truth person identities using the Hungarian algorithm before PGA is computed.

4.8.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (k-means, GT kp)	W&B run
PG-GAT (synthetic only, ablation row)	0.8896	0.8868	`r5ekg0zl`
TriGAT	0.8740	0.8564	`y0tw6vc9`
Δ vs PG-GAT	−0.0156	−0.0304	—

Table 4.9. TriGAT compared with the synthetic-only PG-GAT ablation configuration under the same 50-epoch training budget. TriGAT is 0.0156 lower on synthetic validation and 0.0304 lower in the reported COCO evaluation. Both results are single-seed measurements. The TriGAT COCO result is computed over the 2,180 scenes containing at least one valid triplet, whereas the PG-GAT reference is evaluated on the full 2,307-scene ground-truth-keypoint set. The COCO difference should therefore be interpreted as indicative rather than as a strictly matched-set measurement.

4.8.3 Interpretation

The TriGAT result is negative under the matched training budget. Three observations are useful for interpreting why.

1. The result is not explained by a smaller training budget: TriGAT is trained for the same 50 epochs as the matched PG-GAT configuration, uses the same optimiser and contrastive objective, and operates on a graph of comparable size. A fully visible K-person scene contains approximately $19K$ triplet nodes compared with $17K$ joint nodes in the standard representation. The experiment therefore does not give PG-GAT an obvious advantage through additional training time or a substantially smaller graph.

Under these conditions, TriGAT reaches a synthetic-validation PGA 0.0156 below PG-GAT. The main architectural differences are the replacement of individual joints with triplet nodes and the additional step required to convert triplet-level predictions back to joint-level assignments. The negative result therefore points toward the representation and read-out introduced by TriGAT rather than a simple difference in training budget.

2. The voting stage introduces an additional source of error: Each joint can participate in several triplets, and those triplets are clustered independently at the representation level before their assignments are combined. The voting procedure of Section 3.11.3 must therefore resolve potentially different cluster labels from the triplets containing the same joint. If one or more of these triplets is incorrectly clustered, its vote can change the final joint assignment.

This creates an error-amplification mechanism that is absent from joint-level PG-GAT. A triplet-level mistake can influence more than one joint because each triplet contains three keypoints, while the same joint can also receive conflicting evidence from several overlapping triplets. Weighted voting reduces this ambiguity but does not remove it. The observed result is therefore consistent with the voting stage discarding part of the benefit gained from the richer articulation features. Without a separate oracle-voting or triplet-level accuracy ablation, however, the experiment cannot assign the full performance gap to voting alone.

3. The result does not rule out articulation features themselves: TriGAT was motivated by the observation that local joint geometry can contain information that is not represented explicitly by an individual keypoint position. The pivot angle is a rotation-invariant descriptor of local articulation, which is the original motivation for including it.

The negative result therefore applies to the particular TriGAT pipeline evaluated here - triplet nodes are embedded, clustered at the triplet level, and then converted back to joint identities through a deterministic vote. In this formulation, any benefit provided by the additional articulation information is not large enough to overcome the errors introduced elsewhere in the pipeline.

A possible alternative would be to use the triplet embeddings as context while predicting joint identities directly, avoiding the intermediate triplet-clustering and voting stages. Such a model would test the articulation signal more directly but would require a substantial change to the architecture and training objective. It is therefore left as a possible direction for future work rather than included in the present experimental investigation.

4.9 END-TO-END INTEGRATION WITH HIGHERHRNET

This section closes the experimental evaluation by replacing the ground-truth keypoint input used in the preceding COCO experiments with detections produced by HigherHRNet-W32-512 [3]. The resulting evaluation therefore measures PG-GAT under the end-to-end protocol of Section 3.12, where the grouping module operates on detected rather than annotated keypoints.

Two configurations are reported. In the oracle-K configuration, the number of people K is taken from the ground-truth annotation, while the keypoint locations themselves are supplied by HigherHRNet. This separates the performance of the PG-GAT embedding and k-means read-out from errors in estimating the number of clusters. In the autonomous configuration, K is instead predicted by the frozen-backbone K-estimation head of Section 3.12.4. The latter therefore represents the complete operational pipeline - HigherHRNet detects the keypoints, PG-GAT embeds them, the K head estimates the number of people, and k-means produces the final grouping without using ground-truth information at inference.

4.9.1 Setup

The detector used for the end-to-end evaluation is HigherHRNet-W32-512 with COCO-trained weights. It is held fixed throughout the experiment - no detector fine-tuning, head replacement, or retraining is performed. HigherHRNet produces a heatmap for each joint type, from which its internal non-maximum suppression extracts the keypoint candidates used for grouping. The same candidate set is provided to HigherHRNet's native associative-embedding (AE) grouping and to PG-GAT, ensuring that differences between the two grouping results are not caused by differences in the detected keypoints.

For evaluation, detected keypoints are matched to the ground-truth annotations using a per-joint-type Hungarian assignment with a 10-pixel distance threshold. Each successfully matched detection is thereby assigned a ground-truth person identity, which is used only to compute the metric. Unmatched detections are excluded from the PGA calculation, so the resulting score measures grouping accuracy on the matched detections rather than keypoint detection recall. Both AE and PG-GAT are evaluated against this same set of matched detections.

The PG-GAT model is the `meng-headline` checkpoint selected in Section 4.6. For the oracle-K configuration, k-means uses the ground-truth number of people in the scene. For the autonomous configuration, this value is replaced by the output of the frozen-backbone K-estimation head described in Section 3.12.4, trained on embeddings from the COCO distribution. Ground-truth identities attached during Hungarian matching are used only after grouping to compute PGA and do not enter the autonomous inference pipeline.

4.9.2 Headline numbers

Configuration	COCO E2E PGA (HigherHRNet detections)	W&B run
HigherHRNet AE (native grouping reference)	0.9847	—
PG-GAT + k-means (synthetic only)	0.8766	`r5ekg0zl`
PG-GAT + k-means (legacy 20-ep COCO fine-tune)	0.9430	`ugzbgepn`
PG-GAT + k-means (`meng-headline`, oracle K)	0.9591	`b5noyg7t`
PG-GAT + k-means (`meng-headline`, predicted K)	**0.9644**	`b5noyg7t`
Hyperbolic PG-GAT, Option B ($d = 32$, COCO fine-tuned)	0.9336	`lragtfzu`

Table 4.10. End-to-end grouping results on COCO using detections from HigherHRNet-W32-512. HigherHRNet's native associative-embedding grouping is evaluated on the same matched detections and provides the detector-specific reference. The `meng-headline` PG-GAT reaches 0.9591 PGA when k-means is given the ground-truth person count and 0.9644 when the cluster count is supplied by the frozen-backbone K-estimation head. The latter is the fully autonomous PG-GAT configuration and requires no ground-truth information at inference. The remaining gap to HigherHRNet AE is 0.0203 for the autonomous configuration.

4.9.3 Interpretation

Three observations close the experimental chapter.

1. The remaining gap to detector-native grouping is small: With the ground-truth person count supplied to k-means, the `meng-headline` checkpoint reaches an end-to-end PGA of 0.9591, compared with 0.9847 for HigherHRNet's native AE grouping. The resulting difference is 0.0256.

This gap decreases consistently over the course of the investigation. The synthetic-only two-layer PG-GAT configuration reaches 0.8766, leaving a difference of 0.1081 to the same AE reference. The synthetic-only architecture-sweep winner reduces this difference to 0.0842, the legacy COCO fine-tune reduces it further to 0.0417, and the fine-tune-sweep checkpoint reaches the final oracle-K difference of 0.0256. The progression shows that both architecture selection and adaptation to the COCO distribution contribute to closing the gap to detector-native grouping.

The 0.0256 difference should not be interpreted as a direct measurement of the cost of modularity alone. HigherHRNet AE and PG-GAT differ in their representation, training objective, and access to detector features in addition to whether grouping is trained jointly with detection. The result instead measures the remaining empirical gap between the modular PG-GAT grouping pipeline and the detector's native grouping mechanism under the evaluation protocol used here. Section 5.1.3 considers the possible sources of this difference.

2. PG-GAT demonstrates grouping independently of the detector's grouping mechanism: HigherHRNet AE reaches 0.9847 on the matched detections and provides the detector-specific reference for this experiment. PG-GAT reaches 0.9591 on the same evaluation protocol using the detector only as a source of keypoint candidates. It does not consume HigherHRNet's AE tags or internal grouping assignments, and the HigherHRNet parameters remain frozen throughout the experiment.

This result demonstrates the practical separation between detection and grouping that motivates PG-GAT. Once keypoints and their joint types have been extracted, grouping is performed by a separately trained embedding network rather than by a grouping representation learned jointly with the detector. This differs from architectures in which detection and grouping are components of the same trained model - including HigherHRNet AE and other integrated pose-estimation approaches such as DEKR [8] and ED-Pose [12]. The detector-swap experiments reported separately provide the stronger empirical test of whether this separation transfers to detectors other than HigherHRNet.

3. The autonomous pipeline slightly exceeds the oracle-K diagnostic: Replacing the ground-truth value of K with the prediction from the frozen-backbone K head produces the fully autonomous configuration. HigherHRNet supplies the keypoint candidates, PG-GAT embeds them, the K head estimates the cluster count, and k-means produces the final partition. No ground-truth quantity is required by this sequence at inference.

The autonomous configuration reaches an end-to-end PGA of 0.9644, which is 0.0053 higher than the 0.9591 obtained when k-means is given the annotation-level person count. The K head itself achieves 76.1% exact-count accuracy on the COCO validation set, rising to 91.9% when predictions within one person of the target are counted.

At first sight, a predicted cluster count outperforming the ground-truth count appears counter-intuitive. The distinction is that the ground-truth K describes the people annotated in the image, whereas k-means operates only on the keypoints that survive detection and matching. Under the 10-pixel matching threshold, HigherHRNet recovers 53,131 of 66,998 evaluated ground-truth keypoints, corresponding to 79.3% recall. The graph presented to PG-GAT is therefore an incomplete representation of the annotated scene.

One plausible explanation for the 0.0053 improvement is that the annotation-level K can exceed the number of people that are meaningfully represented by the detected keypoints in some scenes. In such cases, forcing k-means to use the annotation-level count may split otherwise coherent detected groups. The K head instead predicts from the detected embeddings and can therefore produce a count that is better aligned with the observed graph. Confirming this mechanism directly would require comparing $\hat{K}$ with the number of distinct ground-truth identities represented among the matched detections rather than with annotation-level K alone.

Regardless of the cause of the small improvement, the autonomous result establishes the operational result required by Section 3.12 - the complete pipeline can produce person-grouped keypoints without using ground-truth information at inference. Its PGA of 0.9644 leaves a difference of 0.0203 to

HigherHRNet's native AE grouping on the same evaluation protocol and is the final autonomous result of the experimental investigation.

4.9.4 Relationship between PGA and AP@OKS

The grouping baselines reviewed in Chapter 2 are primarily reported using AP under the OKS threshold family, whereas the results in this chapter use PGA. The two metrics evaluate different parts of the pose-estimation problem, and their numerical values are therefore not directly comparable.

AP@OKS evaluates the complete pose-estimation pipeline. A predicted person instance receives credit according to the similarity between its assembled keypoints and a ground-truth pose under the OKS criterion. Detection failures, keypoint localisation errors, and incorrect grouping can therefore all reduce AP. PGA instead conditions on the set of detections that have already been matched to ground-truth keypoints and evaluates whether those detections are assigned to the correct person (Section 3.12.3). It consequently removes detection recall and localisation error from the quantity being measured and isolates the grouping decision more directly.

Grouping errors also affect the two metrics differently. Under PGA, an incorrectly grouped keypoint contributes directly as an incorrect assignment. Under AP@OKS, the effect depends on how that error changes the assembled person instances. For example, exchanging keypoints between two people can reduce the OKS of both predicted poses and may therefore affect the AP contribution of both instances. A fixed change in PGA consequently has no fixed numerical correspondence to a change in AP@OKS.

HigherHRNet provides a useful reference between the two evaluation views. For the W32-512 configuration, HigherHRNet reports an AP of 0.671 on COCO `val2017` under its published single-scale evaluation with flip testing [3], while its native AE grouping reaches 0.9847 PGA under the matched-detection protocol used in this work. These values should not be interpreted as a formal decomposition of AP, since the metrics and evaluation procedures differ. They nevertheless show that, once HigherHRNet keypoints have been successfully detected and matched, its native grouping mechanism makes relatively few person-assignment errors. The substantially lower end-to-end AP must therefore reflect error sources that PGA intentionally excludes, in addition to any grouping errors that remain.

This distinction explains the use of PGA as the primary metric in the present investigation. PG-GAT replaces the grouping stage rather than the upstream keypoint detector, so PGA measures the component of the pipeline that the proposed method is designed to change. Reporting AP@OKS for the complete modular system would still be useful as a system-level evaluation, but changes in that metric would combine PG-GAT's grouping behaviour with detection and localisation errors inherited unchanged from the upstream detector. PGA is therefore used to isolate the contribution of the grouping method, while AP@OKS remains the appropriate metric for comparison of complete pose-estimation systems.

4.9.5 Detector-swap evaluation

This section reports the detector-swap experiment of Section 3.12.2. The same frozen `meng-headline` checkpoint and frozen K head are applied zero-shot to detections from two additional pose detectors belonging to different architectural families. Neither detector contributed training examples to PG-GAT or to the K-estimation head.

Detector (family)	Recall	Native	PG-GAT (oracle K)	PG-GAT (pred. K)	Gap
HigherHRNet-W32-512 (heatmap + AE)	0.793	0.9847	0.9591	0.9644	0.0203
YOLO11x-pose (single-stage regression)	0.816	0.9863	0.9647	0.9653	0.0210
RTMO-L (one-stage bottom-up)	0.774	0.9961	0.9437	0.9741	0.0220

Table 4.11. Detector-swap results on COCO `val2017`. Native denotes each detector's original person assignment evaluated on its matched-detection subset. The PG-GAT columns discard these assignments, pool the detected keypoints, and re-group them using the same frozen `meng-headline` checkpoint, with K either taken from the ground truth or predicted by the frozen K head. Gap is the difference between native grouping and autonomous predicted-K PG-GAT. The matched subsets contain 2,165, 2,196, and 2,121 scored images for HigherHRNet, YOLO11x-pose, and RTMO-L respectively. Consequently, comparisons between native and PG-GAT are matched within each detector row, while absolute PGA values should be compared across detectors with caution. The RTMO native reference uses the publicly released multi-dataset `body7` weights.

Two findings follow from Table 4.11.

1. The gap to detector-native grouping is stable across detector families: The autonomous PG-GAT pipeline trails each detector's native grouping by 0.0203, 0.0210, and 0.0220 PGA for HigherHRNet, YOLO11x-pose, and RTMO-L respectively. The spread between these three gaps is only 0.0017 - despite substantial differences in how the detectors produce and group their keypoints.

This result provides the direct empirical test of the detector-swap claim introduced in Section 3.12.2. PG-GAT and the K head are unchanged between all three evaluations and operate only on the pooled keypoint coordinates, joint types, and associated input features. In particular, neither component is retrained or adapted to YOLO11x-pose or RTMO-L. The fact that the autonomous pipeline remains within approximately 0.02 PGA of native grouping across all three detectors shows that its performance is not specific to the HigherHRNet candidate distribution on which the original end-to-end evaluation was performed.

The gap should not be interpreted as a direct numerical measurement of the cost of modularity alone. Each native grouping mechanism differs from PG-GAT in its representation, training data, objective, and access to detector-internal features. What the experiment establishes is that replacing these detector-specific assignments with the same external grouping module incurs a small, and remarkably consistent, empirical gap across the three detector families tested. For YOLO11x-pose and RTMO-L, the native person assignment is also produced as part of the detector itself rather than by a detachable grouping module. PG-GAT therefore provides a form of interchangeability that the native assignments do not.

2. The behaviour of predicted *K* is consistent with an effective-person-count effect: The difference between predicted-*K* and oracle-*K* PG-GAT changes substantially with the detector. For YOLO11x-pose, which has the highest matched-keypoint recall at 0.816, predicted *K* changes PGA by only +0.0006. For HigherHRNet at 0.793 recall, the difference increases to +0.0053, while for RTMO-L at 0.774 recall it reaches +0.0304. In the ground-truth-keypoint evaluation, where the input contains the annotated keypoints rather than an incomplete detector output, predicted *K* instead trails the true count by 0.0066.

These four operating points form a monotonic pattern - as the keypoint set becomes less complete, the annotation-level person count becomes less useful as the cluster count for the graph actually presented to PG-GAT, while the count estimated from that graph becomes relatively more useful. This behaviour is consistent with the explanation proposed in the HigherHRNet experiment. If some annotated people are only weakly represented among the surviving detections, forcing *k*-means to use the annotation-level K can introduce unnecessary clusters and split otherwise coherent detected groups. The K head, which predicts from the observed embeddings, can instead return a count closer to the number of people represented strongly enough to support grouping.

The detector swap strengthens this interpretation because the same trend appears across three independently generated detection sets and reverses in the ground-truth-keypoint control. It does not, however, prove that recall alone causes the effect - detector identity, visibility patterns, localisation errors, and the distribution of missing joints also change between the three rows. A direct test would compare both K and $\hat{K}$ with the number of distinct ground-truth identities represented by the matched detections in each scene. The present result is therefore evidence consistent with the effective-person-count explanation rather than a causal isolation of that mechanism.

4.10 MODEL SIZE AND THE MODULARITY CONTRIBUTION

This section reports the parameter counts of the main components in the pose-grouping pipeline and clarifies how model size should be interpreted when comparing PG-GAT with the detector-based reference methods. An earlier reading of the results treated the grouping network's parameter count as if it represented the size of the complete system. This is not an equivalent comparison, since the modular pipeline still requires an upstream keypoint detector at inference.

The accounting below therefore separates the parameters belonging to the grouping module from those required by the complete pose-estimation pipeline. This distinction does not change the grouping results reported in the preceding sections, but it does change how the modularity contribution should be framed - PG-GAT is a comparatively small, detachable grouping component rather than a complete pose-estimation system of the same size.

4.10.1 Parameter accounting

Component	Parameter count
PG-GAT (`meng-headline`, sweep-selected architecture)	333,552
PG-GAT (legacy two-layer architecture)	170,560
PG-GAT v2 (legacy architecture + visual projection)	197,280
MobileNetV2 backbone (frozen, PG-GAT v2 only)	$\sim$ 2,200,000
K-head (frozen-backbone MLP, `meng-headline` embedding)	35,009
HigherHRNet AE tag head	$\sim$ 561
HigherHRNet-W32 (complete detector)	28,645,331

Table 4.12. Parameter counts of the principal learned components used in the evaluation. PG-GAT is a small grouping module relative to the full HigherHRNet detector, but it is not smaller than HigherHRNet's native grouping mechanism: the AE tag head contains only approximately 561 parameters, making the `meng-headline` PG-GAT roughly 600× larger as a standalone grouping component. Complete-system comparisons must also include the upstream detector required by PG-GAT. The MobileNetV2 parameters apply only to the PG-GAT v2 visual-feature variant and are frozen during its training.

4.10.2 Correcting an early framing

An earlier version of the analysis described PG-GAT as "168× smaller than HigherHRNet" by comparing the 170,560 parameters of the legacy two-layer PG-GAT with the 28,645,331 parameters of the complete HigherHRNet-W32 model. The arithmetic is correct for those two components, but the comparison does not measure the parameter cost of the respective grouping methods. It also predates the architecture sweep - the sweep-selected `meng-headline` PG-GAT contains 333,552 parameters, making it approximately 86× smaller than the full HigherHRNet model.

Neither ratio should be interpreted as a grouping-model size advantage. PG-GAT replaces HigherHRNet's grouping mechanism rather than its detector, and the relevant native grouping component is the AE tag head, which contains approximately 561 parameters. On that comparison, the relationship is reversed - the headline PG-GAT contains roughly 600× as many parameters as the AE tag head. The earlier "168× smaller" statement therefore compared components with different responsibilities, a standalone grouping network against an entire detector and its grouping output, and overstated the model-size case for PG-GAT.

The parameter count is consequently not presented as an advantage of the proposed method. The contribution instead lies in the separation of grouping from detection - PG-GAT is a separately trained and replaceable grouping module, whereas HigherHRNet's AE mechanism is integrated into the detector and trained jointly with it. The detector-swap results of Section 4.9.5 evaluate the practical value of that separation directly.

4.10.3 What the contribution actually is

PG-GAT is substantially larger than the grouping output that it replaces. The sweep-selected model contains 333,552 parameters, compared with approximately 561 parameters in HigherHRNet's AE tag head, making PG-GAT approximately 595$\times$ larger on this component-level comparison. Parameter efficiency is therefore not an advantage of PG-GAT relative to AE.

The distinction instead lies in how the grouping mechanism is coupled to the detector. HigherHRNet's AE tags are predicted from internal detector features and are trained jointly with the keypoint heatmaps. The grouping representation is therefore tied to the HigherHRNet feature pipeline - applying the same learned mechanism to an unrelated detector would require access to compatible feature maps and additional integration or training. PG-GAT operates at a different interface. Its inputs are already-detected keypoints and their joint types, so the embedding network does not depend on the internal representation of the detector that produced them.

The parameter-count comparison is therefore secondary to the modularity being evaluated. In the fully autonomous HigherHRNet experiment of Section 4.9, PG-GAT reaches 0.9644 PGA against 0.9847 for HigherHRNet's native AE grouping, leaving a gap of 0.0203. More importantly, the same frozen PG-GAT checkpoint and K-estimation head can be applied to detections from other architectures without retraining. The detector-swap experiment of Section 4.9.5 exercises this property directly with YOLO11x-pose and RTMO-L, where the autonomous pipeline remains within 0.0210 and 0.0220 PGA of each detector's native grouping, respectively.

The resulting trade-off is explicit. PG-GAT introduces approximately 334,000 parameters to perform a task that HigherHRNet implements with a much smaller detector-integrated tag head, and it does not improve the absolute grouping accuracy of that native mechanism. In return, grouping becomes a separately trained component that can be placed after different keypoint detectors through a common keypoint interface. Across the three detectors evaluated in this chapter, the observed gap to native grouping remains between 0.0203 and 0.0220 PGA without detector-specific retraining.

PG-GAT should therefore not be interpreted as a parameter-efficient replacement for associative embedding. Its contribution is the separation of detection and grouping, with the additional parameter count and the approximately 0.02 PGA gap representing the measured cost of that design choice in the experiments reported here. Whether this trade-off is useful depends on the deployment setting. It is most relevant where the grouping component must be developed independently of the detector, reused across detector architectures, or replaced without retraining the complete pose-estimation pipeline. This trade-off is discussed further in Section 5.1.3.

4.11 PG-GAT V2 — VISUAL FEATURE AUGMENTATION

This section reports the results for the visual-feature-augmented PG-GAT variant introduced in Section 3.9. The experiment tests whether pre-trained ImageNet features sampled at detected keypoint locations provide person-identity information that is not available from the positional and joint-type inputs of Section 3.3.2.

The result is negative as a practical extension of PG-GAT. Adding the visual features does not improve on the headline PG-GAT, reaching 0.9670 and 0.9682 COCO val PGA against 0.9715, while requiring a frozen MobileNetV2 backbone with approximately 2.2×10^6 additional parameters. The visual augmentation therefore increases the size and complexity of the grouping pipeline substantially without producing a corresponding improvement in grouping accuracy. The results below quantify this trade-off and motivate retaining the position-based PG-GAT as the headline configuration.

4.11.1 Setup

Two training schedules are evaluated. The first trains PG-GAT v2 for 20 epochs with an initial learning rate of 10^{-3}, cosine-annealed to 10^{-5}. The second extends this run to 50 epochs by resuming from the 20-epoch checkpoint and training for a further 30 epochs with the initial learning rate reduced to 5×10^{-4}. The extended schedule tests whether the visual features require additional optimisation before their contribution becomes apparent.

Both configurations use the cached COCO MobileNetV2 features described in Section 3.9.2, comprising 54,564 training samples and 2,258 validation samples. Training uses only the contrastive objective of Equation (3.27) - no additional loss is introduced for the visual features. The MobileNetV2 backbone remains frozen throughout, while the projection block of Equation (3.49) is optimised jointly with the PG-GAT layers. Validation is performed every two epochs on the cached COCO validation set, using the same grouping metric as the other COCO experiments.

4.11.2 Headline numbers

Configuration	COCO val PGA (k-means, GT kp)	Grouping params.	Visual backbone	W&B run
PG-GAT v1 (`meng-headline`)	**0.9715**	333,552	—	`b5noyg7t`
PG-GAT v2, 20 epochs	0.9670	197,280	2.2M	`vg43ee4u`
PG-GAT v2, 50 epochs	0.9682	197,280	2.2M	`o8978au7`

Table 4.13. COCO ground-truth-keypoint results for the visual-feature PG-GAT v2 experiments. PG-GAT v2 is based on the legacy two-layer architecture rather than the sweep-selected architecture used by the `meng-headline` checkpoint, so the experiment is not a controlled visual-feature ablation of the headline model. The 20-epoch configuration reaches 0.9670 PGA and extending training to 50 epochs increases this only to 0.9682. The latter remains 0.0033 below the no-visual-feature headline result of 0.9715. Each v2 configuration additionally requires the frozen MobileNetV2 backbone used to extract the cached visual features, contributing approximately 2.2×10^6 parameters beyond the PG-GAT v2 network itself.

4.11.3 Interpretation

Three observations follow from Table 4.13.

1. The visual-feature variant does not beat the headline PG-GAT: The 20-epoch PG-GAT v2 configuration reaches 0.9670 COCO val PGA, which is 0.0045 below the no-visual-feature headline result of 0.9715. Extending training to 50 epochs raises the result to 0.9682, reducing the difference to 0.0033 but not closing it. Neither visual-feature configuration therefore improves on the final position-and-type-only PG-GAT.

This comparison requires one qualification. PG-GAT v2 was implemented on the legacy two-layer architecture, whereas the `meng-headline` model uses the larger architecture selected by the Bayesian sweep of Section 4.5. The experiment is consequently not a controlled ablation in which visual features are added to an otherwise identical headline model. It instead answers the practical question posed by the implemented variant - whether adding MobileNetV2 features to the existing PG-GAT v2 architecture produces a model that surpasses the final position-only system. It does not. A strict test of the marginal contribution of visual features to the sweep-selected architecture would require retraining that architecture with and without the visual projection under an otherwise identical schedule.

2. Additional training produces little further improvement: The 50-epoch schedule resumes from the 20-epoch checkpoint with the learning rate reduced from 10^{-3} to 5×10^{-4} and continues for a further 30 epochs. Across those additional epochs, COCO val PGA increases from 0.9670 to 0.9682, a gain of only 0.0012. The extended run therefore provides little evidence that substantially more optimisation of the same configuration would reverse the result.

This does not prove that the architecture has reached its global optimum, but it does rule out the simplest under-training explanation for the negative result. The initial 20-epoch schedule is not hiding a large late-training improvement - increasing the total training budget by 150% produces only a small change and leaves the model below the no-visual-feature headline.

3. The visual features carry a substantial deployment cost: The PG-GAT v2 network contains 197,280 parameters, approximately 27,000 more than the legacy two-layer PG-GAT on which it is based. More significantly, producing its visual input requires the frozen MobileNetV2 backbone, which contributes approximately 2.2×10^6 additional parameters. These parameters are not optimised during PG-GAT v2 training, but the backbone must still be available at inference time to extract the per-keypoint visual features.

When HigherHRNet-W32 is used as the upstream detector, the complete deployable v2 pipeline therefore contains approximately 28.6 million detector parameters, 2.2 million MobileNetV2 parameters, and 0.2 million PG-GAT v2 parameters, or approximately 31.0 million parameters in total. By comparison, HigherHRNet's native detector and AE grouping are contained within the approximately 28.6 million parameters of the detector itself. Visual augmentation therefore removes the model-size advantage that might otherwise be inferred from considering the PG-GAT grouping network in isolation, while still failing to improve on the position-only PG-GAT result.

Negative result: The visual-feature experiment does not provide evidence that the implemented MobileNetV2 augmentation improves the final PG-GAT system. The strongest v2 configuration reaches 0.9682 COCO val PGA with ground-truth keypoints, compared with 0.9715 for the `meng-headline` position-and-type-only model under the same ground-truth-keypoint evaluation. The additional 30 training epochs recover only 0.0012, while inference requires an additional 2.2×10^6-parameter feature extractor.

The result should therefore be read as a negative result for this particular visual-augmentation design rather than as evidence that appearance information cannot assist pose grouping in general. In particular, the experiment does not test visual features within the sweep-selected PG-GAT architecture or end-to-end adaptation of the visual backbone to the grouping objective. Those alternatives would require additional experiments and substantially greater training cost. Given that the implemented variant increases pipeline complexity without exceeding the position-only headline, visual augmentation is not pursued further in this work. The measurements reported here provide a baseline for future approaches that revisit appearance features using a stronger backbone, the sweep-selected PG-GAT architecture, or joint visual-and-grouping training.

CHAPTER 5 DISCUSSION

This chapter interprets the results of Chapter 4 by considering the experimental findings together rather than treating each method in isolation. Several patterns recur across otherwise different experiments, and these patterns provide a clearer account of where the pose-grouping problem is constrained and which changes produce improvements that survive transfer to real data.

Three findings structure the discussion. First, the experiments with learned grouping heads show that grouping performance is determined primarily by the quality of the embedding rather than by the complexity of the downstream read-out. Second, the synthetic-to-real distribution shift has a larger effect on the final COCO result than several architectural changes that appear promising under synthetic-only training. Third, the end-to-end and detector-swap experiments quantify the trade-off introduced by separating grouping from detection. This includes the initially unexpected result that the learned estimate of the person count can outperform the annotation-level count when grouping incomplete detector outputs.

The chapter develops these findings using evidence from the successful and negative experiments reported in Chapter 4. The final section returns to the seven research questions introduced in Section 1.5 and summarises the answer to each question together with the experimental evidence that supports it.

5.1 THE EMBEDDING IS THE LEVER

The most consistent finding across the experiments is that pose-grouping performance depends more strongly on the quality of the per-keypoint embedding than on the algorithm used to partition those embeddings into person identities. This pattern appears across three separate groups of experiments. Replacing k-means with learned grouping heads does not produce a consistent improvement, and the more specialised SA-DMoN objective also fails to improve the underlying representation. In contrast, changes made directly to the embedding network produce gains that are retained, and subsequently strengthened, after transfer to COCO.

This section considers these results together rather than as independent negative and positive experiments. The DEC, slot-attention, and graph-partitioning results show what happens when additional modelling capacity is placed after or alongside an embedding that still contains ambiguous person assignments. The SA-DMoN experiments provide a stronger test of the same idea, by introducing structural information through the clustering objective, but again fail to move beyond the performance of the simpler baseline. PG-GAT changes where that structural information enters the pipeline - instead of asking the grouping head to recover person structure from the final embeddings, it uses joint type, relative geometry, and same-type relationships during message passing while those embeddings are being formed.

Taken together, these experiments suggest that the important distinction is not simply between simple and learned grouping methods but between introducing structure *after* the representation has been formed and using that structure to shape the representation itself. The remainder of this section examines the evidence for this interpretation and relates each PG-GAT modification to the failure modes observed in the earlier grouping-head experiments.

5.1.1 Why every learned head failed on the same embedding

Three learned grouping heads, DEC, slot attention, and graph partitioning, were attached to the standard GAT backbone of Section 3.5 and trained jointly with the embedding under matched optimiser and schedule settings (Section 4.2). None produces a grouping result that improves on the no-depth k-means baseline once evaluated on COCO. The contrast is clearest for graph partitioning - it is the strongest learned head on synthetic validation at 0.9714, but its own COCO read-out falls to 0.7514. Applying k-means to the embeddings produced by the same jointly trained model recovers the score to 0.8366, which is still below the standard GAT baseline of 0.8841. The three heads fail in different ways, and this consistency across different grouping mechanisms is what makes the combined result more informative than any individual negative experiment.

DEC adds little because its clustering objective has limited ability to correct errors already present in the embedding. Its KL-divergence objective compares the soft assignment q with a sharpened target p that is itself constructed from q (Equation (3.12)). When k-means++ initialisation already produces confident assignments, the target distribution mainly reinforces those assignments rather than introducing independent supervisory information about person identity. This behaviour is particularly limiting for the ambiguous cases that matter most for pose grouping. If two same-type keypoints from different people lie close together in embedding space, an incorrect initial assignment can be sharpened along with the correct ones. DEC therefore has no independent structural cue with which to resolve the error. This is consistent with its COCO result of 0.8798, which remains close to but below the 0.8841 baseline.

Slot attention behaves differently. Its synthetic-validation PGA of 0.9025 exceeds the standard GAT baseline of 0.8783, showing that a learned competitive assignment mechanism can exploit the relatively clean synthetic representation. The improvement does not survive transfer - its own COCO read-out reaches only 0.7986, while k-means on the jointly trained embeddings reaches 0.8254. One limitation is that the slot competition does not explicitly encode the type-exclusivity constraint available from the pose representation. Two detections of the same joint type must belong to different people, but the slot mechanism is not constructed to enforce this relationship directly. On the synthetic distribution the embedding can often resolve the ambiguity without that prior - the COCO result suggests that this becomes less reliable once the representation is exposed to the synthetic-to-real shift. The experiment does not isolate type-exclusivity as the sole cause of the transfer failure, but its absence provides a structural explanation consistent with the observed behaviour.

Graph partitioning provides the strongest example of a synthetic gain that does not transfer. Its 0.9714 synthetic-validation PGA is well above the 0.8783 baseline, yet its own COCO grouping falls to 0.7514. The k-means read-out on the same jointly trained embeddings recovers to 0.8366, indicating that a substantial part of the failure lies in the partitioning read-out rather than only in the embedding. The edge classifier converts pairwise affinities into binary decisions, which are subsequently thresholded and assembled into connected components. This creates a sensitivity that is absent from centroid-based k-means: changes in the affinity distribution around the decision threshold can alter individual edges, and a small number of incorrect edges can change the connectivity of an entire component. The large synthetic-to-COCO deterioration is therefore consistent with a partitioning mechanism that is particularly sensitive to distribution shift.

Taken together, the three experiments provide evidence against the grouping head being the main limitation of the baseline pipeline. DEC adds an unsupervised sharpening objective, slot attention introduces a learned competitive assignment mechanism, and graph partitioning reformulates grouping as supervised pairwise classification. Despite these substantially different approaches, none produces a COCO result above the 0.8841 k-means baseline. In several cases, applying k-means to the jointly trained embeddings performs better than the head for which those embeddings were trained.

The common result is therefore more important than the individual failure mechanisms. Increasing the complexity of the grouping stage does not reliably recover information that is absent or ambiguous in the representation supplied to it. This observation motivates moving the structural priors earlier in the pipeline, so that they influence the construction of the embedding itself rather than being introduced only when that embedding is partitioned. The SA-DMoN and PG-GAT experiments provide the next two tests of this interpretation.

5.1.2 Why ten SA-DMoN variants plateaued

The SA-DMoN experiments extend the grouping-head investigation in a different direction. Rather than learning a direct assignment from the final embeddings, SA-DMoN introduces a graph-structural objective based on modularity maximisation over the keypoint kNN graph. The ten configurations reported in Section 4.3 all converge to a narrow synthetic-validation band of approximately 0.77–0.79, around 0.09 below the no-depth baseline. More informative than the best individual result is the fact that substantial changes to the null model, adjacency, and type loss do not move the model outside this band. This suggests that the limitation lies in the compatibility between the modularity objective and the graph on which it operates rather than in a particular SA-DMoN hyperparameter.

Newman modularity (Equation (3.16)) measures whether a proposed partition contains more within-community edge mass than would be expected under a chosen null model. This formulation is well suited to graphs in which observed connectivity carries information about the communities of interest, and the null model provides a reasonable description of connectivity in the absence of those communities. The keypoint kNN graph is misaligned with this assumption. Its edges are constructed explicitly from spatial proximity, whereas the desired partition is person identity. Two keypoints can therefore be neighbours because they occupy nearby image coordinates even when they belong to different people. This occurs precisely in the crowded and overlapping poses for which grouping is most difficult.

The consequence is a mismatch between the structure emphasised by the graph and the structure required by the task. A modularity objective operating on this graph is encouraged to preserve patterns of local connectivity, but those patterns need not correspond to person identity. In crowded scenes they may instead connect joints belonging to different people. The spectral objective is therefore being asked to recover identity communities from a graph whose edges were not constructed primarily from identity information. This provides a structural explanation for why changing the details of the modularity formulation has little effect on the final grouping accuracy.

The SA-DMoN variants test this explanation from several directions. Changing the spatial bandwidth σ modifies how strongly proximity enters the skeleton-aware null model, but no tested parameterisation produces a useful operating point. With unconstrained optimisation, σ grows to approximately 341, making the spatial kernel nearly uniform and effectively weakening the intended spatial correction. Constraining σ to $[0.05, 0.5]$ prevents this divergence, but the learned value moves to the upper boundary, again indicating that the optimisation prefers a weaker spatial null. Fixing σ from the per-graph median pairwise distance moves in the opposite direction and also reduces performance. The behaviour across these variants is more consistent with a poorly matched objective than with a single badly chosen bandwidth.

The v2 experiments alter other parts of the formulation but reach the same conclusion. Feature-based adjacency breaks the direct dependence between the message-passing graph and the spatial null but makes the adjacency itself change as the representation is learned. The modularity objective must then optimise against a graph that is moving with the embedding. Reducing the spectral weight partially recovers the loss in performance but does not exceed the earlier variants. Replacing the hinge-style type term with an entropy formulation changes the smoothness of the optimisation without changing the information available to the objective, and similarly produces no meaningful gain. Combining the two changes does not improve the result.

The narrow performance range across all ten configurations is therefore the main evidence from the SA-DMoN investigation. It does not establish that modularity objectives are unsuitable for pose grouping in general - a graph whose edges encode learned identity affinity rather than spatial proximity could behave differently. It does show that, for the keypoint kNN graph used here, increasingly elaborate modifications to the modularity objective do not recover the performance of the simpler contrastive-embedding baseline.

Together with the learned-head experiments, this result substantially narrows the useful design space. The earlier experiments show that DEC, slot attention, and pairwise graph partitioning do not reliably improve grouping once the representation transfers to COCO. SA-DMoN then shows that introducing pose structure through a modularity-based clustering objective does not resolve the problem either. These results motivate moving the structural information into the representation-learning stage itself. PG-GAT follows this direction by retaining the simple k-means read-out and instead changing how information is exchanged between keypoints while their embeddings are being formed.

5.1.3 Why the three PG-GAT modifications are complementary

The PG-GAT isolation ablation of Section 4.4 tests each of the three skeleton-aware modifications independently before combining them in the full model. The three individual configurations are tightly grouped on synthetic validation at 0.8825, 0.8825, and 0.8834, compared with 0.8783 for the standard GAT baseline. Enabling all three modifications increases PGA to 0.8896, which is 0.0062 above the strongest individual configuration and 0.0113 above the baseline. The individual differences are small enough that they should not be interpreted as a ranking of the three mechanisms. The more useful result is that no single modification accounts for the performance of the full model. Their combination produces a larger improvement than any modification in isolation, consistent with the three mechanisms providing complementary rather than redundant information.

This interpretation follows from the different roles of the three modifications. The type-pair attention bias introduces learnable additive terms for the four edge categories used by PG-GAT: same-type, skeletal-neighbour, same-limb, and cross-body. In the standard GATv2 formulation, attention is derived from the features of the incident nodes and does not explicitly distinguish these structural relationships. The type-pair term makes the edge category available directly to the attention calculation, allowing otherwise similar node pairs to receive different attention biases according to their relationship in the pose graph.

The skeleton-aware position encoding provides a complementary source of information. It exposes L2 distance, skeleton-hop distance, and the same-type indicator directly to the attention mechanism. Although absolute position is already present in the node representation, a standard GATv2 layer must infer relative geometric relationships through the interaction between the two node features. Providing these relationships explicitly reduces the burden on the attention layer and, in the case of skeleton-hop distance, introduces information that cannot be recovered from image coordinates alone.

The repulsion mechanism addresses a narrower problem. One of the four attention heads is restricted to same-type edges, concentrating part of the model capacity on comparisons such as left-wrist against left-wrist or right-knee against right-knee. These are particularly informative for person separation because two detections of the same joint type cannot belong to the same person. The dedicated head therefore gives the network a separate pathway in which the contrastive objective can act on the subset of relationships most directly associated with cross-person discrimination.

The isolation results are consistent with these three mechanisms being complementary. Type-pair attention contributes categorical edge structure, position encoding contributes relative geometric and topological information, and the repulsion head reserves capacity for same-type discrimination. Their individual gains are modest, but the combined configuration performs better than any of them alone. Given that these are single-run measurements and the individual differences are close to the observed run-to-run variation, the ablation does not establish the precise interaction between their gradients. It does, however, support retaining all three modifications in the architecture that is subsequently taken into the Bayesian sweep.

The architecture sweep then produces a much larger improvement than any of the isolation changes. Starting from the full two-layer PG-GAT configuration at 0.8896, the sweep over network depth, attention-head count, hidden dimension, output dimension, joint-type embedding dimension, and dropout selects a three-layer configuration with a 256-dimensional output and reaches 0.9634 synthetic-validation PGA. This is an increase of 0.0738 over the matched-architecture ablation configuration. By comparison, the subsequent loss sweep does not find a configuration that meaningfully exceeds the inherited contrastive settings - its best result of 0.9617 is slightly below the architecture-sweep winner.

The relative scale of these effects gives a more complete interpretation of the PG-GAT result. The skeleton-aware modifications provide the inductive biases that distinguish PG-GAT from the standard GATv2 baseline, but their benefit depends strongly on the architecture in which they operate. The two-layer isolation experiment establishes that the modifications provide a useful signal at matched compute, while the architecture sweep shows that considerably more of that signal can be exploited with an additional layer and a larger embedding space. In contrast, varying the contrastive margin, number of repulsion heads, learning rate, and weight decay does not produce a comparable gain.

The main lever is therefore not a more elaborate grouping objective or a finely tuned loss but the combination of task-specific structure and sufficient representation capacity. PG-GAT introduces information that the standard GATv2 formulation does not expose explicitly, and the architecture sweep identifies a network size that makes better use of that information. The loss sweep, in turn, provides evidence that the improvement is not primarily the result of a favourable choice of contrastive hyperparameters. This distinction is important when interpreting the headline model - its performance is not attributable to a single architectural modification or tuned loss value but to the combination of skeleton-aware message passing and an architecture sized to exploit it.

5.1.4 Synthesis

Reading Sections 5.1.1–5.1.3 together gives a consistent picture of where the main performance constraint lies. The learned grouping heads fail for different reasons, yet none improves on the simple k-means baseline after transfer to COCO. The SA-DMoN experiments reach the same conclusion from a graph-structural direction - changing the null model, adjacency, spectral weight, and type-loss formulation does not overcome the mismatch between modularity and the spatial kNN graph. In contrast, modifying the embedding network itself produces a measurable improvement at matched architecture and a much larger gain once the architecture is sized by the Bayesian sweep.

The evidence therefore identifies the embedding network as the stronger lever for pose-grouping performance in the setting studied here. This does not imply that the grouping algorithm is irrelevant or that a different grouping formulation could never improve the result. Rather, within the range of grouping methods evaluated in this work, additional complexity at the read-out stage contributes less than improving the representation that the read-out receives. The effectiveness of k-means in the final pipeline follows from this result - once the embedding separates person identities sufficiently well - a simple partitioning method is adequate and avoids introducing another learned component whose behaviour must transfer across domains.

This provides the experimental answer to *RQ1: the lever*. For the pose-grouping formulation and data considered in this work, the main opportunity for improvement lies in the construction of the per-keypoint embedding rather than in replacing its downstream grouping algorithm. This finding also motivates the later experiments in the chapter, which retain k-means and investigate how the representation behaves under domain transfer, alternative geometries, and detector-independent evaluation.

The PG-GAT ablation and architecture sweep provide the corresponding answer to *RQ2: inductive biases*. Type-pair attention, skeleton-aware position encoding, and same-type repulsion each produce a small positive change when introduced independently at the matched baseline architecture. Their combination gives the strongest isolation result, indicating that the three sources of pose structure are complementary at the resolution of the experiment. The architecture sweep then shows that the benefit of these inductive biases depends strongly on the capacity of the network that uses them, selecting a three-layer, 256-dimensional configuration that substantially outperforms the two-layer ablation model.

Taken together, RQ1 and RQ2 establish the design basis for the remainder of the discussion. The first identifies *where* additional modelling effort is most productive - the embedding rather than the grouping head. The second identifies *how* that embedding can be improved - by exposing pose-specific categorical, geometric, and same-type relationships directly to the attention mechanism and providing sufficient representation capacity to use them. The subsequent COCO fine-tuning and end-to-end experiments test whether these gains remain useful once the model leaves the synthetic distribution on which these design decisions were developed.

5.2 WHAT SURVIVES SYNTHETIC-TO-REAL, AND WHAT DOES NOT

The second cross-cutting finding is that the synthetic-to-real distribution shift is one of the strongest determinants of performance in this work. Several configurations that perform well, or appear promising, on the synthetic distribution lose part or all of that advantage when evaluated on COCO. Conversely, the largest improvement in the experimental chapter comes not from introducing another architectural mechanism, but from adapting the selected PG-GAT model to the COCO distribution through fine-tuning.

This pattern appears across several experiments that otherwise have little in common. Removing the depth channel reduces synthetic performance but improves zero-shot COCO transfer. Fine-tuning PG-GAT on COCO produces a substantially larger improvement than any individual PG-GAT modification. The reduced-dimensional hyperbolic representation is competitive in the synthetic-only setting but does not retain that advantage after COCO fine-tuning. TriGAT introduces articulation information that is meaningful in principle, yet performs worse after transfer, while the visual-feature variant adds appearance information without improving on the final position-and-type-only PG-GAT. These experiments alter different parts of the pipeline, but each exposes the same underlying difficulty - a representation that is useful on the synthetic distribution is not necessarily the representation that transfers best to real keypoint data.

The evidence therefore suggests that synthetic validation should be treated primarily as a controlled environment for developing and isolating architectural ideas, rather than as a reliable predictor of their final COCO performance. This distinction is important because the synthetic data remains useful throughout the investigation. It provides known person identities, controlled scene composition, and a repeatable setting in which individual design changes can be compared without the additional variation introduced by a real detector. The limitation is not that synthetic evaluation is uninformative, but that improvements measured there cannot automatically be interpreted as improvements in real-data grouping.

The remainder of this section examines this transfer effect through the depth-removal experiment, the COCO fine-tuning results, and the hyperbolic, TriGAT, and visual-feature variants. Considered together, these experiments show both sides of the same problem - features that make the synthetic task easier can become liabilities after transfer, while relatively modest adaptation to the target distribution can produce larger gains than substantial changes to the model itself. This distinction between *synthetic discriminability* and *transferable discriminability* is central to interpreting the experimental results that follow.

5.2.1 Depth removal: the diagnostic that exposed the shift

The depth-removal diagnostic of Section 4.1 provides the clearest early evidence of the synthetic-to-real distribution shift. With depth included, the standard GAT reaches a synthetic-validation PGA of 0.9990 but transfers to COCO at only 0.8366. Removing the depth channel reduces synthetic-validation PGA to 0.8783, a drop of 0.1207, while increasing COCO PGA to 0.8841, a gain of 0.0475. The direction of the two changes is more informative than either number alone - the feature that makes the synthetic grouping problem almost trivial is also the feature that produces the weaker real-data representation.

The explanation lies in how depth is obtained in the two domains. The synthetic generator of Section 3.4.1 provides metric depth from the Blender scene, whereas the COCO pipeline of Section 3.4.2 estimates depth using MiDaS [82]. The latter provides relative depth rather than a metric camera-space coordinate, with scale and calibration that can vary between images. A model trained on the synthetic depth channel can therefore exploit relationships that are highly stable in the rendered data, but are not represented in the same way at COCO inference time. Removing depth sacrifices this particularly strong synthetic cue, but also removes a source of distribution mismatch.

This result also explains why the depth-on synthetic configuration is a poor setting in which to evaluate subsequent embedding modifications. At 0.9990 PGA there is almost no remaining grouping error for a new architectural mechanism to correct. The model can rely heavily on the metric depth coordinate to separate people in three-dimensional scenes, leaving little room for changes to the attention mechanism to produce a measurable improvement. Removing depth makes the synthetic problem harder, but in doing so produces a more useful experimental setting for studying whether the embedding network can learn person separation from signals that are also available consistently on real data.

The depth-removal experiment therefore serves two purposes in the methodology. First, it motivates the no-depth configuration used by the subsequent PG-GAT experiments. Second, it establishes an important distinction for interpreting the remainder of the results, making the synthetic task easier is not equivalent to learning a representation that transfers better. A synthetic improvement is useful evidence only when the mechanism responsible for that improvement remains meaningful under the corresponding real-data distribution. The experiments that follow repeatedly return to this distinction.

5.2.2 COCO fine-tune: the largest lift in the pipeline

The COCO fine-tuning experiment of Section 4.6 provides the clearest measurement of how strongly the target-data distribution affects the final grouping result. The synthetic-pretrained architecture-sweep winner reaches 0.9010 COCO val PGA before fine-tuning and 0.9715 after the fine-tune sweep, an improvement of 0.0705 on the same PG-GAT architecture. This gain is substantially larger than the approximately 0.011 improvement produced by combining the three PG-GAT modifications at the matched baseline architecture (Section 5.1.3), and is comparable in magnitude to the 0.0738 improvement obtained from the architecture sweep itself. Adaptation to the target distribution is therefore one of the largest individual effects measured in the experimental chapter.

Two aspects of this result help explain its significance. First, the COCO improvement does not come at the expense of the synthetic representation. The fine-tuned checkpoint retains a synthetic-validation PGA of 0.9634, equal to the value measured for the architecture-sweep checkpoint before fine-tuning.

The corresponding synthetic-test result of 0.9580 provides the same general picture. Within the resolution of these measurements, the fine-tune therefore adapts the representation to COCO without removing the person-separation structure learned during synthetic pre-training. The two stages appear complementary rather than competitive, synthetic training establishes a useful grouping representation, and COCO fine-tuning adjusts that representation to the statistics of the target domain.

Second, relatively little additional performance is obtained by optimising the fine-tuning recipe itself. A 20-epoch fine-tune of the architecture-sweep winner reaches 0.9695 COCO val PGA, while the 12-run Bayesian sweep improves this to 0.9715. The difference of 0.0020 is small relative to the variation of the metric and should not be interpreted as a substantive gain. Instead, the sweep shows that the inherited recipe lies within a relatively flat region of the fine-tuning search space. Its main contribution is therefore to support the choice of learning rate, weight decay, and training duration rather than to provide another major source of performance.

The contrast between these two effects is informative. Changing the model's exposure from synthetic-only training to COCO fine-tuning produces a 0.0705 gain, whereas searching over reasonable ways of performing that fine-tuning produces only a further 0.0020. The important variable is therefore the target-domain data itself, rather than a narrowly tuned optimisation schedule. This reinforces the interpretation of the depth experiment - representations learned from synthetic data can provide a strong starting point, but their final real-data performance depends heavily on whether the information they encode remains appropriate for the target distribution.

This result provides the reference point for the architectural variants considered next. Hyperbolic embedding, the triplet representation, and visual-feature augmentation each introduce additional representational structure that could, in principle, improve person separation. Their value cannot be judged from synthetic performance alone, however. The more demanding question is whether the additional structure produces an advantage once the model is evaluated or adapted on the real-data distribution. The results of those experiments show that this condition is considerably harder to satisfy.

5.2.3 Hyperbolic Option B: the geometric prior loses to fine-tuning

The hyperbolic PG-GAT experiments of Section 4.7 provide a useful example of an advantage that appears under synthetic-only training but does not survive adaptation to the target distribution. Under

H2, a 32-dimensional hyperbolic embedding reaches 0.8856 synthetic-validation PGA, compared with 0.8896 for the 128-dimensional Euclidean ablation model. The difference is small relative to the variation of the metric, so the result supports the intended efficiency interpretation - in this setting, the hyperbolic representation retains approximately the same grouping quality with a four-fold reduction in embedding dimension.

The important test is whether that dimensional advantage persists once the models are exposed to COCO. Option B fine-tunes the 32-dimensional hyperbolic model and reaches 0.9399 COCO val PGA. This is substantially higher than its synthetic-only COCO transfer result of 0.8969, showing that the hyperbolic representation also benefits from target-domain adaptation. The improvement is not enough, however, to make it competitive with the strongest Euclidean configurations. It remains below the 0.9565 legacy Euclidean fine-tune result and 0.0316 below the sweep-selected Euclidean headline of 0.9715. The dimensional efficiency observed in the synthetic-only experiment therefore does not translate into a competitive fine-tuned COCO result.

One possible explanation is the scale of the hierarchy represented by the pose graph. The theoretical motivation for hyperbolic embeddings is strongest for hierarchical and tree-like structures whose number of nodes grows rapidly with depth, where Euclidean representations require increasing dimension to represent the structure with low distortion [68]. The COCO skeleton is small by comparison - it contains only 17 joint types and has a maximum topological depth of only a few hops. The pose-grouping problem may therefore provide too little hierarchical structure for the geometric advantage of hyperbolic space to become large at the embedding dimensions considered here. This interpretation is consistent with the dim-32 result, where hyperbolic geometry appears to preserve useful structure efficiently, but does not create a higher-quality representation than the larger Euclidean model.

The fine-tuning results expose a second and empirically stronger effect. The Euclidean PG-GAT gains 0.0705 COCO PGA when the architecture-sweep checkpoint is adapted from synthetic-only training to the final COCO-fine-tuned configuration. The hyperbolic Option B model also improves after fine-tuning, but its gain is smaller - 0.8969 to 0.9399, or 0.0430. Whatever representational advantage the hyperbolic geometry provides at low dimension is therefore not sufficient to compensate for the stronger target-domain performance reached by the Euclidean model. The experiment does not isolate whether this difference is caused by embedding dimension, manifold geometry, optimisation behaviour, or an interaction between them, but it does establish that the synthetic efficiency result alone is not predictive of the final COCO ranking.

The two pre-registered hypotheses should consequently be interpreted separately. H1, which predicts a quality advantage for hyperbolic embedding at comparable dimension, is not supported by the experiments. H2 receives partial support - the 32-dimensional hyperbolic model approximately matches the 128-dimensional Euclidean ablation model under synthetic-only training, demonstrating that useful grouping structure can be represented at substantially lower dimension. That advantage does not survive strongly enough after COCO fine-tuning to produce a competitive final model. Hyperbolic PG-GAT is therefore best viewed as an efficiency observation under the synthetic regime, rather than as an alternative to the Euclidean headline configuration.

5.2.4 TriGAT: a richer primitive whose downstream voting step is lossy

TriGAT (Section 4.8) explores a different alternative to the standard PG-GAT formulation. Rather than changing the embedding geometry or adding another feature source, it changes the graph primitive from individual joints to triplets of three connected joints. The motivation is that a triplet provides local articulation information through its pivot angle, which is invariant to global rotation and is not directly available from the position of a single joint. Under the matched synthetic training regime, however, TriGAT reaches 0.8740 synthetic-validation PGA compared with 0.8896 for the corresponding PG-GAT ablation configuration, a difference of -0.0156.

The main weakness of the implemented formulation appears in the conversion from triplet-level clusters back to joint-level assignments. A joint can belong to several overlapping triplets, and those triplets are not guaranteed to receive the same cluster label after k-means. The weighted-majority rule of Section 3.11.3 must therefore reconcile potentially inconsistent triplet assignments before the final joint partition can be formed. This introduces an additional inference step that is absent from the joint-based PG-GAT pipeline.

Errors at the triplet level can also propagate through this conversion. A mis-clustered triplet contains three joints and overlaps with neighbouring triplets, so its incorrect assignment can influence the final labels of several joints. The voting rule can correct an isolated error when the remaining triplets agree, but it cannot recover reliably when several overlapping assignments disagree. The additional articulation information must therefore provide enough improvement in the triplet embedding to compensate for the error introduced by the subsequent voting stage. The measured result indicates that this does not happen in the present architecture.

This result does not show that articulation is an unhelpful cue. The triplet representation explicitly contains pivot-angle information that is absent from the single-joint input, and the pivot angle is a rotation-invariant descriptor of local articulation. Instead, the experiment shows that the implemented route for exploiting this information loses more during the triplet-to-joint conversion than it gains from the richer representation. The negative result therefore applies to the combination of triplet-level embedding, triplet-level clustering, and joint-level voting rather than to the articulation signal itself.

A natural follow-on would be to retain the articulation information while avoiding the intermediate triplet clustering. For example, overlapping triplet embeddings could be aggregated into a joint-level representation before clustering, or a learned mapping could predict joint assignments directly from the surrounding triplet representations. Either approach would preserve the joint as the final grouping primitive while still allowing local articulation to contribute to its embedding. These alternatives require a different architecture and are therefore left for future work.

The TriGAT experiment consequently answers the graph-primitive question more narrowly than a simple comparison between joints and triplets might suggest. Changing to a richer primitive does not improve grouping merely because that primitive contains additional information - the architecture must also preserve that information through to the final joint-level decision. For the TriGAT formulation evaluated here, the evidence points to the triplet-to-joint voting stage as the main bottleneck, making the implemented design a documented negative result rather than evidence against articulation-based features more generally.

5.2.5 PG-GAT v2: visual features do not beat the sweep-defended baseline

The PG-GAT v2 experiment tests whether appearance information can add useful identity cues beyond the position-and-type representation used by the standard PG-GAT (Section 4.11). Pre-trained MobileNetV2 features are sampled at each keypoint location and projected into the node representation before message passing. The 20-epoch configuration reaches 0.9670 COCO val PGA, while extending training to 50 epochs raises this to 0.9682. Both remain below the `meng-headline` result of 0.9715, by 0.0045 and 0.0033 respectively. The additional visual information therefore does not produce a better operating point in the configuration evaluated here.

This result is consistent with the diminishing returns observed in the earlier feasibility experiments. When MobileNetV2 features were added to a simple position-based *k*-means representation, the improvement was approximately $+0.07$ COCO PGA. As the underlying representation became stronger, however, the additional gain from the same visual signal became much smaller. This suggests that appearance features are most useful when the existing representation contains relatively little identity information. Once position, joint type, skeleton structure, and learned message passing already provide a strong separation between people, much of the useful information supplied by the visual features may be redundant.

The fully trained v2 experiment follows this pattern. PG-GAT already uses joint type and relative skeleton structure to organise the embedding, and COCO fine-tuning adapts that representation directly to the target distribution. At this operating point, adding generic ImageNet features does not improve the final PGA. This does not imply that the visual features contain no identity information - the earlier feasibility result shows that they do. Rather, their marginal contribution decreases as the non-visual embedding becomes stronger. The additional signal is not sufficient to produce an improvement over the sweep-selected and COCO-fine-tuned PG-GAT configuration.

There is an important qualification to this comparison. PG-GAT v2 extends the legacy two-layer architecture rather than the architecture-sweep-selected three-layer headline model. The experiment therefore does not isolate the marginal effect of adding visual features to the exact `meng-headline` architecture. A fully matched ablation would require adding the same visual pathway to the sweep-selected PG-GAT and repeating the fine-tune procedure. The present result supports the narrower conclusion that the implemented v2 configuration does not provide a better practical operating point than the final position-and-type model - it does not establish that visual features could never improve a stronger or differently trained PG-GAT.

The computational cost makes the measured trade less attractive. The v2 network contains approximately 27,000 additional parameters relative to the legacy two-layer PG-GAT that it extends, and the frozen MobileNetV2 feature extractor contributes a further approximately 2.2×10^6 parameters. More importantly, the MobileNetV2 backbone must be executed at inference time because its feature maps are required to construct the visual node features. When combined with the HigherHRNet detector and PG-GAT v2 network, the resulting deployable pipeline contains approximately 31.0×10^6 parameters, compared with approximately 28.6×10^6 for HigherHRNet alone.

The added computation is not accompanied by an accuracy gain in the experiments reported here. The extended v2 configuration reaches 0.9682 COCO val PGA, below the 0.9715 no-visual-feature PG-GAT headline. The HigherHRNet AE figure of 0.9847 is also higher, although that value is an end-to-end PGA measured on matched detector outputs and should not be treated as a strictly like-for-like comparison with the ground-truth-keypoint v2 value. It is nevertheless relevant to the deployment trade - the visual pathway increases the total system size without closing the accuracy gap to the native detector-specific grouping mechanism.

The experiment therefore provides a negative result for the visual augmentation implemented in PG-GAT v2. Appearance features are useful when added to a weak positional representation, but their marginal value decreases as the underlying embedding improves. At the operating points tested here, the additional MobileNetV2 pathway increases model size and inference requirements without exceeding the simpler PG-GAT headline. For that reason, visual augmentation is not pursued further in this work. A future investigation could revisit the idea using a matched sweep-selected architecture, detector-native visual features, or jointly trained appearance features, but those experiments would test a different formulation from the frozen ImageNet feature pathway evaluated here.

5.2.6 Synthesis

Reading Sections 5.2.1–5.2.5 together shows that the synthetic-to-real shift affects each of the alternative representations in a different way. The depth-on baseline is the clearest example - near-perfect synthetic performance falls to 0.8366 on COCO because the depth signal changes between the two domains. The low-dimensional hyperbolic model retains useful structure efficiently under synthetic training, but its advantage is not sufficient to remain competitive after COCO fine-tuning. TriGAT introduces articulation information but loses more through its triplet-to-joint voting stage than it gains from the richer primitive. The visual-feature variant adds appearance information at considerable computational cost, yet the implemented configuration remains below the position-and-type-only headline model. Although the mechanisms differ, the common result is that an advantage observed or motivated in the controlled setting does not automatically translate into an advantage at the final COCO operating point.

The COCO fine-tuning experiment provides the strongest counterpoint. Adapting the architecture-sweep-selected PG-GAT from synthetic training to COCO increases COCO val PGA from 0.9010 to 0.9715, a gain of 0.0705, while preserving its 0.9634 synthetic-validation PGA. This improvement is comparable to the gain produced by the architecture sweep itself and larger than the changes associated with the individual architectural alternatives considered in this section. The result therefore identifies exposure to the target distribution as a major determinant of real-data grouping performance, rather than treating synthetic performance as a sufficient proxy for it.

This provides the experimental answer to *RQ6: synthetic-to-real transfer*. Synthetic data provides a useful environment for isolating architectural effects and learning an initial person-separation representation, but performance measured there is not by itself a reliable predictor of the final COCO ranking. Features that are highly discriminative in the synthetic domain can transfer poorly when their statistics change, as demonstrated most directly by depth. Conversely, a representation that is already strong on synthetic data can still gain substantially when adapted to the target distribution. The synthetic and real stages should therefore be treated as complementary, the former supports controlled architectural development, while the latter is necessary to establish whether the resulting representation remains useful under realistic data.

This finding also constrains the answers to *RQ4: embedding geometry* and *RQ5: graph primitive*. For RQ4, the hyperbolic experiments provide evidence that a lower-dimensional representation can retain comparable grouping information under synthetic-only training, but they do not establish a quality advantage over the Euclidean embedding at the final COCO operating point. The result is therefore best interpreted as an efficiency observation, rather than as evidence for replacing the Euclidean headline model. For RQ5, the TriGAT result shows that introducing articulation through a triplet primitive does not improve grouping in the architecture evaluated here. The negative result is associated with the additional triplet-clustering and voting stage, rather than demonstrating that articulation itself is uninformative.

The visual-feature experiment provides a related negative result. The implemented MobileNetV2 augmentation reaches 0.9682 COCO val PGA, compared with 0.9715 for the no-visual-feature headline model, while requiring an additional visual feature extractor at inference. Because the v2 experiment uses the legacy two-layer architecture rather than the sweep-selected headline architecture, it does not isolate the marginal effect of appearance features on the final PG-GAT configuration. It does, however, show that the implemented visual pathway does not provide a better practical operating point than the simpler position-and-type model.

The methodological consequence is not that architectural alternatives are unimportant, but that they must be judged after the relevant domain shift rather than by their synthetic behaviour alone. In the experiments reported here, the most reliable path to the final result is to retain the skeleton-aware PG-GAT representation selected by the architecture sweep and then adapt that representation to COCO. The architecture sweep determines a model with sufficient capacity to exploit the pose-specific inductive biases, while the fine-tuning stage adapts those features to the target distribution. The headline result is the product of these two stages, architecture selection establishes the representation, and target-domain fine-tuning makes that representation effective on real data.

5.3 THE COST OF MODULARITY, AND THE SURPRISE OF AUTONOMY

The third cross-cutting finding concerns the cost of separating grouping from detection and the behaviour of the resulting autonomous pipeline. Unlike HigherHRNet's associative-embedding (AE) head, PG-GAT is trained as a separate grouping module and operates only on detected keypoints. This separation provides detector independence, but removes the joint-training advantage available to a grouping representation that is learned together with the detector. On the HigherHRNet end-to-end evaluation, the consequence is measurable - the oracle-K PG-GAT pipeline reaches 0.9591 PGA compared with 0.9847 for HigherHRNet's native AE grouping on the same matched detections, a difference of 0.0256.

This gap is an expected consequence of the modular design. The AE tags are learned alongside the keypoint heatmaps and can adapt directly to the detector features and errors encountered during training. PG-GAT, by contrast, receives only the resulting keypoint coordinates, joint types, and associated node features. It has no access to the detector's intermediate representation and cannot influence which keypoints are produced. The comparison therefore measures a trade-off rather than a failure to reproduce the AE architecture - PG-GAT gives up some grouping accuracy in exchange for a grouping representation that can be applied independently of the detector that produced the keypoints.

The detector-swap experiments make this interpretation stronger than the HigherHRNet comparison alone. The same frozen PG-GAT checkpoint and K-head can be applied to HigherHRNet, YOLO11x-pose, and RTMO-L without retraining, and the autonomous grouping result remains approximately 0.02 below each detector's native assignment. The stability of this difference across three detector families suggests that the measured cost is associated with the modular grouping formulation rather than with a particular interaction between PG-GAT and HigherHRNet. The benefit being purchased is correspondingly concrete - the grouping network can be reused when the upstream detector changes.

The second part of the result is less intuitive. Replacing the ground-truth person count K with the prediction of the frozen-backbone K-head might be expected to introduce another source of error. Instead, the autonomous HigherHRNet pipeline reaches 0.9644 PGA, compared with 0.9591 when k-means is supplied with the ground-truth K. The difference is small, but its direction is consistent with the detector-swap experiments, which makes an explanation beyond random variation in a single evaluation worth considering.

The key distinction is between the number of people annotated in the image and the number of people represented by the detections that actually reach the grouping stage. These quantities are identical when ground-truth keypoints are used, but they need not remain identical after detection and matching. HigherHRNet recovers only a subset of the annotated keypoints under the evaluation protocol. A person may therefore be present in the ground truth while contributing too few, or no, matched detections to form a meaningful cluster. Supplying the annotation-level K to k-means nevertheless forces the algorithm to produce that many clusters from the incomplete detection set.

This creates what can be viewed as a phantom-cluster effect. When the specified K is larger than the number of people effectively represented in the detected embeddings, k-means must allocate additional centroids somewhere. These centroids can split the detections of a well-represented person rather than recover a person whose keypoints are absent. The ground-truth value is therefore an oracle for the original image, but not necessarily an oracle for the incomplete representation presented to the grouping algorithm.

The learned K-head operates on that incomplete representation instead. Its prediction is derived from pooled PG-GAT embeddings and the observed keypoint count, so it can implicitly adapt to the number of people that remain represented after detection. Its 76.1% exact-K accuracy therefore does not tell the complete story - a prediction that differs from the annotation-level count can still provide a better value for partitioning the detected subset. In this setting, being "wrong" against the annotated K can be useful if the prediction is closer to the effective number of recoverable people.

This interpretation is supported by the detector-swap results. As detector recall decreases, the advantage of predicted K over oracle K increases, +0.0006 for YOLO11x-pose at 0.816 recall, +0.0053 for HigherHRNet at 0.793, and +0.0304 for RTMO-L at 0.774. In the ground-truth-keypoint control, where the keypoint set is complete and the annotation-level K is genuinely the correct partition size, predicted K instead carries a penalty. The pattern is therefore consistent with the effective-person-count explanation, K-estimation becomes useful not because it predicts the annotation more accurately than the oracle, but because it adapts the clustering problem to the information that survives detection.

The two halves of the finding are consequently related. Detector independence introduces a modest grouping cost because PG-GAT gives up joint optimisation with the detector, but operating on detached detections also changes what the appropriate person count means. Once the detector produces an incomplete representation of the scene, the annotation-level K is no longer necessarily the best value for clustering that representation. The autonomous pipeline can therefore outperform its oracle-K diagnostic while remaining fully independent of ground truth at inference. The following sections examine the modularity cost and this effective-person-count mechanism in more detail.

5.3.1 The 0.0256 detector-independence gap

The end-to-end evaluation of Section 4.9 provides the clearest measurement of the cost introduced by separating grouping from detection. On the matched HigherHRNet detections, the detector's native associative-embedding (AE) grouping reaches 0.9847 PGA, while the headline PG-GAT configuration reaches 0.9591 with oracle K. The difference is 0.0256. This gap also narrows consistently as the PG-GAT pipeline is improved, the synthetic-only two-layer configuration trails AE by 0.1081, the legacy COCO fine-tune reduces the difference to 0.0417, and the sweep-selected architecture and fine-tune recipe reduce it further to 0.0256.

The detector-swap experiment of Section 4.9.5 shows that this behaviour is not specific to HigherHRNet. The same frozen PG-GAT checkpoint and K-head are applied zero-shot to detections from YOLO11x-pose and RTMO-L, despite neither detector contributing training data to the grouping module. The autonomous pipeline trails their native person assignments by 0.0210 and 0.0220 PGA respectively, compared with 0.0203 for the autonomous HigherHRNet configuration. The similarity of these gaps across three detector families is stronger evidence for the modularity claim than the HigherHRNet result alone - changing the upstream detector does not require the grouping network to be retrained, and the measured cost remains close to 0.02 PGA.

A likely reason for the remaining gap is the advantage available to a jointly trained grouping representation. HigherHRNet learns its AE tags together with the keypoint detector on the COCO training distribution. The tag representation can therefore adapt to the same intermediate features and image statistics used to produce the heatmaps. PG-GAT does not have access to this information. It operates after detection and must infer person identity from the resulting keypoint representation, which deliberately removes the dependency on detector-specific internal features. The two approaches consequently solve the grouping problem under different information constraints.

This distinction is also important when interpreting the parameter counts. HigherHRNet's AE tag head contains only approximately 561 parameters, whereas the sweep-selected PG-GAT contains 333,552. PG-GAT therefore does not obtain its modularity through a smaller grouping model. The advantage is architectural separation. The AE tag head forms part of a representation learned jointly with the HigherHRNet backbone and is not, in the form evaluated here, a standalone grouping module that can simply be transferred to the output of another detector. PG-GAT instead accepts detected keypoints through a detector-independent interface, which is what permits the same frozen checkpoint to operate on HigherHRNet, YOLO11x-pose, and RTMO-L.

The comparison should therefore be understood as a trade-off in capability rather than as a claim that PG-GAT is uniformly preferable to AE. On HigherHRNet detections, native AE grouping provides the higher PGA. If the detector is fixed and the complete detection-and-grouping pipeline can be trained jointly, there is little reason on the evidence of this work to give up that advantage. In such a setting, the native grouping mechanism is both more accurate and substantially smaller than PG-GAT.

The situation changes when the grouping component must remain independent of the detector. The detector-swap results demonstrate that PG-GAT can be placed behind three different keypoint detectors without retraining its embedding network, while remaining approximately 0.02 PGA below each detector's native assignment. This capability cannot be inferred from the HigherHRNet comparison alone - it is directly demonstrated by the zero-shot detector swaps. The additional PG-GAT parameters and the remaining accuracy difference are therefore the measured costs of maintaining a separately trainable and reusable grouping component.

Whether that trade is worthwhile depends on the deployment setting. For a fixed detector whose native grouping mechanism is available and can be retained, the results favour the native approach. For a system in which detectors may be exchanged, upgraded, or developed independently of the grouping stage, the modular PG-GAT interface provides a capability that the coupled alternatives evaluated here do not provide. The experimental contribution is therefore not that PG-GAT removes the cost of decoupling but that it quantifies that cost - approximately 0.020–0.022 PGA for the autonomous pipeline across the three detector families tested, in exchange for zero-shot reuse of the same grouping network.

5.3.2 Why predicted K exceeds oracle K

The autonomous evaluation of Section 4.9 produces a counter-intuitive result. Using the K-head prediction gives an end-to-end PGA of 0.9644, compared with 0.9591 when k-means is given the ground-truth person count. This occurs even though the K-head predicts the exact ground-truth K in only 76.1% of cases (91.9% within one person). If ground-truth K were always the correct number of clusters for the data presented to k-means - an imperfect estimate should reduce grouping accuracy. The fact that it does not suggests that the relevant cluster count changes once detection errors are introduced.

The distinction is between the number of people annotated in the image and the number represented in the detected keypoints. HigherHRNet recovers 79.3% of the ground-truth keypoints under the 10-pixel matching threshold of Section 3.12.3. This is a keypoint-level recall measurement rather than a person-level one, so it does not imply that 20.7% of people are completely absent. Missing detections are distributed unevenly across people. Some people remain well represented, others contribute only a small number of matched keypoints, and some may contribute none. The annotation-level person count can therefore be larger than the number of people meaningfully represented in the set that reaches the grouping stage.

This changes the interpretation of "oracle K". The ground-truth count is an oracle for the complete annotated image, but it is not necessarily an oracle for an incomplete detected-keypoint set. When k-means is required to produce the full annotation-level number of clusters from a set in which one or more people are poorly represented or absent, the additional cluster centres must still be placed somewhere in the embedding space. In practice, this can split the detections belonging to a well-represented person across multiple clusters. The nominally correct K can therefore produce a worse partition of the evidence that is actually available.

The K-head is exposed to a different source of information. It predicts the person count from the detected representation itself, using mean-pooled embeddings, max-pooled embeddings, and the number of detected keypoints. The detection count provides a direct indication of how much pose evidence is present, while the pooled embeddings describe how that evidence is distributed in the learned representation. The head can therefore learn a count that is better calibrated to the detected keypoint set than the annotation-level count. Consistent with this interpretation, its mean prediction on the HigherHRNet detections is 2.46, compared with a mean ground-truth count of 2.74.

The 0.0053 advantage on HigherHRNet alone would be too small to establish this mechanism. The detector-swap experiment provides a stronger test because it changes the detection recall while leaving the PG-GAT checkpoint and K-head fixed. For YOLO11x-pose, recall is 0.816 and predicted K improves PGA over oracle K by only 0.0006. At HigherHRNet's recall of 0.793, the improvement is 0.0053. With RTMO-L, recall falls to 0.774 and the improvement increases to 0.0304. The direction of the effect is therefore consistent across all three detector outputs - the larger the recall deficit, the greater the measured benefit of allowing the K-head to estimate the cluster count from the detections.

The ground-truth-keypoint control provides an important reference point. When ground-truth keypoints are supplied directly, detection recall is effectively perfect and the annotation-level person count is the correct number of clusters for the input. Under this condition, predicted K is 0.0066 PGA below oracle K. The sign of the effect therefore reverses when the mismatch between annotation-level K and the detected evidence is removed. This is the behaviour expected under the effective-person-count explanation, with complete keypoints, prediction error is harmful, with incomplete detections, a count inferred from the observed evidence can compensate for people who are no longer fully represented.

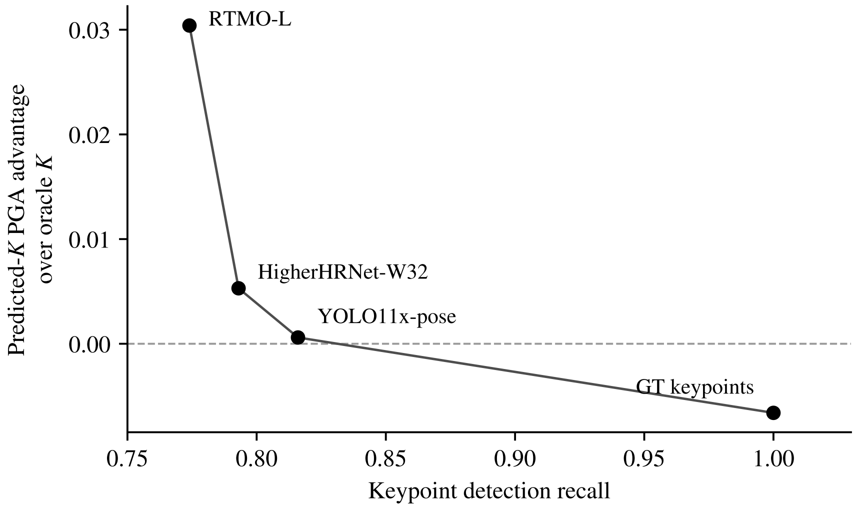

Figure 5.1. Difference in PGA between predicted-K and oracle-K grouping as a function of keypoint recall for the three detector evaluations and the ground-truth-keypoint control. The predicted-K advantage increases across the three detector operating points as recall decreases, while the ground-truth-keypoint control reverses the effect when the input is complete. This pattern is consistent with the effective-person-count explanation - annotation-level K becomes a less appropriate clustering constraint as the detected keypoint set becomes less complete.

The result also clarifies what the K-head's 76.1% exact-count accuracy does and does not measure. That figure evaluates the prediction against the annotated number of people, whereas the downstream clustering problem operates only on the keypoints that survived detection and matching. A prediction counted as incorrect by the former criterion can therefore still be useful to the latter. The K-head is not outperforming an oracle estimate of the effective person count - rather, it can outperform the annotation-level count when that count no longer describes the incomplete input to k-means.

The autonomous result should therefore not be interpreted as evidence that estimated counts are generally preferable to known counts. With complete observations, the ground-truth-keypoint control shows the opposite. The result is specific to the detector-based setting, where the grouping module receives an incomplete representation of the annotated scene. In that setting, the predicted count acts as a form of calibration to the available evidence. This explains why the autonomous pipeline can slightly outperform the oracle-K diagnostic while also removing the final ground-truth dependency from inference.

5.3.3 Synthesis

Reading Sections 5.3.1 and 5.3.2 together gives a more complete picture of the cost and benefit of separating grouping from detection. On HigherHRNet detections, the oracle-K PG-GAT pipeline trails the jointly trained AE reference by 0.0256 PGA. In return, the grouping network is no longer tied to the detector that produced the keypoints. The detector-swap experiments show that the same frozen PG-GAT checkpoint can operate on HigherHRNet, YOLO11x-pose, and RTMO-L without retraining, while remaining approximately 0.02 PGA below each detector's native grouping. This is the experimental answer to *RQ3: detector independence*, detector-independent grouping is achievable, but it carries a small and measurable accuracy cost relative to detector-specific grouping learned as part of the upstream model.

Whether this trade is worthwhile depends on the intended deployment setting. A fixed pipeline in which the detector and grouping mechanism can always be trained and deployed together has little reason to give up the additional accuracy of the native grouping method. The modular design becomes useful when the grouping component must remain reusable across detectors, or when detection and grouping need to be developed, updated, or deployed independently. Under those conditions, the approximately 0.020–0.022 PGA difference measured across the three detectors is the observed cost of obtaining that flexibility.

The autonomous-pipeline result adds a second dimension to this finding. Removing the ground-truth K from the inference path might be expected to introduce another accuracy penalty because errors made by the K-head can propagate directly into k-means. Instead, the predicted-K HigherHRNet configuration reaches 0.9644 PGA - compared with 0.9591 for oracle K. The detector-swap results show the same pattern, with the predicted-K advantage increasing as keypoint recall decreases. The ground-truth-keypoint control reverses the effect when the input is complete. Taken together, these measurements support the effective-person-count explanation developed in Section 5.3.2.

This result changes the role of K estimation in the modular pipeline. The K-head is not simply an imperfect replacement for information that would ideally be supplied by an oracle. In the detector-based setting, the annotation-level person count and the number of people represented by the available keypoints can differ. The K-head operates on the detected representation and can therefore adapt its prediction to the evidence that actually reaches the grouping stage. Its prediction can be wrong relative to the annotated person count while still providing a better clustering constraint for the incomplete detection set.

This provides the experimental answer to *RQ7: autonomous deployment*. The final pipeline does not require ground-truth person counts at inference, and removing that dependency does not reduce the measured grouping accuracy on the detector outputs tested here. For HigherHRNet and RTMO-L it improves it measurably, while for YOLO11x-pose the difference is close to neutral. The ground-truth control is equally important to this conclusion - it shows that predicted K is not intrinsically superior to known K, but becomes useful when detection errors create a mismatch between the annotated scene and the evidence presented to the grouping algorithm.

The combined result is therefore stronger than the oracle-K modularity experiment alone. PG-GAT pays a modest accuracy cost to separate grouping from detection, and the detector-swap experiments show that this separation is operational across multiple detector families. The K-head then removes the remaining ground-truth dependency without introducing the expected additional penalty and, under incomplete detection, can partially offset the oracle-K grouping cost. The contribution is consequently not that modularity is free, but that its cost is measured, stable across the detectors tested, and compatible with a fully autonomous inference pipeline.

5.4 RESEARCH QUESTION RECAP

This closing section consolidates the answers to the seven research questions of Section 1.5 for the reader's convenience. The substantive arguments and evidence appear in Sections 5.1–5.3 of this chapter and the corresponding experimental sections of Chapter 4; the table below maps each question to its answer and the section in which the supporting evidence is developed.

Research question	Answer	Evidence
RQ1 — The lever	The binding constraint is the per-keypoint embedding, not the grouping algorithm. Three learned heads and ten modularity variants all fail to outperform plain k-means on the standard embedding, while embedding-side modifications are the only changes whose gains survive transfer to COCO.	§5.1.1, §5.1.2, §5.1.3

continued on next page

Research question	Answer	Evidence
RQ2 — Inductive biases	The three skeleton-aware modifications (type-pair attention, skeleton-relative positional encoding, same-type repulsion heads) each contribute approximately +0.005 in isolation and approximately +0.011 in combination. The architecture sweep over depth and capacity dominates the modifications (+0.0738 over the ablation row). The loss sweep produces no configuration above the defaults.	§5.1.3
RQ3 — Detector independence	The modular grouper reaches 0.9591 E2E PGA against HigherHRNet AE's 0.9847 on the same matched detections, a gap of 0.0256. Applied zero-shot to YOLO11x-pose and RTMO-L detections, the autonomous pipeline trails each detector's native grouping by 0.0210 and 0.0220; the price of detector independence is therefore stable at approximately 0.02 PGA across three detector families.	§5.3.1, §4.9.5
RQ4 — Embedding geometry	The hyperbolic variant matches the Euclidean variant at four-fold smaller embedding dimension under synthetic-only training (H2 supported) but loses to the Euclidean variant by 0.0166 COCO PGA after COCO fine-tuning. Hyperbolic geometry is an efficiency observation at low dimension, not a quality win at the headline operating point. H1 (quality at matched dimension) is unsupported.	§5.2.3

continued on next page

Research question	Answer	Evidence
RQ5 — Graph primitive	The triplet primitive underperforms the joint primitive by 0.0156 synth val PGA at matched training compute. The dominant failure mode is the voting step that converts per-triplet cluster labels into per-joint labels; small per-triplet errors propagate to multiple joint memberships. The negative result is attributed to the triplet-to-joint voting stage rather than to the articulation signal itself.	§5.2.4
RQ6 — Synthetic-to-real transfer	COCO fine-tuning lifts the embedding by approximately +0.07 COCO val PGA at the architecture-sweep winner, the single largest gain measured in the experimental chapter. Synthetic-validation accuracy is preserved through fine-tuning (no catastrophic forgetting at the sweep operating point), and the lift is comparable in magnitude to the entire architecture sweep.	§5.2.2

RQ7 — Autonomous deployment	The K-head trained on the meng-headline COCO embeddings achieves 76.1% exact K-prediction accuracy and 91.9% off-by-one accuracy on COCO val. The resulting autonomous pipeline produces 0.9644 E2E PGA, 0.0053 *above* the oracle-K row at 0.9591. The evidence is consistent with the K prediction calibrating to the effective person count in the matched-detection set, whereas oracle K describes the annotated scene; the advantage is monotone in the recall deficit across four operating points (+0.0006 to +0.0304 as recall falls from 0.816 to 0.774, with a sign flip at perfect recall). The autonomous pipeline is therefore not a compromise relative to oracle K but the operational measurement of the complete system.	§5.3.2, §4.9.5

Table 5.1. Concise mapping of each research question to its answer and the discussion section in which the supporting argument is developed. The substantive interpretation of each result appears in the referenced section; this table is a navigation aid and not the substance of the chapter.

CHAPTER 6 CONCLUSION

This chapter brings together the experimental results of Chapter 4 and the discussion developed in Chapter 5. It evaluates each part of the composite hypothesis introduced in Section 1.4 against the evidence collected during the investigation, before summarising the main contributions of the work and identifying directions for future research.

6.1 VERDICT ON THE HYPOTHESIS

The hypothesis stated in Section 1.4 consists of three separate claims. Each clause is evaluated against the experimental evidence in turn, before the overall verdict on the hypothesis is given at the end of the section.

6.1.1 Clause 1: pose grouping is an embedding-quality problem

Supported: The first clause proposes that pose-grouping performance is constrained primarily by the quality of the per-keypoint embedding rather than by the choice of grouping algorithm applied afterwards. The experiments provide consistent evidence for this claim. Three learned grouping heads (DEC, slot attention, and graph partitioning) were evaluated on the standard GAT backbone under matched training conditions (Section 4.2), followed by ten SA-DMoN variants that explored a separate family of graph-structural grouping objectives (Section 4.3). None produced a configuration that outperformed plain *k*-means once transfer to COCO was considered.

The graph-partitioning result is particularly illustrative. It reaches 0.9714 synthetic-validation PGA, making it the strongest of the learned heads on the synthetic distribution, but its own grouping read-out falls to 0.7514 on COCO. By comparison, the no-depth standard GAT with *k*-means reaches 0.8841 on the same COCO evaluation setting. The SA-DMoN experiments show the same broader pattern from a different direction - changing the grouping objective and its graph construction does not recover the performance lost when the underlying embedding is not sufficiently transferable.

The embedding-side experiments produce the opposite result. The three skeleton-aware PG-GAT modifications of Section 3.8 improve the representation at matched architecture size, the architecture sweep of Section 4.5 raises synthetic-validation PGA from 0.8896 to 0.9634, and subsequent COCO fine-tuning raises COCO val PGA from 0.9010 to 0.9715. These gains are substantially larger than those obtained by changing the grouping head while leaving the basic embedding formulation unchanged.

Taken together with the analysis of Section 5.1, the evidence supports the first clause of the hypothesis. For the pose-grouping setting studied here, the main performance lever is the representation in which the detected keypoints are embedded. Once that representation separates people effectively, a simple k-means read-out is sufficient - increasing the complexity of the grouping stage does not provide a corresponding improvement.

6.1.2 Clause 2: skeleton-aware GAT with two-stage training

Supported, with qualification: The second clause proposes that the modular grouping approach can achieve quality competitive with a jointly trained end-to-end alternative. The architecture and fine-tune sweeps produce the `meng-headline` checkpoint (W&B run `b5noyg7t`), which reaches 0.9715 COCO validation PGA when evaluated on ground-truth keypoints and 0.9591 end-to-end PGA when applied to HigherHRNet detections. On the same matched detections, HigherHRNet's native associative-embedding grouping reaches 0.9847, leaving a difference of 0.0256 PGA.

The modular method therefore does not match or exceed the jointly trained reference on this evaluation. The result instead establishes a relatively small measured cost for separating the grouping stage from the detector. This distinction matters because the two methods provide different capabilities. HigherHRNet's AE representation is trained jointly with its detector and achieves the higher grouping accuracy, whereas PG-GAT can be applied as a separate grouping module to detections produced by different architectures. The detector-swap experiments strengthen this interpretation by showing a similar autonomous gap of approximately 0.02 PGA across HigherHRNet, YOLO11x-pose, and RTMO-L.

The hypothesis described the target as "quality competitive with an end-to-end joint-trained alternative" without defining a numerical equivalence margin in advance. The verdict must therefore remain qualitative rather than being presented as a statistical non-inferiority result. Within that limitation, a difference of 0.0256 against a reference operating at 0.9847 PGA is considered sufficiently small to support the intended claim of competitive grouping quality, particularly in settings where detector independence is itself a requirement.

The qualification is that this conclusion depends on what the deployment setting values. If the objective is solely to maximise PGA with a fixed HigherHRNet detector, the native AE grouping remains the better choice by 0.0256. If grouping must remain independent of the detector, the experiments show that PG-GAT provides that capability while retaining 0.9591 PGA on the same detections. The second clause is therefore supported as a claim of competitive rather than superior performance, with the measured accuracy difference forming part of the modularity trade-off rather than being hidden by the verdict.

6.1.3 Clause 3: autonomous deployment and detector swap

Supported, with the autonomy result stronger than originally expected: The third clause concerns whether the proposed grouping pipeline can operate without ground-truth knowledge of the number of people and can be transferred between keypoint detectors without retraining. Both parts are supported by the experiments.

The autonomous evaluation of Section 4.9 replaces the ground-truth person count with the prediction of the learned K-head. The head achieves 76.1% exact-count accuracy and 91.9% off-by-one accuracy on COCO validation. Despite this imperfect agreement with the annotated person count, the resulting pipeline reaches 0.9644 end-to-end PGA, compared with 0.9591 when k-means is supplied with the ground-truth K. Predicted K therefore improves the measured PGA by 0.0053 rather than introducing the degradation that might normally be expected when an oracle quantity is replaced by an estimate.

As discussed in Section 5.3.2, this result follows from the distinction between the annotated number of people and the number represented by an incomplete detection set. The K-head predicts from the detected embeddings and keypoint count, allowing it to adapt to the evidence that actually reaches the grouping stage. Ground-truth K describes the complete annotated image and can therefore overestimate the appropriate number of clusters when one or more people are poorly represented by the detector. The detector-swap experiments and ground-truth-keypoint control support this interpretation - the predicted-K advantage grows as detection recall decreases and reverses when complete ground-truth keypoints are supplied. The autonomy result is therefore stronger than the original requirement that predicted K should merely preserve acceptable grouping quality.

The detector-independence part of the clause is supported both by the architecture and by direct evaluation. PG-GAT operates on detected keypoints rather than detector-specific intermediate feature maps, so its grouping network has no architectural dependency on the model that produced those keypoints. More importantly, this property is exercised experimentally rather than inferred from the interface alone. The detector-swap evaluation of Section 4.9.5 applies the same frozen `meng-headline` checkpoint and frozen K-head zero-shot to HigherHRNet, YOLO11x-pose, and RTMO-L outputs. No grouping retraining is performed for either of the swapped detectors.

The autonomous PG-GAT pipeline remains close to each detector's native person assignment, the measured gaps are 0.0203 for HigherHRNet, 0.0210 for YOLO11x-pose, and 0.0220 for RTMO-L. The consistency of these values across three substantially different detector families is the stronger result. It shows not only that detector swapping is possible, but that the accuracy cost remains approximately 0.02 PGA across the detectors tested.

The third clause is therefore supported in both respects. The pipeline can estimate the person count from the detected evidence without a ground-truth dependency, and the same frozen grouping components can be used across multiple detector families without retraining. The autonomous component goes slightly beyond the original expectation - predicted K does not merely approximate the oracle-K result under real detector outputs, but can provide a better clustering constraint when detection incompleteness makes the annotation-level person count misaligned with the evidence available to the grouping stage.

6.2 SUMMARY OF CONTRIBUTIONS

The verdict of Section 6.1 brings the experimental evidence together in relation to the composite hypothesis. This section considers the same evidence from the perspective of the contributions made by the work. Each contribution is tied to a specific experimental result from Chapter 4, so that the claimed contributions remain separated from observations that were exploratory or produced negative results.

6.2.1 Methodological contribution: PG-GAT

The principal technical contribution of this work is PG-GAT - the skeleton-aware graph attention network introduced in Section 3.8. PG-GAT modifies the standard GATv2 attention mechanism in three ways that make the structure of the COCO skeleton explicit: type-pair attention bias, skeleton-relative position encoding, and dedicated same-type repulsion heads. These modifications target different aspects of the grouping problem rather than relying on the network to recover the relevant edge structure from node features alone.

The isolation ablation of Section 4.4 shows that the full configuration produces the strongest result under matched architecture and training compute, while the individual margins of the three modifications are small and close to the run-to-run variation of the metric. The architecture is then selected through a 32-run Bayesian sweep over six architectural parameters, which raises synthetic-validation PGA to 0.9634. A separate 24-run loss sweep finds no alternative that improves meaningfully on the inherited contrastive-loss settings, providing experimental support for retaining those defaults. Finally, the COCO fine-tuning procedure is evaluated through a 12-run Bayesian sweep over fine-tuning duration, learning rate, and weight decay.

The resulting `meng-headline` checkpoint reaches 0.9715 COCO validation PGA when grouping ground-truth keypoints and 0.9591 end-to-end PGA when grouping HigherHRNet detections. The contribution is therefore not only the three attention modifications themselves, but a sweep-defended PG-GAT configuration whose architecture, loss settings, and fine-tuning procedure have each been evaluated experimentally rather than selected solely by manual tuning.

6.2.2 Scientific contribution: the embedding is the binding constraint

The principal scientific contribution is the evidence that, for the bottom-up pose-grouping setting studied in this work, the main constraint on grouping performance is the quality of the per-keypoint embedding rather than the complexity of the grouping algorithm applied to it. This conclusion is supported by two independent families of head-side experiments. Three learned grouping heads (DEC, slot attention, and graph partitioning) were evaluated on the standard GAT backbone, followed by ten SA-DMoN configurations that approached grouping through a modularity-based objective. None produced a configuration that outperformed plain k-means on the corresponding embeddings once synthetic-to-real transfer was considered.

The embedding-side experiments show the opposite pattern. Introducing the skeleton-aware PG-GAT modifications and subsequently selecting the network architecture produces improvements substantially larger than those obtained by replacing or modifying the grouping head. The architecture sweep alone raises synthetic-validation PGA from 0.8896 for the matched PG-GAT ablation configuration to 0.9634, while the subsequent COCO fine-tuning raises COCO validation PGA from 0.9010 to 0.9715. Across the experiments, improving the representation therefore has a much larger effect than increasing the sophistication of the downstream grouping procedure.

This finding leads to a practical design rule for this work - keep the final grouping operation simple and direct modelling capacity towards the embedding that precedes it. From Section 3.8 onward, k-means is therefore retained as the read-out while the investigation focuses on the structure, capacity, and domain adaptation of the embedding network. The contribution is both empirical and methodological, the experiments identify where the dominant performance limitation lies and use that evidence to determine where additional model complexity is most effectively spent.

6.2.3 Operational contribution: autonomous deployment without an oracle

The third contribution is the autonomous person-count estimation component that removes the final ground-truth dependency from the grouping pipeline. As evaluated in Section 4.9, a small K-estimation head is trained on top of the frozen `meng-headline` embedding and reaches 76.1% exact-count accuracy and 91.9% off-by-one accuracy on COCO validation. When its predicted person count is supplied to the downstream k-means grouping, the complete pipeline reaches 0.9644 end-to-end PGA, which is 0.0053 above the corresponding oracle-K result of 0.9591.

The significance of this result is not that the K-head estimates the annotation-level person count more accurately than the oracle, but that the two counts describe different quantities once detection is incomplete. Ground-truth K describes all annotated people in the image, whereas the grouping stage only receives the subset represented by the detected keypoints. Supplying the annotation-level count can therefore force k-means to create clusters for people who are poorly represented or absent from the detected set. The K-head instead predicts from the detected embeddings themselves and consequently adapts to the effective number of people represented by the available evidence.

This interpretation is supported by the detector-swap experiments and the ground-truth-keypoint control, where the advantage of predicted K increases as detector recall decreases and reverses when complete ground-truth keypoints are used. The result therefore contributes more than a mechanism for removing an oracle from the inference path - it identifies a distinction between the annotated and effective person counts that becomes relevant whenever a modular grouper operates downstream of an imperfect detector. Operationally, the experiment demonstrates that PG-GAT can run without ground-truth knowledge of K and, under the detector conditions evaluated here, does so without an accuracy penalty relative to the oracle-K configuration.

6.2.4 Documented negative results

Four documented negative results complement the supported contributions and help define the limits of the approaches explored in this work:

- **Hyperbolic embedding.** The hyperbolic PG-GAT variant matches the Euclidean PG-GAT at a four-fold smaller embedding dimension under synthetic-only training, supporting a potential representation-efficiency advantage at low dimension. This advantage does not survive COCO fine-tuning, where the Euclidean configuration achieves the stronger result. Hyperbolic geometry is therefore retained as an efficiency observation rather than a quality improvement at the headline operating point.
- **Triplet-level graph representation.** TriGAT underperforms the matched joint-level PG-GAT configuration under the same training budget. The main limitation is the additional voting stage required to convert independently clustered triplet embeddings back into joint assignments. Errors at the triplet level can propagate to several joints through this operation, offsetting the additional articulation information provided by the triplet representation. The negative result therefore applies to the particular triplet-clustering-and-voting formulation tested here, rather than ruling out articulation features more generally.
- **Visual-feature augmentation.** PG-GAT v2 augments each keypoint with pre-trained MobileNetV2 features, but neither the initial nor extended training schedule exceeds the no-visual-features PG-GAT headline. The visual pipeline also requires the frozen MobileNetV2 backbone, adding approximately 2.2×10^6 parameters at inference. At the operating point reached by the sweep-selected PG-GAT, the additional visual information therefore does not justify its computational and parameter cost.
- **Modularity-driven grouping.** The ten SA-DMoN variants remain within a narrow 0.77–0.79 synthetic-validation PGA band, well below the no-depth baseline. Changes to the spatial bandwidth, null model, spectral weighting, adjacency construction, and type-loss formulation do

not remove this plateau. The experiments indicate that the limitation is structural - modularity is poorly matched to a spatial-proximity kNN graph whose edges primarily encode geometric proximity rather than person identity. The tested null-model modifications do not recover a useful identity-separating signal from this graph.

These negative results are contributions in their own right because they identify not only which alternatives failed, but also where their limitations arise. Chapter 5 develops the corresponding mechanistic explanations, allowing future work to distinguish directions that are unlikely to benefit from further parameter tuning from those that may remain viable if the identified structural limitation is removed.

6.2.5 Infrastructure contribution: reproducibility chain

A further contribution of the work is the reproducibility of the experimental record. Every reported result in Chapter 4 is linked to a Weights & Biases run in the `multiperson-pose-grouping` project. Each run records the training configuration, source-code snapshot, training metrics, and final checkpoint, providing a direct trace from the numbers reported in the thesis to the experiment that produced them.

The four Bayesian sweeps (architecture, loss, hyperbolic configuration, and COCO fine-tuning) are likewise retained under their respective sweep identifiers. This makes it possible to recover both the selected configurations and the alternatives against which they were evaluated. For the final PG-GAT configuration, the principal architecture, loss, and fine-tuning choices are therefore supported by explicit searches over their specified operating ranges rather than being reported solely as inherited or manually selected values.

This experimental record strengthens the interpretation of the results in two ways. First, it makes the progression from the baseline through the ablations and sweeps to the final `meng-headline` checkpoint traceable. Second, it distinguishes hyperparameters that were selected because they produced the strongest measured result from those retained because the corresponding sweep found no meaningful improvement over the existing setting. The contribution is therefore not a new modelling mechanism, but a reproducible and auditable experimental basis for the technical conclusions drawn in this work.

6.3 FUTURE WORK

The supported hypothesis and the negative results reported in this work leave several questions open. Five directions follow directly from the experimental findings, each addressing a limitation or unresolved mechanism identified during the investigation.

6.3.1 Constraint-aware grouping read-outs

The experiments in this work use unconstrained k-means as the final grouping read-out. This keeps the grouping stage simple and makes the effect of changes to the embedding easier to isolate, but it does not use all of the structure available in the COCO skeleton. In particular, a valid person instance cannot contain two keypoints of the same joint type. Standard k-means has no mechanism for enforcing this type-exclusivity constraint.

A constraint-aware read-out could impose this rule directly during inference while retaining the same learned embedding. Such a method would not require the PG-GAT network to be retrained and could therefore be evaluated as a direct replacement for k-means. This would also provide a useful continuation of the learned-head experiments - rather than increasing the capacity of the grouping stage, the read-out would remain simple but make use of a known structural constraint. Measuring how much of the remaining grouping error can be recovered in this way is a natural extension of the present work.

6.3.2 Fully hyperbolic architecture

The hyperbolic PG-GAT evaluated in Section 4.7 is a hybrid architecture. Message passing remains Euclidean and only the final representation is projected onto the Lorentz hyperboloid. Under this design, H1 (improved or matched quality at the same embedding dimension) is not supported. H2 shows a more promising result at low dimension, where the hyperbolic model matches the larger Euclidean representation under synthetic-only training, but this advantage does not survive COCO fine-tuning.

These results do not completely resolve the geometric-prior question because most of the network still operates in Euclidean space. A fully hyperbolic PG-GAT would instead perform the relevant transformations and aggregation operations directly in hyperbolic space. Possible components include Möbius operations for the learnable transformations, Fréchet-mean aggregation during message passing, and attention mechanisms defined consistently with the chosen hyperbolic geometry.

Such an architecture would require substantially more implementation and numerical-stability work than the hybrid model tested here. The hybrid design was therefore a reasonable first test of whether the additional complexity was likely to be justified. Its negative H1 result does not rule out a fully hyperbolic model, however, because the Euclidean message-passing layers remain a confounding factor. A fully hyperbolic implementation would provide a cleaner test of whether the skeleton's tree structure offers a useful geometric prior for pose grouping.

6.3.3 End-to-end joint training with the detector

The headline modular pipeline trails HigherHRNet's jointly trained AE grouping by 0.0256 PGA on the same matched detections. The present work interprets this difference as the measured cost of detector independence, but the experiments do not determine how much of the gap follows from the absence of joint training and how much follows from differences between the grouping mechanisms themselves.

Jointly training PG-GAT with a keypoint detector would separate these effects. The experiment would deliberately give up the detector-swap property established in this work but would provide a controlled measurement of the benefit obtained from detector-specific feature adaptation. If joint training closes the gap to AE, the result would show that the remaining difference is largely the price paid for modularity. If a substantial gap remains, this would suggest that the native AE formulation benefits from properties that are not recovered simply by exposing PG-GAT to the detector during training.

This experiment would therefore quantify the accuracy–modularity trade-off more directly and help identify the source of the remaining end-to-end difference.

6.3.4 Detector-recall sensitivity of autonomous K estimation

The detector-swap experiments of Section 4.9.5 provide four operating points for the relationship between detection recall and the advantage of predicted K over oracle K. Across the three detectors, the predicted-K advantage increases as recall decreases, while the ground-truth-keypoint control reverses the sign when recall is effectively complete. This supports the effective-person-count explanation developed in Section 5.3.2.

The existing comparison cannot isolate recall completely, however, because changing the detector also changes localisation noise, duplicate detections, confidence calibration, training data, and the distribution of missed joints. A controlled experiment could vary recall while holding the detector fixed, for example by changing its detection threshold or by systematically removing detections from a fixed output set. The resulting experiment would trace predicted-K versus oracle-K performance across a continuous range of recall values while removing the main between-detector confounding factors.

The current four operating points establish a credible relationship. A controlled within-detector sweep would determine its shape and could identify the recall range at which predicting the effective person count becomes preferable to using the annotation-level count.

6.3.5 Multi-seed variance characterisation

The experiments reported in Chapter 4 use a single training seed per configuration. This is most relevant for the synthetic experiments, where validation is performed on only 100 scenes and the smallest reported differences are approximately 0.005 PGA. Differences of this size should therefore be interpreted cautiously, as already done for the individual PG-GAT ablation rows. The much larger COCO validation set reduces evaluation variance but does not remove variation introduced by training initialisation.

A useful follow-on would repeat the main synthetic experiments over three to five seeds and report the resulting mean and variance. The PG-GAT isolation ablation of Section 4.4 is the highest-priority case, because its individual modification gains are small. Repeating selected SA-DMoN configurations would similarly provide a tighter empirical bound on the 0.77–0.79 plateau and distinguish training-seed variation from the already small differences between variants.

The same experiment could independently vary the graph neighbourhood size k. In the present implementation, k is coupled to embedding dimension ($k = 8$ below dimension 256 and $k = 16$ at dimension 256) rather than treated as an independent hyperparameter (Section 3.3.2). Separating these variables would show whether part of the architecture-sweep gain attributed to embedding capacity is associated with the corresponding change in graph connectivity.

A three- to five-seed replication of the synthetic experiments would require only a fraction of the compute used for the COCO fine-tuning investigation and would provide a stronger statistical basis for a publication-oriented version of the ablation and SA-DMoN results.

REFERENCES

[1] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

[2] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5686–5696.

[3] B. Cheng, B. Xiao, J. Wang, H. Shi, T. S. Huang, and L. Zhang, "HigherHRNet: Scale-aware representation learning for bottom-up human pose estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5385–5394.

[4] Y. Xu, J. Zhang, Q. Zhang, and D. Tao, "ViTPose: Simple vision transformer baselines for human pose estimation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 38 571–38 584.

[5] R. Khirodkar *et al.*, "Sapiens: Foundation for human vision models," in *European Conference on Computer Vision (ECCV)*, 2024, pp. 206–228.

[6] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime multi-person 2D pose estimation using part affinity fields," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1302–1310.

[7] A. Newell, Z. Huang, and J. Deng, "Associative embedding: End-to-end learning for joint detection and grouping," in *Advances in Neural Information Processing Systems (NeurIPS)*,

vol. 30, 2017.

[8] Z. Geng, K. Sun, B. Xiao, Z. Zhang, and J. Wang, "Bottom-up human pose estimation via disentangled keypoint regression," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 671–14 681.

[9] D. Shi, X. Wei, L. Li, Y. Ren, and W. Tan, "End-to-end multi-person pose estimation with transformers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 059–11 068.

[10] S. Kreiss, L. Bertoni, and A. Alahi, "PifPaf: Composite fields for human pose estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 11 969–11 978.

[11] G. Brasó, N. Kister, and L. Leal-Taixé, "The center of attention: Center-keypoint grouping via attention for multi-person pose estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 11 833–11 843.

[12] J. Yang, A. Zeng, S. Liu, F. Li, R. Zhang, and L. Zhang, "Explicit box detection unifies end-to-end multi-person pose estimation," in *International Conference on Learning Representations (ICLR)*, 2023.

[13] S. Jin *et al.*, "Differentiable hierarchical graph grouping for multi-person pose estimation," in *European Conference on Computer Vision (ECCV)*, 2020, pp. 718–734.

[14] M. Maher, R. Fathalla, and M. Shaheen, "Graph edge classification for keypoint grouping in multi-person pose estimation," *Journal of Image and Graphics*, vol. 13, no. 2, pp. 130–139, 2025.

[15] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations (ICLR)*, 2022.

[16] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[17] F. Locatello *et al.*, "Object-centric learning with slot attention," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11 525–11 538.

[18] L. Yang, D. Chen, X. Zhan, R. Zhao, C. C. Loy, and D. Lin, "Learning to cluster faces via confidence and connectivity estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 13 366–13 375.

[19] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 127, pp. 1–21, 2023.

[20] G. Jocher and J. Qiu, "Ultralytics YOLO11," https://github.com/ultralytics/ultralytics, 2024, software, version 11.0.0.

[21] P. Lu, T. Jiang, Y. Li, X. Li, K. Chen, and W. Yang, "RTMO: Towards high-performance one-stage real-time multi-person pose estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 1491–1500.

[22] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[23] Y. Chen, Y. Tian, and M. He, "Monocular human pose estimation: A survey of deep learning-based methods," *Computer Vision and Image Understanding*, vol. 192, p. 102897, 2020.

[24] C. Zheng *et al.*, "Deep learning-based human pose estimation: A survey," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–37, 2023.

[25] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, "2D human pose estimation: New benchmark and state of the art analysis," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 3686–3693.

[26] A. Toshev and C. Szegedy, "DeepPose: Human pose estimation via deep neural networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1653–1660.

[27] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, “Convolutional pose machines,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4724–4732.

[28] A. Newell, K. Yang, and J. Deng, “Stacked hourglass networks for human pose estimation,” in *European Conference on Computer Vision (ECCV)*, 2016, pp. 483–499.

[29] B. Xiao, H. Wu, and Y. Wei, “Simple baselines for human pose estimation and tracking,” in *European Conference on Computer Vision (ECCV)*, 2018, pp. 472–487.

[30] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[31] H.-S. Fang, S. Xie, Y.-W. Tai, and C. Lu, “RMPE: Regional multi-person pose estimation,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2353–2362.

[32] Y. Li *et al.*, “TokenPose: Learning keypoint tokens for human pose estimation,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 11 293–11 302.

[33] Y. Yuan *et al.*, “HRFormer: High-resolution transformer for dense prediction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 7281–7293.

[34] Y. Xu, J. Zhang, Q. Zhang, and D. Tao, “ViTPose++: Vision transformer for generic body pose estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 2, pp. 1212–1230, 2024.

[35] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 213–229.

[36] Y. Xiao *et al.*, “QueryPose: Sparse multi-person pose regression via spatial-aware part-level query,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 12 464–12 477.

[37] H. Liu *et al.*, "Group Pose: A simple baseline for end-to-end multi-person pose estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 14 983–14 992.

[38] X. Sun, B. Xiao, F. Wei, S. Liang, and Y. Wei, "Integral human pose regression," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 536–553.

[39] A. Nibali, Z. He, S. Morgan, and L. Prendergast, "Numerical coordinate regression with convolutional neural networks," *arXiv preprint arXiv:1801.07372*, 2018.

[40] J. Li *et al.*, "Human pose regression with residual log-likelihood estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 11 005–11 014.

[41] Y. Li *et al.*, "SimCC: A simple coordinate classification perspective for human pose estimation," in *European Conference on Computer Vision (ECCV)*, 2022, pp. 89–106.

[42] T. Jiang *et al.*, "RTMPose: Real-time multi-person pose estimation based on MMPose," *arXiv preprint arXiv:2303.07399*, 2023.

[43] T. Jiang, X. Xie, and Y. Li, "RTMW: Real-time multi-person 2D and 3D whole-body pose estimation," *arXiv preprint arXiv:2407.08634*, 2024.

[44] J. Yang, A. Zeng, R. Zhang, and L. Zhang, "X-Pose: Detecting any keypoints," in *European Conference on Computer Vision (ECCV)*, 2024, pp. 249–268.

[45] Z. Geng, C. Wang, Y. Wei, Z. Liu, H. Li, and H. Hu, "Human pose as compositional tokens," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 660–671.

[46] J. Li, C. Wang, H. Zhu, Y. Mao, H.-S. Fang, and C. Lu, "CrowdPose: Efficient crowded scenes pose estimation and a new benchmark," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10 855–10 864.

[47] S.-H. Zhang *et al.*, "Pose2Seg: Detection free human instance segmentation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 889–898.

[48] Y. Xiu, J. Li, H. Wang, Y. Fang, and C. Lu, "Pose Flow: Efficient online pose tracking," in *British Machine Vision Conference (BMVC)*, 2018, p. 53.

[49] G. Ning, J. Pei, and H. Huang, "LightTrack: A generic framework for online top-down human pose tracking," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 4456–4465.

[50] M. Andriluka *et al.*, "PoseTrack: A benchmark for human pose estimation and tracking," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 5167–5176.

[51] S. Yang *et al.*, "Detecting and grouping keypoints for multi-person pose estimation using instance-aware attention," *Pattern Recognition*, vol. 136, p. 109232, 2023.

[52] G. Papandreou, T. Zhu, L.-C. Chen, S. Gidaris, J. Tompson, and K. Murphy, "PersonLab: Person pose estimation and instance segmentation with a bottom-up, part-based, geometric embedding model," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 282–299.

[53] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *AAAI Conference on Artificial Intelligence (AAAI)*, 2018, pp. 7444–7452.

[54] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 018–12 027.

[55] L. Zhao, X. Peng, Y. Tian, M. Kapadia, and D. N. Metaxas, "Semantic graph convolutional networks for 3D human pose regression," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3420–3430.

[56] H. Ci, C. Wang, X. Ma, and Y. Wang, "Optimizing network structure for 3D human pose estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 2262–2271.

[57] J. Liu *et al.*, “A graph attention spatio-temporal convolutional network for 3D human pose estimation in video,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 3374–3380.

[58] B. Doosti, S. Naha, M. Mirbagheri, and D. J. Crandall, “HOPE-Net: A graph-based model for hand-object pose estimation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6607–6616.

[59] M. E. J. Newman, “Modularity and community structure in networks,” *Proceedings of the National Academy of Sciences*, vol. 103, no. 23, pp. 8577–8582, 2006.

[60] J. Shi and J. Malik, “Normalized cuts and image segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000.

[61] U. von Luxburg, “A tutorial on spectral clustering,” *Statistics and Computing*, vol. 17, no. 4, pp. 395–416, 2007.

[62] L. Yang, X. Zhan, D. Chen, J. Yan, C. C. Loy, and D. Lin, “Learning to cluster faces on an affinity graph,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2293–2301.

[63] R. Hadsell, S. Chopra, and Y. LeCun, “Dimensionality reduction by learning an invariant mapping,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 2, 2006, pp. 1735–1742.

[64] S. Chopra, R. Hadsell, and Y. LeCun, “Learning a similarity metric discriminatively, with application to face verification,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2005, pp. 539–546.

[65] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[66] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International Conference on Machine Learning (ICML)*, 2020, pp. 1597–1607.

[67] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.

[68] R. Sarkar, “Low distortion Delaunay embedding of trees in hyperbolic plane,” in *International Symposium on Graph Drawing (GD)*, 2011, pp. 355–366.

[69] F. Sala, C. De Sa, A. Gu, and C. Ré, “Representation tradeoffs for hyperbolic embeddings,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 4460–4469.

[70] M. Nickel and D. Kiela, “Learning continuous hierarchies in the Lorentz model of hyperbolic geometry,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 3779–3788.

[71] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *International Conference on Machine Learning (ICML)*, 2017, pp. 1263–1272.

[72] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[73] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.

[74] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.

[75] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.

[76] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[77] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.

[78] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008.

[79] K. Cho *et al.*, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[80] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[81] Blender Online Community, “Blender — a 3D modelling and rendering package,” http://www.blender.org, 2024, software.

[82] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.

[83] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[84] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.

[85] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations (ICLR)*, 2017.

[86] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[87] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018.

[88] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[89] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[90] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[91] M. Kochurov, R. Karimov, and S. Kozlukov, "Geoopt: Riemannian optimization in PyTorch," *arXiv preprint arXiv:2005.02819*, 2020.

[92] L. Biewald, "Experiment tracking with Weights and Biases," https://www.wandb.com/, 2020, software.

APPENDIX A REPRODUCIBILITY: RUN AND SWEEP IDENTIFIERS

Every experimental result in this document traces to a logged run in the Weights & Biases project deanmills/multiperson-pose-grouping. Each training run records its full configuration, source-code state, training curves, and checkpoint; evaluation results are attached to the source run's summary under eval/* prefixes (Section 3.13.2). This appendix lists the identifier for every run and sweep cited in the body of the document. The run identifier doubles as the on-disk output directory name under outputs/<config>/<run>/.

A.1 BAYESIAN SWEEPS

Sweep	Sweep ID	Runs	Winner (run)
Architecture	wg95w0k0	32	champion-arch (08d0ffde)
Loss / regularisation	oyt1isr0	24	champion-loss (mkmtc8om)
Hyperbolic dimension	qpggbsdn	8	champion-hyperbolic (ua4wdb3p)
Fine-tune recipe	i9vpg4t1	12	meng-headline (b5noyg7t)

Table A.1. The four Bayesian sweeps that defend every PG-GAT hyperparameter, with their W&B sweep identifiers and winning runs.

A.2 TRAINING RUNS

Table A.2. W&B run identifiers for every training run cited in Chapter 4.

Result	Section	Configuration	W&B run
Baseline GAT, depth on	4.1	train_knn_only	su8rmvti
Baseline GAT, depth off	4.1	train_knn_no_depth	jkd1ln4a
DEC head	4.2	train_dec	5q0p9it9

Table A.2. (continued) W&B run identifiers for every training run cited in Chapter 4.

Result	Section	Configuration	W&B run
Slot attention head	4.2	`train_slot`	`2kq3i8yv`
Graph partitioning head	4.2	`train_partition`	`s3qr04xm`
Vanilla DMoN (Exp 3-bis)	4.3	`train_dmon_no_depth`	`124ykt2c`
SA-DMoN v1 (Exp 4)	4.3	`train_sa_dmon_no_depth`	`69yfqshy`
SA-DMoN v1 clamped (Exp 5)	4.3	`..._clamped`	`rvd7qaow`
SA-DMoN v1 λ=1 (Exp 6)	4.3	`..._lambda1`	`2ok6u2k3`
SA-DMoN v1 fixed σ (Exp 7)	4.3	`..._fixed_sigma`	`usjwc380`
SA-DMoN v1 no spectral (Exp 8)	4.3	`..._no_spectral`	`2ucbx3e0`
SA-DMoN v2a	4.3	`..._v2_feat_only`	`x6v3wfnv`
SA-DMoN v2b	4.3	`..._v2_feat_only_low_lambda`	`qn7rv98j`
SA-DMoN v2c	4.3	`..._v2_entropy_only`	`zz2sj7an`
SA-DMoN v2d	4.3	`..._v2_full`	`02jqs33r`
PG-GAT type-pair only	4.4	`train_sa_gat_type_pair`	`8peqkcyh`
PG-GAT pos-enc only	4.4	`train_sa_gat_pos_enc`	`ffl2tde3`
PG-GAT repulsion only	4.4	`train_sa_gat_repulsion`	`gfqkbqnw`
PG-GAT full (all three)	4.4	`train_sa_gat_full`	`r5ekg0zl`
Legacy COCO fine-tune	4.6	`finetune_sa_gat_coco`	`ugzbgepn`
Architecture-sweep winner	4.5	`pg_gat_arch_sweep`	`08d0ffde`
Loss-sweep winner	4.5	`pg_gat_loss_sweep`	`mkmtc8om`
Fine-tune-sweep winner (headline)	4.6	`pg_gat_finetune_sweep`	`b5noyg7t`
Hyperbolic H1 (dim 128)	4.7	`train_sa_gat_hyperbolic`	`aox7h3di`
Hyperbolic H2 (dim 32)	4.7	`..._hyperbolic_dim32`	`g45tdxi5`
Hyperbolic Option B (COCO ft)	4.7	`finetune_..._dim32_coco`	`lragtfzu`
Hyperbolic-sweep winner	4.7	`pg_gat_hyperbolic_sweep`	`ua4wdb3p`
TriGAT	4.8	`train_trigat`	`y0tw6vc9`
PG-GAT v2, 20 epochs	4.11	`train_sa_gat_v2_features`	`vg43ee4u`
PG-GAT v2, 50 epochs extended	4.11	`..._v2_features_extended`	`o8978au7`

A.3 EVALUATION ATTACHMENTS

Evaluation metrics are attached to the source training run's W&B summary rather than logged as separate runs. The headline run `b5noyg7t` carries the following evaluation prefixes: `eval/e2e` (HigherHRNet end-to-end, oracle and predicted *K*), `eval/e2e_yolo` and `eval/e2e_rtmo` (the detector-swap evaluations of Section 4.9.5), `eval/k_estimation/coco` (K-head accuracy on ground-truth keypoints), and `eval/knn/coco_multi` (the multi-person diagnostic subset). Every other run listed above carries evaluation summary fields from the corresponding COCO evaluation pass. Metric keys retain the codebase's historical field names, in which the *k*-means read-out is recorded as `knn` (for example `knn_pga`); every `knn`-named field therefore reports the *k*-means read-out of Section 3.3.3. The K-head checkpoint used by the autonomous pipeline was trained before the W&B integration and is preserved on disk under `outputs/k_head_meng_headline_coco/a636b1c4/`; its evaluation results are attached to `b5noyg7t` as listed above.